

Generative Models 1

Introduction & probabilistic PCA

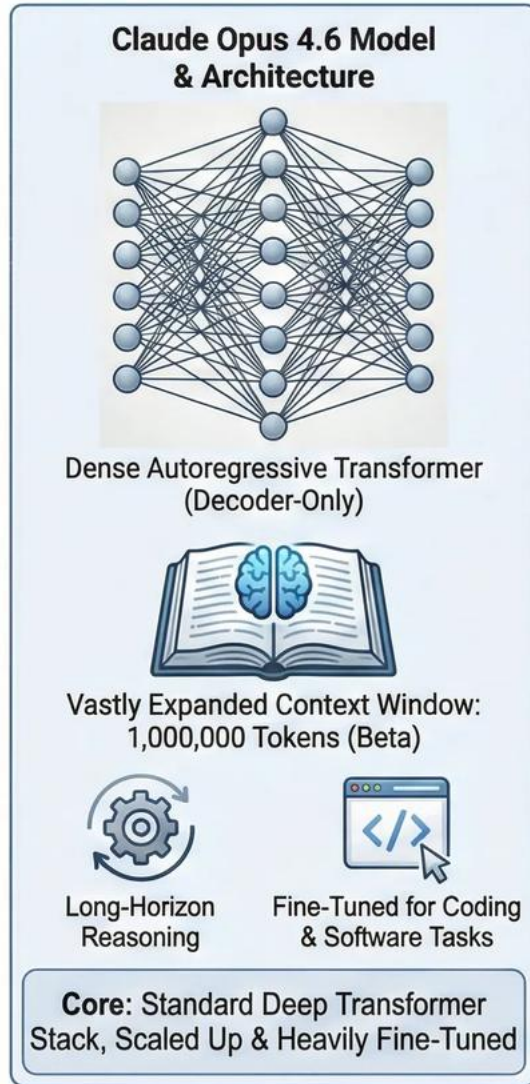
Stavros Makrodimitris

s.makrodimitris@uva.nl

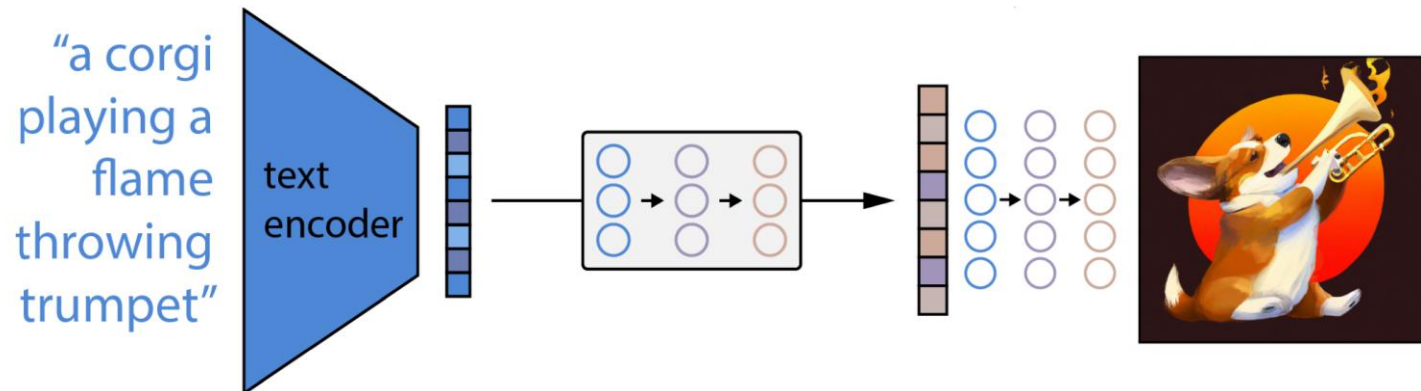


UNIVERSITY OF AMSTERDAM
Swammerdam Institute for Life Sciences

Generative models are everywhere



thispersondoesnotexist.com



Generative models in bioinformatics

Diffusion model

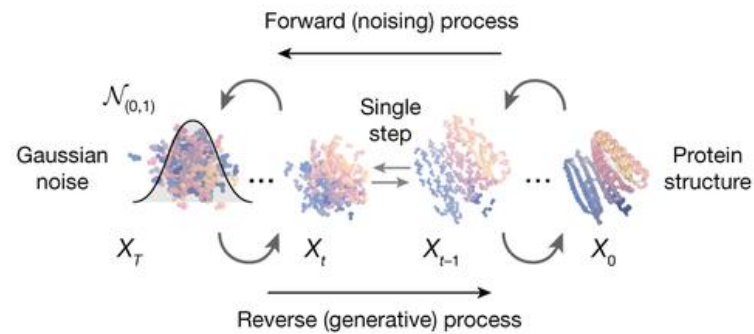
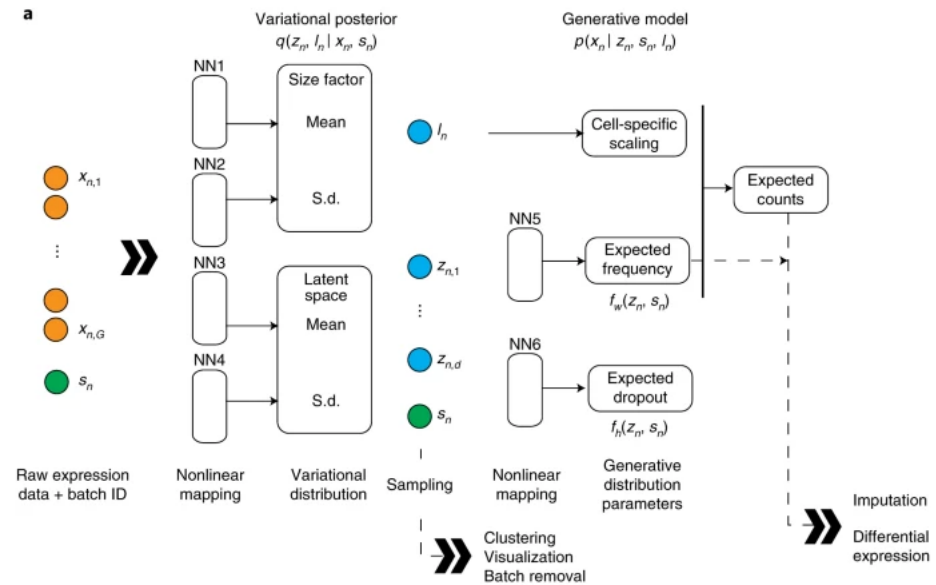
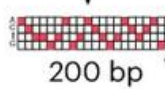


Fig. 1: Overview of scVI.



...ACTGTCAGTCGATCGATGCGAT...

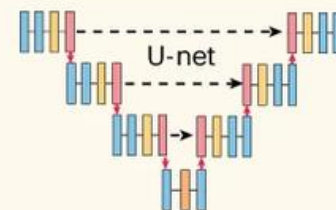


DNA sequence/noise

Timestep

Cell type label

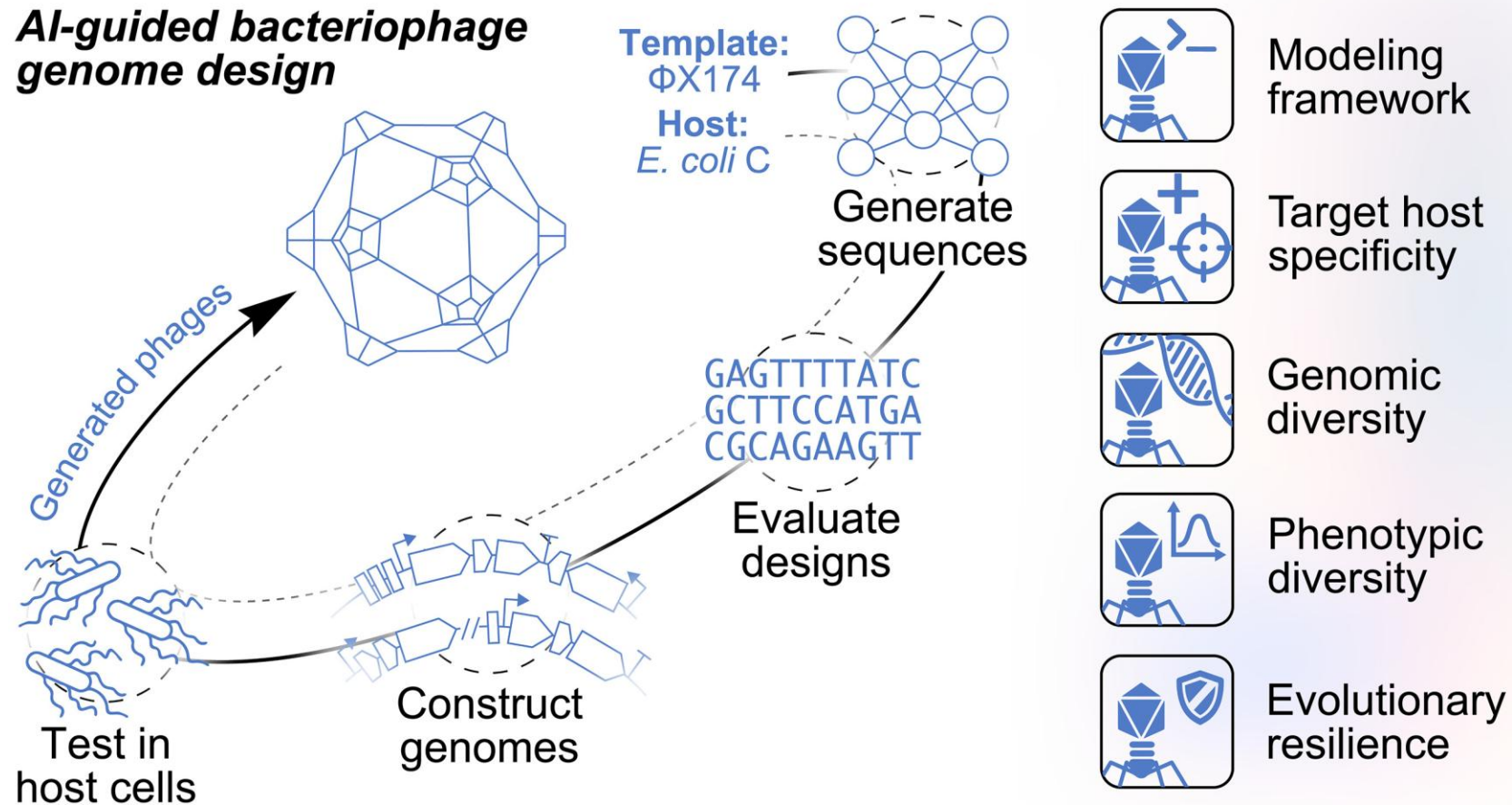
DNA-Diffusion overview



New cell-type-specific DHS site



Generate entire genome!



Learning goals for today

- Understand the theory behind common generative models
- Have intuition about how generative models work
- Explain connections among different GMs

Overview

start	end	topic
09:00	09:15	introduction
09:15	09:30	probabilistic PCA
09:30	09:45	
09:45	10:00	BREAK
10:00	10:15	VAE
10:15	10:30	
10:30	10:45	applications, HVAE
10:45	11:00	BREAK
11:00	11:15	Diffusion models
11:15	11:30	
11:30	11:45	applications
11:45	12:00	BREAK
12:00	12:15	Normalizing flows
12:15	12:30	other
12:30	13:30	LUNCH
13:30	~16:30	HANDS-ON Protein design w/ generative models

Learning goals for session 1

- Recognize what a generative model is
- Use and interpret probabilistic PCA
- Understand maximum likelihood training of PPCA

But what is a generative model?

Generative models ...

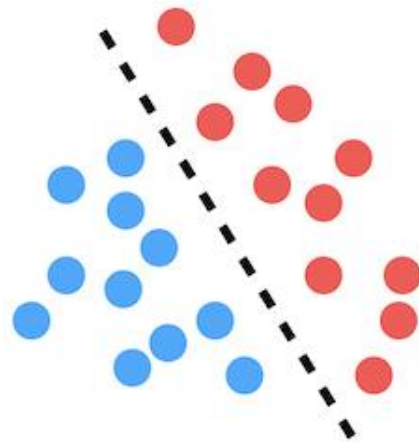
- Model the entire data distribution
- Tell us how likely a data point is
- Can generate new data

Supervised learning/Classification

Discriminative – $p(y|x)$

- Logistic regression

$$p(y|x) = \frac{1}{1 + e^{-Wx}}$$



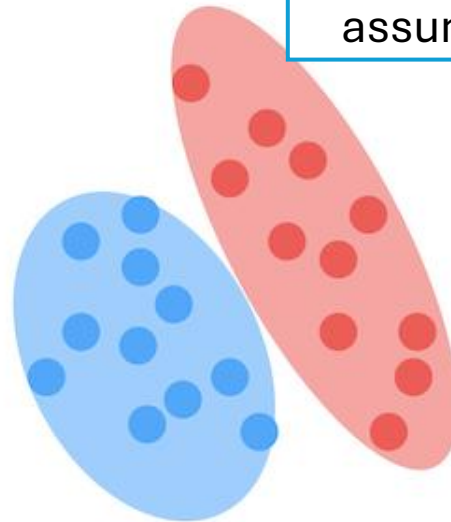
Generative – $p(x,y)$

- Linear discriminant analysis

$$p(x,y) = p(y)p(x|y)$$

Known or
assumed 50-50

Gaussian with
shared covariance



Generative vs Discriminative supervised models

Discriminative - $p(y x)$	Generative - $p(x,y)$
Linear/Logistic regression	Nearest Mean Classifier
Decision Tree	Linear Discriminant Analysis
Random Forest	Quadratic Discriminant Analysis
SVM	Naïve Bayes classifier
k-NN	
MLP	

Generative vs Discriminative supervised models

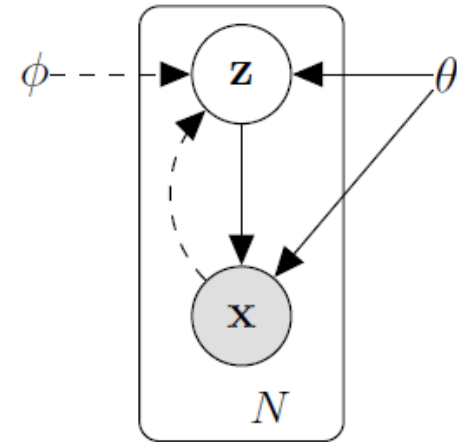
- Generative classification models make strong assumptions
 - Gaussian classes
 - Independent features
- They are less flexible, with higher error
- But can perform better at low sample sizes

GMs for unsupervised learning

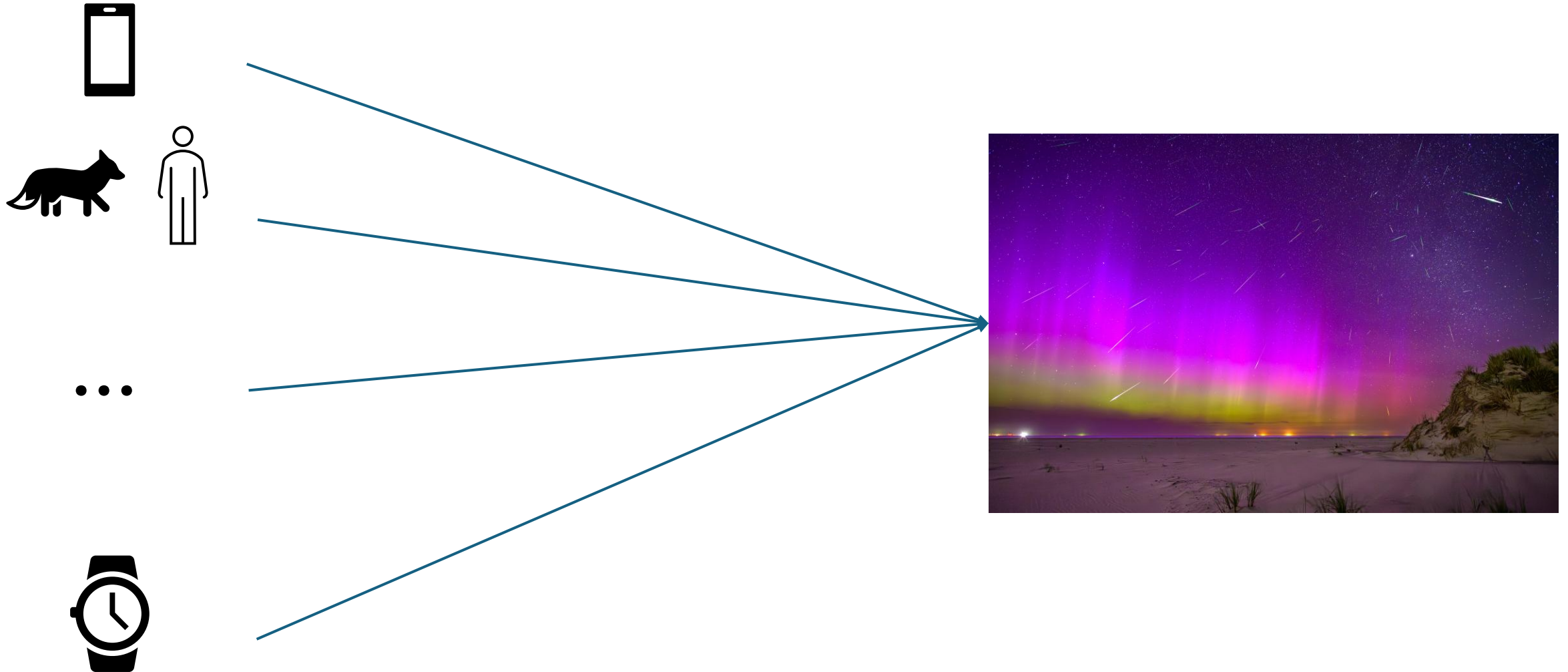
- No labels (y)
- GMs learn the data distribution $p(\mathbf{x})$
- Often using hidden variables $\mathbf{z}$

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z}) d\mathbf{z}$$

- **$\mathbf{z}$** : underlying factors determining $\mathbf{x}$
 - GMs learn the data generating process



Example: generative process for photographs



Let's play.. Is it generative?

PCA and probabilistic PCA?

$$\max_{\mathbf{u}} \mathbf{u}^T \mathbf{S} \mathbf{u} \text{ subject to } \|\mathbf{u}\| = 1$$

$$\max \int \mathcal{N}(\mathbf{x} | \mathbf{W} \mathbf{z} + \boldsymbol{\mu}, \sigma^2 \mathbf{I}) \cdot \mathcal{N}(\mathbf{z} | \mathbf{0}, \mathbf{I}) d\mathbf{z}$$

Not generative - $p(\mathbf{z} \mathbf{x})$	Generative - $p(\mathbf{x},\mathbf{z})$
PCA	probabilistic PCA

Auto-encoder and Variational Auto-encoder?

Not generative - $p(z x)$	Generative - $p(x,z)$
PCA	probabilistic PCA
auto-encoder	variational auto-encoder

GAN?

$$\min_G \max_D V(D, G)$$

$$V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Not generative - $p(z x)$	Generative - $p(x,z)$
PCA	probabilistic PCA
auto-encoder	variational auto-encoder
	GAN

UMAP?

min distances to k-NN
max distances to other points

Not generative - $p(z x)$	Generative - $p(x,z)$
PCA	probabilistic PCA
auto-encoder	variational auto-encoder
UMAP	GAN

t-SNE?

$$\text{KL} (P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

p_{ij} : probability of x_i being NN of x_j

q_{ij} : probability of z_i being NN of z_j

Not generative - $p(z x)$	Generative - $p(x,z)$
PCA	probabilistic PCA
auto-encoder	variational auto-encoder
UMAP	GAN
t-SNE	

Normalizing flows?

$$p_X(\mathbf{x}) = p_Z(f(\mathbf{x})) \left| \det\left(\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}\right) \right|$$

Not generative - $p(z x)$	Generative - $p(x,z)$
PCA	probabilistic PCA
auto-encoder	variational auto-encoder
UMAP	GAN
t-SNE	normalizing flows

Diffusion models?

Not generative - $p(z x)$	Generative - $p(x,z)$
PCA	probabilistic PCA
auto-encoder	variational auto-encoder
UMAP	GAN
t-SNE	normalizing flows
	diffusion

Hidden Markov Model?

$$p(\mathbf{X}) = \sum_{\mathbf{z}} [p(z_1) \prod p(z_n | z_{n-1}) \prod p(\mathbf{x}_m | z_m)]$$

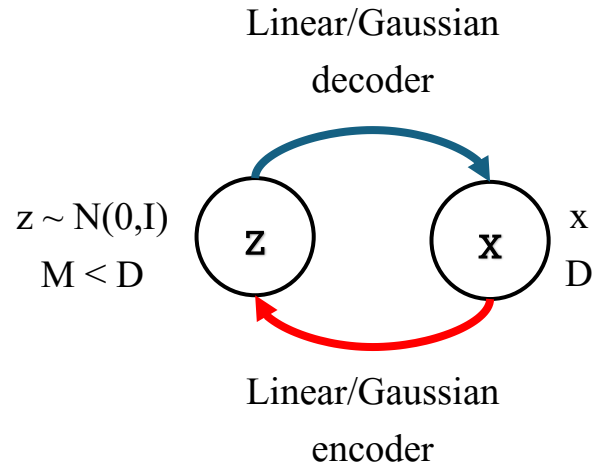
Not generative - $p(\mathbf{z} \mathbf{x})$	Generative - $p(\mathbf{x},\mathbf{z})$
PCA	probabilistic PCA
auto-encoder	variational auto-encoder
UMAP	GAN
t-SNE	normalizing flows
	diffusion
	Hidden Markov Model

Types of generative models

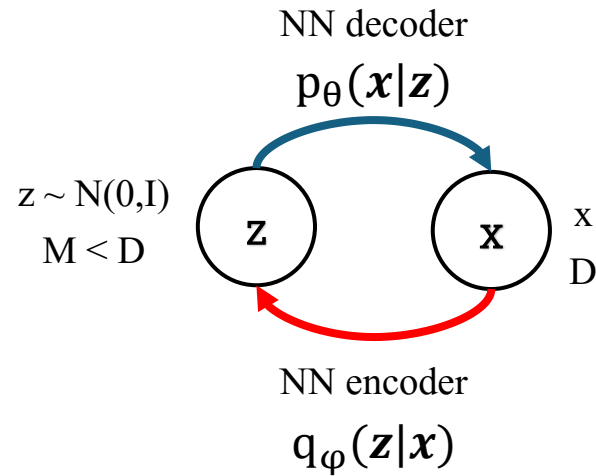
- Latent variables
 - VAE
- Autoregressive
 - sequential data
- Diffusion
 - high-quality generation
- Flow-based
- Energy-based

Types of generative models

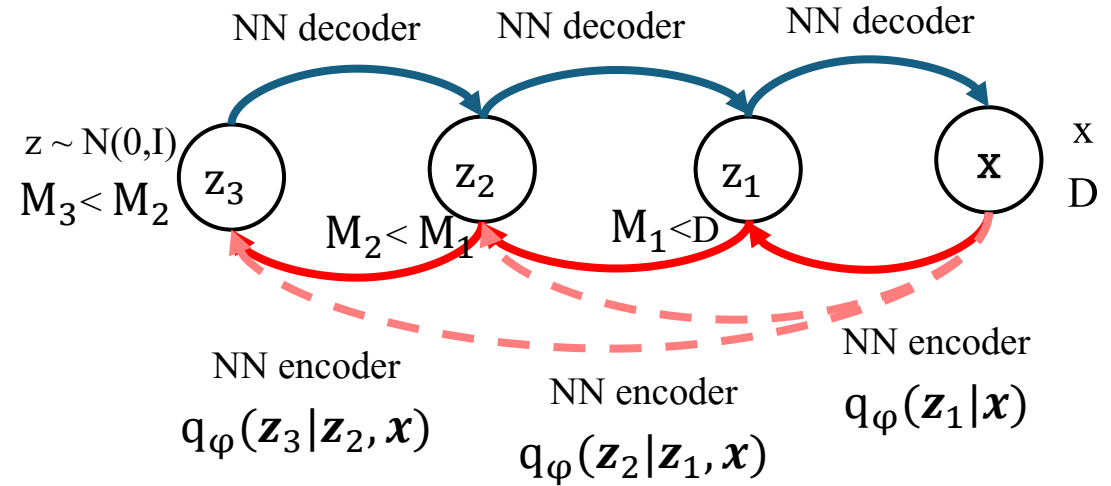
probabilistic PCA



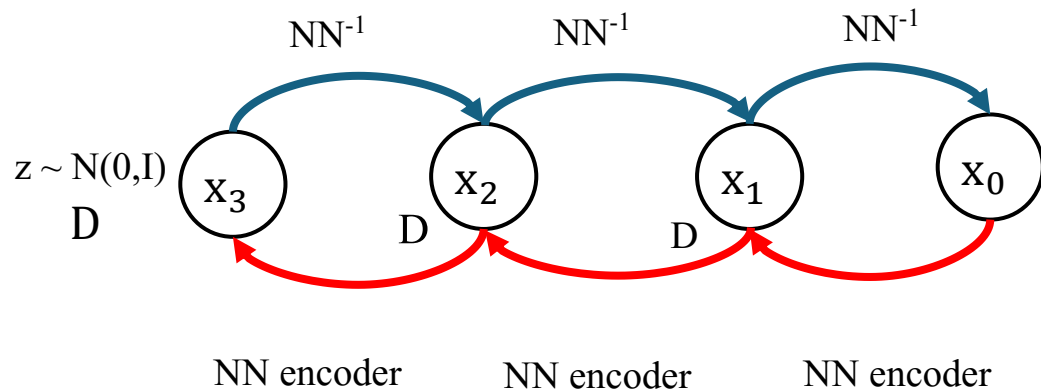
VAE



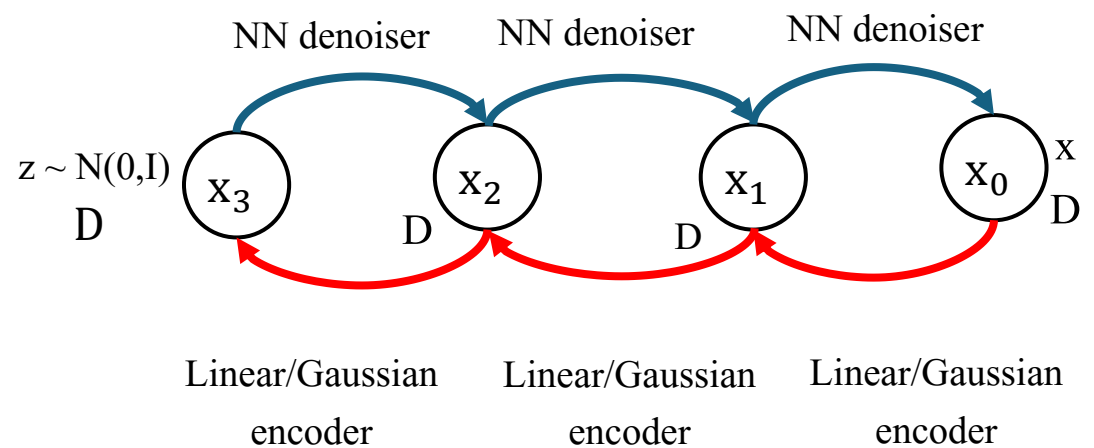
HVAE



Normalizing Flows



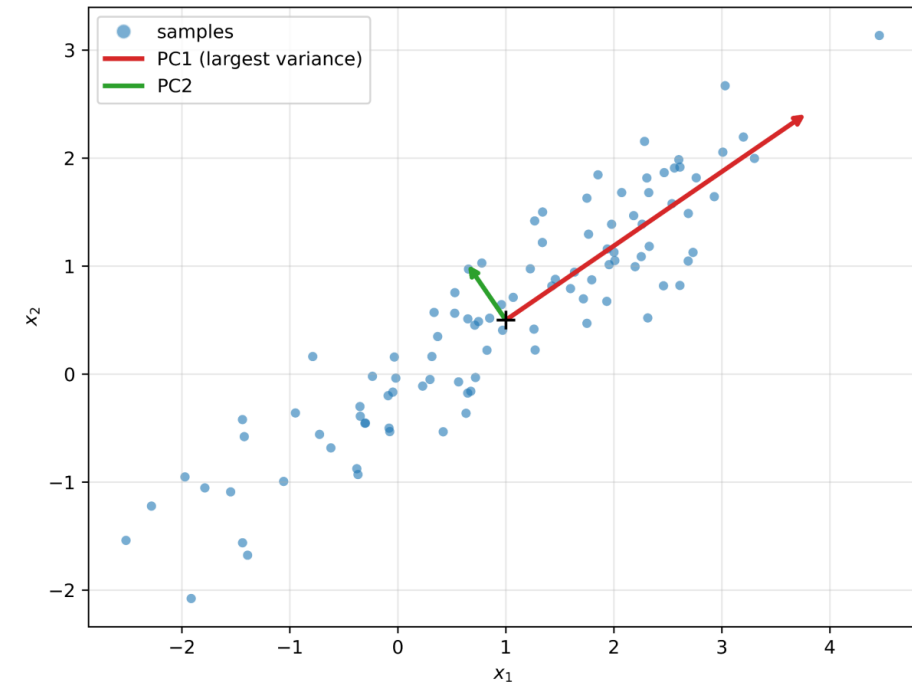
Diffusion



Refresher: PCA

For data points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$ in D dimensions:

Find 1-D linear projection $z_i = \mathbf{u}^T \mathbf{x}_i$
that explains most variance of $\mathbf{x}$



PCA (2)

The variance of z is $\mathbf{u}^T \mathbf{S} \mathbf{u}$

$$\max_{\mathbf{u}} \mathbf{u}^T \mathbf{S} \mathbf{u} \text{ subject to } \|\mathbf{u}\| = 1$$

Leads to

$$\mathbf{S} \mathbf{u} = \lambda \mathbf{u}$$

and the resulting explained variance is

$$\mathbf{u}^T \mathbf{S} \mathbf{u} = \mathbf{u}^T \lambda \mathbf{u} = \lambda \mathbf{u}^T \mathbf{u} = \lambda$$

$\mathbf{u}$ is the principal eigenvector of $\mathbf{S}$

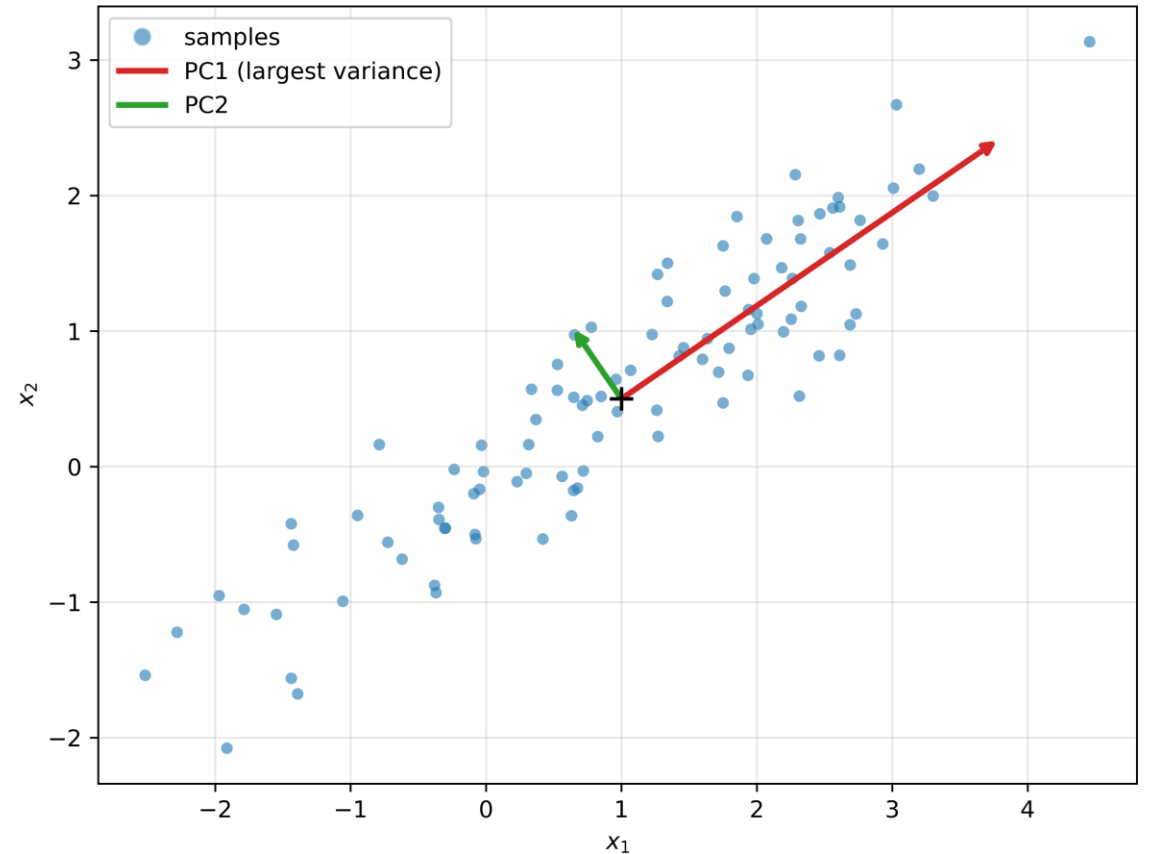
Unexplained variance is $\sum_{d=2}^D \lambda_d$

$$\mathbf{S} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T$$

Covariance of $\mathbf{X}$

PCA wrap-up

- Start from data $\mathbf{x}$
 - Find orthogonal projection $\mathbf{z}$
 - Maximize variance of projection
- Solution: eigenvectors of $\mathbf{S}$



Let's make PCA generative

- Start from a latent variable $\mathbf{z}$ in M dimensions
- Define a linear relationship between $\mathbf{z}$ and $\mathbf{x}$
- Find the same* solution as PCA

Probabilistic PCA

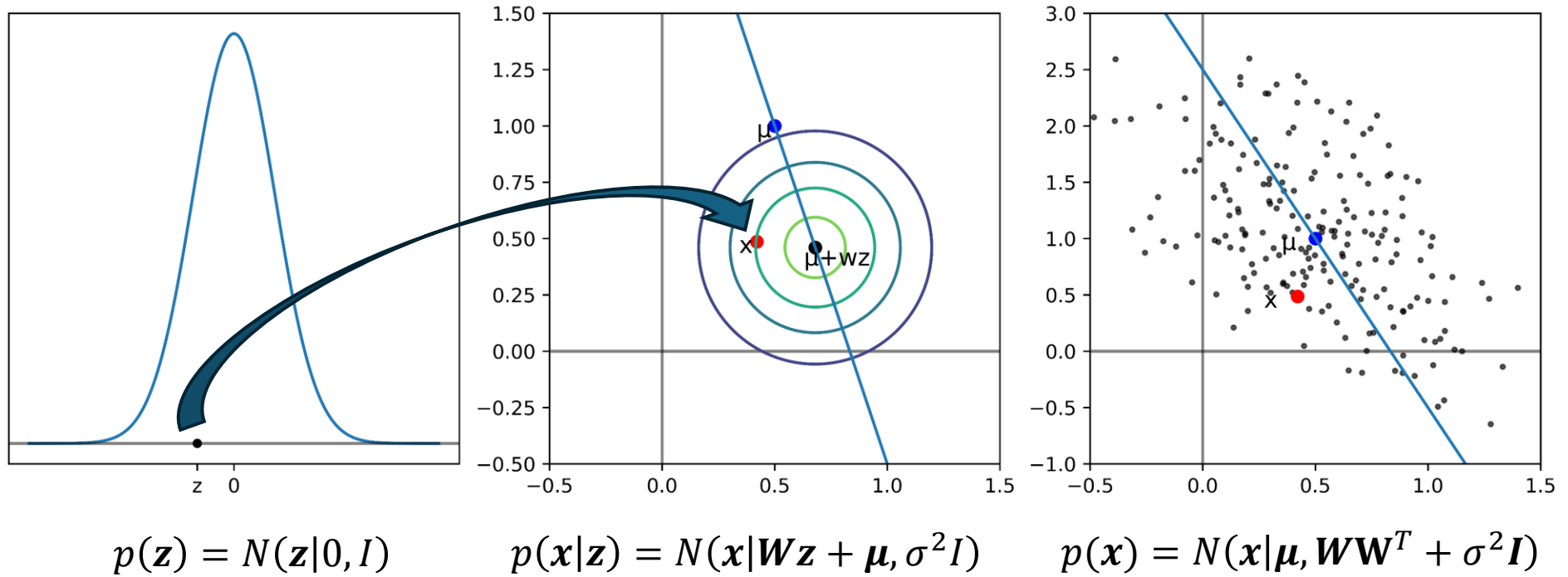
- Latent variables $\mathbf{z}$

$$p(\mathbf{z}) = N(\mathbf{z}|\mathbf{0}, I)$$

- Linear-Gaussian generative process

$$p(\mathbf{x}|\mathbf{z}) = N(\mathbf{x}|\mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \sigma^2 I)$$

PPCA generative model



How to learn $\mathbf{W}$, $\boldsymbol{\mu}$, σ from data points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$?

Maximize the likelihood of the observations

$$p(\mathbf{x}) = \int p(\mathbf{x}, \mathbf{z}) d\mathbf{z} = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$$

We don't observe $\mathbf{z}$
So we average over all possible values

Weighted average based on $p(\mathbf{z})$

Both $p(\mathbf{z})$ and $p(\mathbf{x}|\mathbf{z})$ are Gaussian and $\mathbf{x}$ is a linear function of $\mathbf{z}$

$$p(\mathbf{x}) = N(\mathbf{x}|\boldsymbol{\mu}, \mathbf{W}\mathbf{W}^T + \sigma^2\mathbf{I})$$

Given our data points, this is a function
of $\boldsymbol{\mu}$, $\mathbf{W}$, and σ

Calculating the log-likelihood

$$\ln p(\mathbf{X}|\boldsymbol{\mu}, \mathbf{W}, \sigma^2) = \sum_{n=1}^N \ln p(\mathbf{x}_n|\mathbf{W}, \boldsymbol{\mu}, \sigma^2)$$

$$= -\frac{ND}{2} \ln(2\pi) - \frac{N}{2} \ln|\mathbf{C}| - \frac{1}{2} \sum_{n=1}^N (\mathbf{x}_n - \boldsymbol{\mu})^T \mathbf{C}^{-1} (\mathbf{x}_n - \boldsymbol{\mu})$$

$$\mathbf{C} = \mathbf{W}\mathbf{W}^T + \sigma^2 \mathbf{I}$$

PPCA maximum likelihood solutions

$$\boldsymbol{\mu} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i$$

PCA projection
times an arbitrary
matrix

$$\mathbf{W} = \mathbf{U}_M (\mathbf{L}_M - \sigma^2 \mathbf{I})^{\frac{1}{2}} \mathbf{R}$$

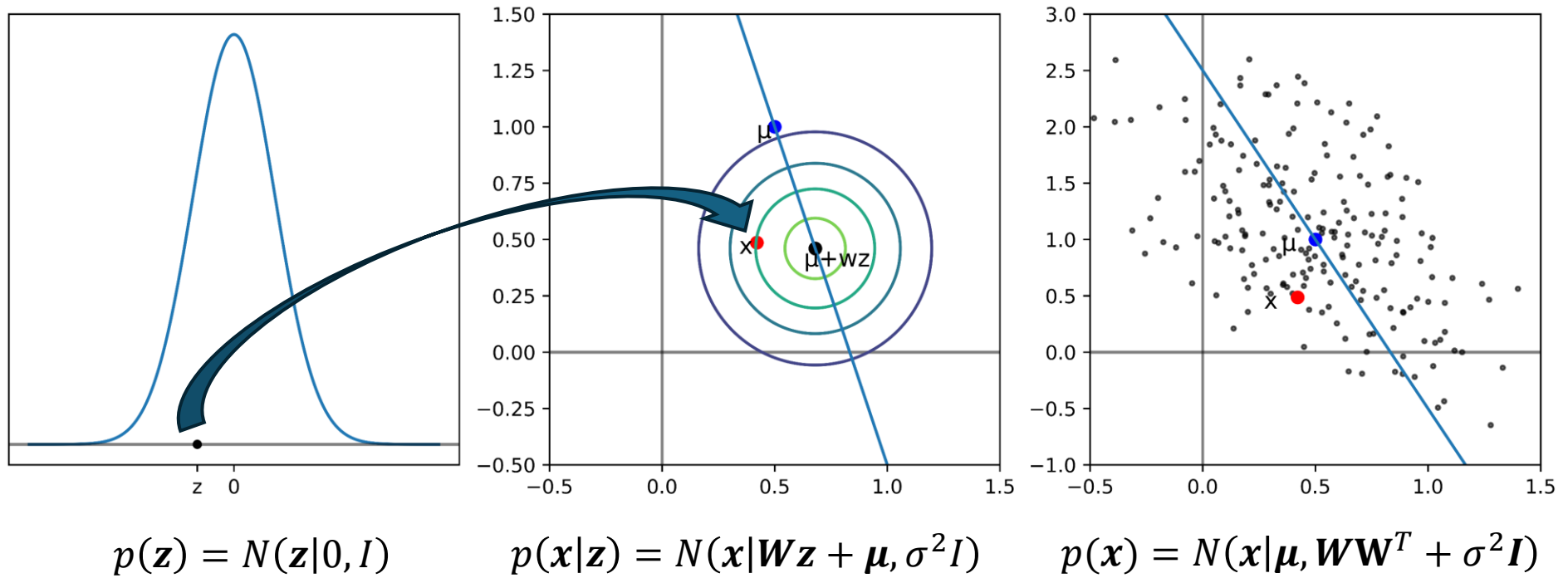
$\mathbf{U}_M$ eigenvectors of $\mathbf{S}$
 $\mathbf{L}_M$ eigenvalues of $\mathbf{S}$
 $\mathbf{R}$ orthonormal

PPCA's error =
PCA's Unexplained
variance

$$\sigma^2 = \frac{1}{D - M} \sum_{i=M+1}^D \lambda_i$$

Remaining
eigenvalues of $\mathbf{S}$

PPCA: linear/Gaussian generative model



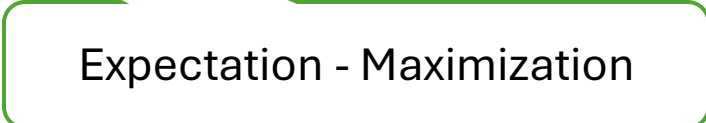
Why would I ever use this?

- It scales better with the number of features D
 - rarely a problem these days
- It can handle missing data
 - common in e.g. clinical datasets
 - Assumes 'missing at random'

PPCA with missing data: the idea

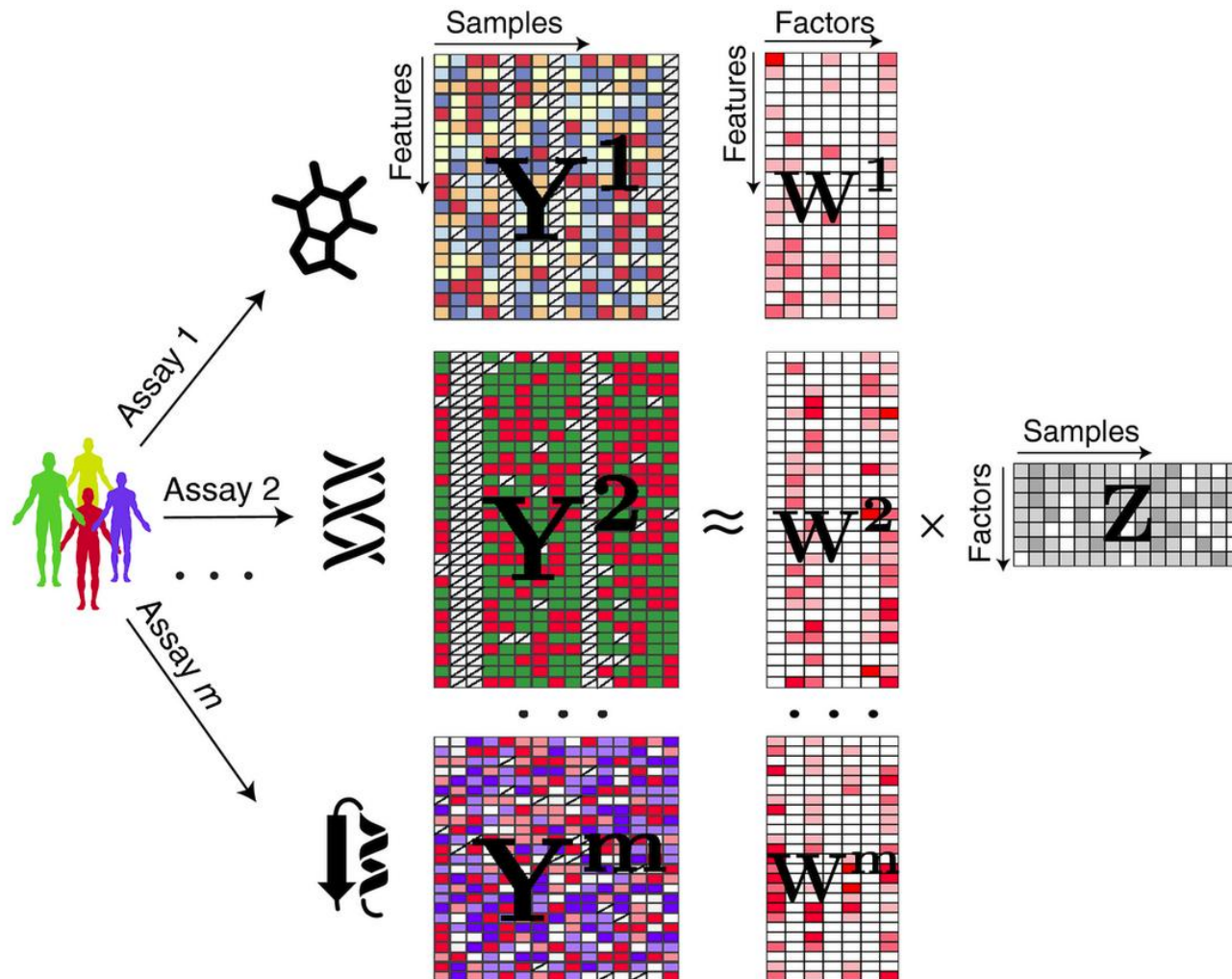
Missing data are modeled as hidden variables, like z

1. Start with random values for the parameters
 - μ , W , and σ
2. Assuming these parameters are correct, estimate posterior distributions for z and missing data
 - (everything is linear/Gaussian)
3. Given these posteriors use maximum likelihood to estimate the optimal μ , W , and σ
4. Repeat steps 2-3 until likelihood stops increasing



Expectation - Maximization

MOFA: multi-omics factor analysis



$$p(\mathbf{z}) = N(\mathbf{z}|0, \mathbf{I})$$

$$p(\mathbf{y}_m|\mathbf{z}) = N(\mathbf{y}_m|\mathbf{W}_m\mathbf{z} + \boldsymbol{\mu}, \sigma^2\mathbf{I})$$

+ sparsity regularizations for $\mathbf{W}$

Summary – part 1

- Generative models learn the entire data distribution $p(\mathbf{x})$ or $p(\mathbf{x},y)$
- PCA is equivalent to a linear/Gaussian generative model
- Formulas for fitting PPCA with maximum likelihood



Generative Models 2

Variational Auto-Encoders

Stavros Makrodimitris

s.makrodimitris@uva.nl



UNIVERSITY OF AMSTERDAM
Swammerdam Institute for Life Sciences

What if the generative process is non-linear?

- Latent variables $\mathbf{z}$

$$p(\mathbf{z}) = N(\mathbf{z}|\mathbf{0}, I)$$

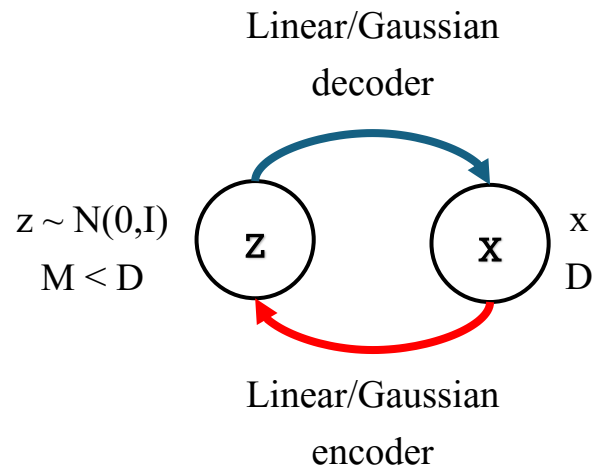
- **Linear**-Gaussian generative process

$$p(\mathbf{x}|\mathbf{z}) = N(\mathbf{x}|\mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \sigma^2 I)$$

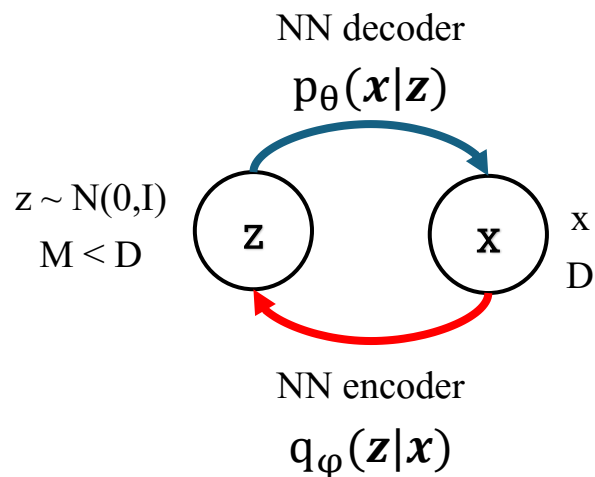
- Maximize data likelihood $p(\mathbf{x})$

Types of generative models

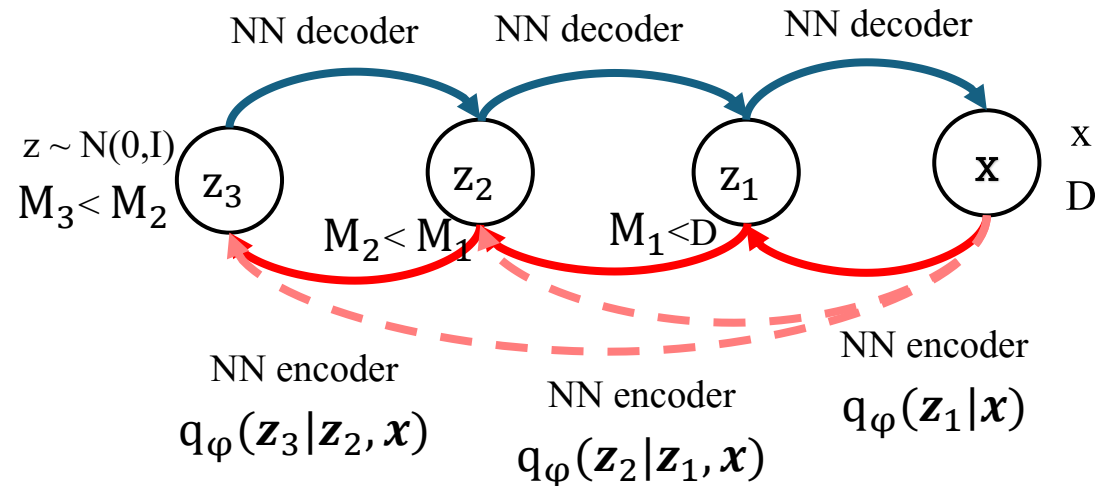
probabilistic PCA



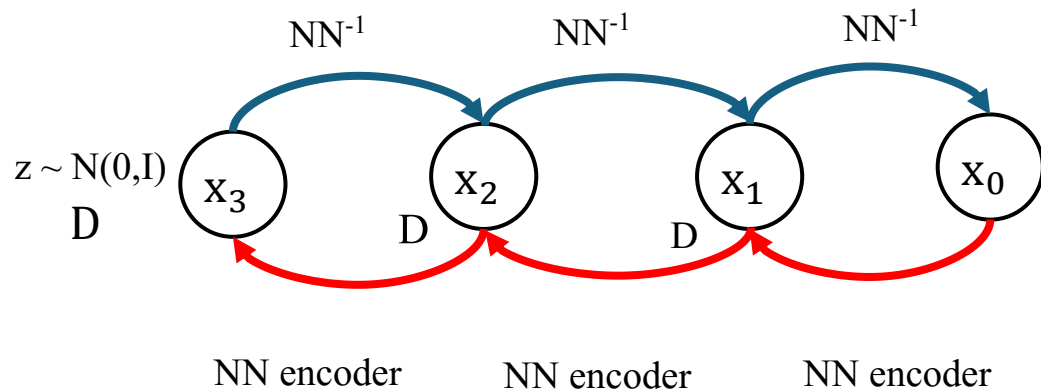
VAE



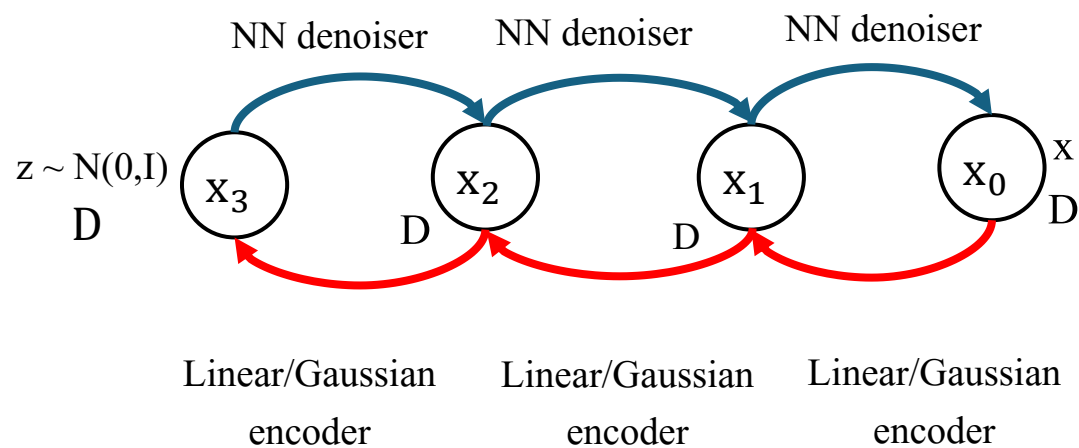
HVAE



Normalizing Flows



Diffusion



Learning goals for part 2

- Derive the ELBO for VAE
- Use variational inference with neural networks
- Understand posterior collapse in VAEs

Variational Auto-Encoder (VAE)

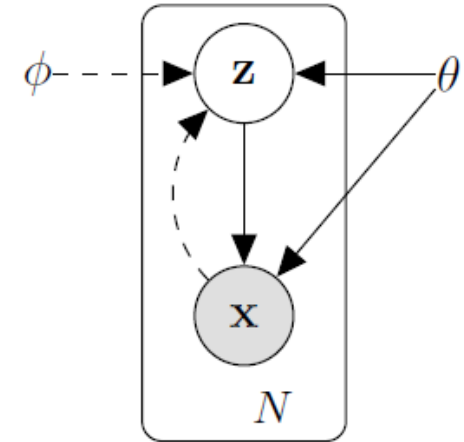
- Latent variables $\mathbf{z}$

$$p(\mathbf{z}) = N(\mathbf{z}|\mathbf{0}, \mathbf{I})$$

- **Arbitrarily complex** generative process

$$p_{\theta}(\mathbf{x}|\mathbf{z}) = N(\mathbf{x}|\mathbf{f}_{\theta}(\mathbf{z}), \sigma^2 \mathbf{I})$$

$\mathbf{f}_{\theta}(\mathbf{z}) \rightarrow$ non-linear, differentiable function



Maximize the likelihood of the observations

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$$
$$= \int N(\mathbf{x}|\mathbf{f}_{\theta}(\mathbf{z}), \sigma^2 I)N(\mathbf{z}|\mathbf{0}, I)d\mathbf{z}$$

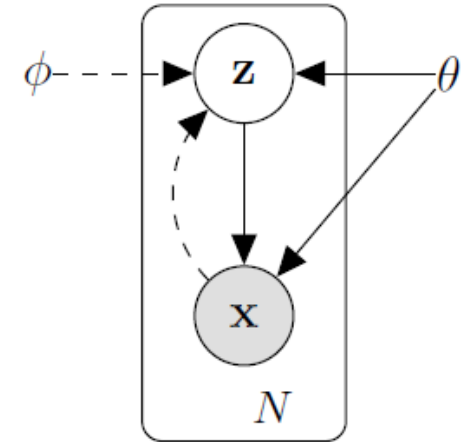
We don't know how to
calculate this

$$p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{\int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}}$$

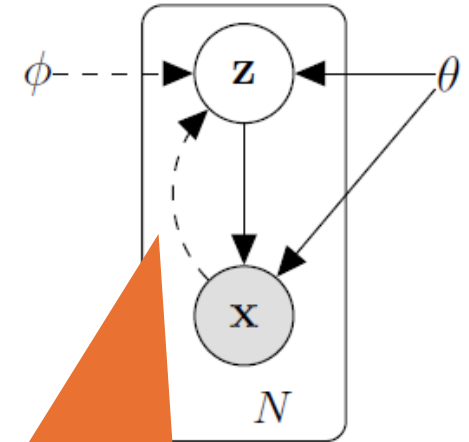
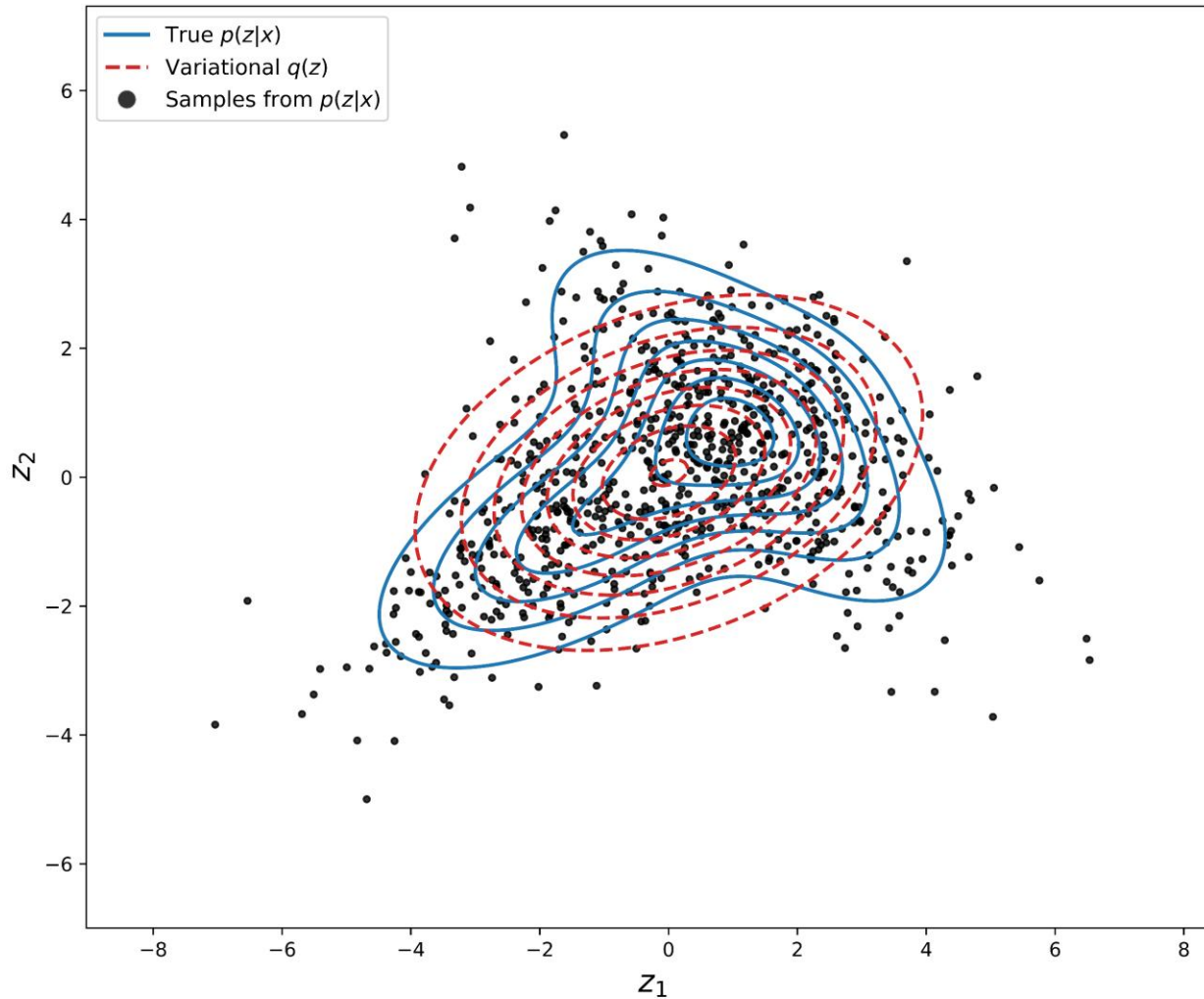
or this

Trick 1: “Invent” an encoder

- The encoder tries to approximate the posterior $p(\mathbf{z}|\mathbf{x})$
 - Variational Bayes approximation
 - We call this approximation $q_\phi(\mathbf{z}|\mathbf{x})$
 - Typically a Gaussian



Trick 1: Approximate $p(z|x)$ by a $q(z)$



This allows us to also approximate the log-likelihood

Trick 1.5: Derive a lower bound on $p(\mathbf{x})$

$$\ln p_{\theta}(\mathbf{x}) = \ln p_{\theta}(\mathbf{x}) \int q_{\varphi}(\mathbf{z}|\mathbf{x}) d\mathbf{z}$$

Multiply by 1

$$= \int \ln p_{\theta}(\mathbf{x}) q_{\varphi}(\mathbf{z}|\mathbf{x}) d\mathbf{z}$$

$p(\mathbf{x})$ independent of $\mathbf{z}$

$$= \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} [\ln p_{\theta}(\mathbf{x})]$$

Definition of expectation

Trick 1.5: Derive a lower bound on $p(\mathbf{x})$

$$\ln p_{\theta}(\mathbf{x}) = \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} [\ln p_{\theta}(\mathbf{x})]$$

$$= \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{p_{\theta}(\mathbf{z}|\mathbf{x})} \right]$$

$p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z}|\mathbf{x})p(\mathbf{x})$

$$= \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p_{\theta}(\mathbf{x}, \mathbf{z}) q_{\varphi}(\mathbf{z}|\mathbf{x})}{p_{\theta}(\mathbf{z}|\mathbf{x}) q_{\varphi}(\mathbf{z}|\mathbf{x})} \right]$$

multiply by 1

$$= \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right] + \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{q_{\varphi}(\mathbf{z}|\mathbf{x})}{p_{\theta}(\mathbf{z}|\mathbf{x})} \right]$$

split into two terms

Trick 1.5: Derive a lower bound on $p(\mathbf{x})$

$$\ln p_{\theta}(\mathbf{x}) = \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right] + \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{q_{\varphi}(\mathbf{z}|\mathbf{x})}{p_{\theta}(\mathbf{z}|\mathbf{x})} \right]$$

$$= \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right] + D_{\text{KL}}(q_{\varphi}(\mathbf{z}|\mathbf{x}) || p_{\theta}(\mathbf{z}|\mathbf{x}))$$

This we can calculate
and maximize

This we cannot
This is always ≥ 0

Trick 1.5: Derive a lower bound on $p(\mathbf{x})$

$$\ln p_{\theta}(\mathbf{x}) = \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right] + \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{q_{\varphi}(\mathbf{z}|\mathbf{x})}{p_{\theta}(\mathbf{z}|\mathbf{x})} \right]$$

$$= \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right] + D_{\text{KL}}(q_{\varphi}(\mathbf{z}|\mathbf{x}) || p_{\theta}(\mathbf{z}|\mathbf{x}))$$

$$\ln p_{\theta}(\mathbf{x}) \geq \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right]$$

Evidence Lower Bound
ELBO

VAE training by maximum likelihood

$$\ln p_{\theta}(\mathbf{x}) \geq \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right]$$

- Maximize ELBO
 - this increases the log-likelihood

Taking a closer look at the ELBO

$$\mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right] = \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p_{\theta}(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right]$$

$p(\mathbf{x}, \mathbf{z}) = p(\mathbf{x}|\mathbf{z})p(\mathbf{z})$

$$= \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} [\ln p_{\theta}(\mathbf{x}|\mathbf{z})] + \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p(\mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right]$$

split terms

$$= \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} [\ln p_{\theta}(\mathbf{x}|\mathbf{z})] - \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{q_{\varphi}(\mathbf{z}|\mathbf{x})}{p(\mathbf{z})} \right]$$

reverse fraction

$$= \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})} [\ln p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_{\varphi}(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}))$$

Taking a closer look at the ELBO

$$\text{ELBO} = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\ln p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}))$$

reconstruction

Posterior similar
to prior

How to maximize this: KL

$$\text{ELBO} = \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})}[\ln p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_{\varphi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

Because $q_{\varphi}(\mathbf{z}|\mathbf{x})$, $p(\mathbf{z})$ Gaussian $\rightarrow$ KL has a closed form

$$p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$$

$$q_{\varphi}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}(\mathbf{x}), \boldsymbol{\sigma}(\mathbf{x})^2 \mathbf{I})$$

Use a NN to learn these

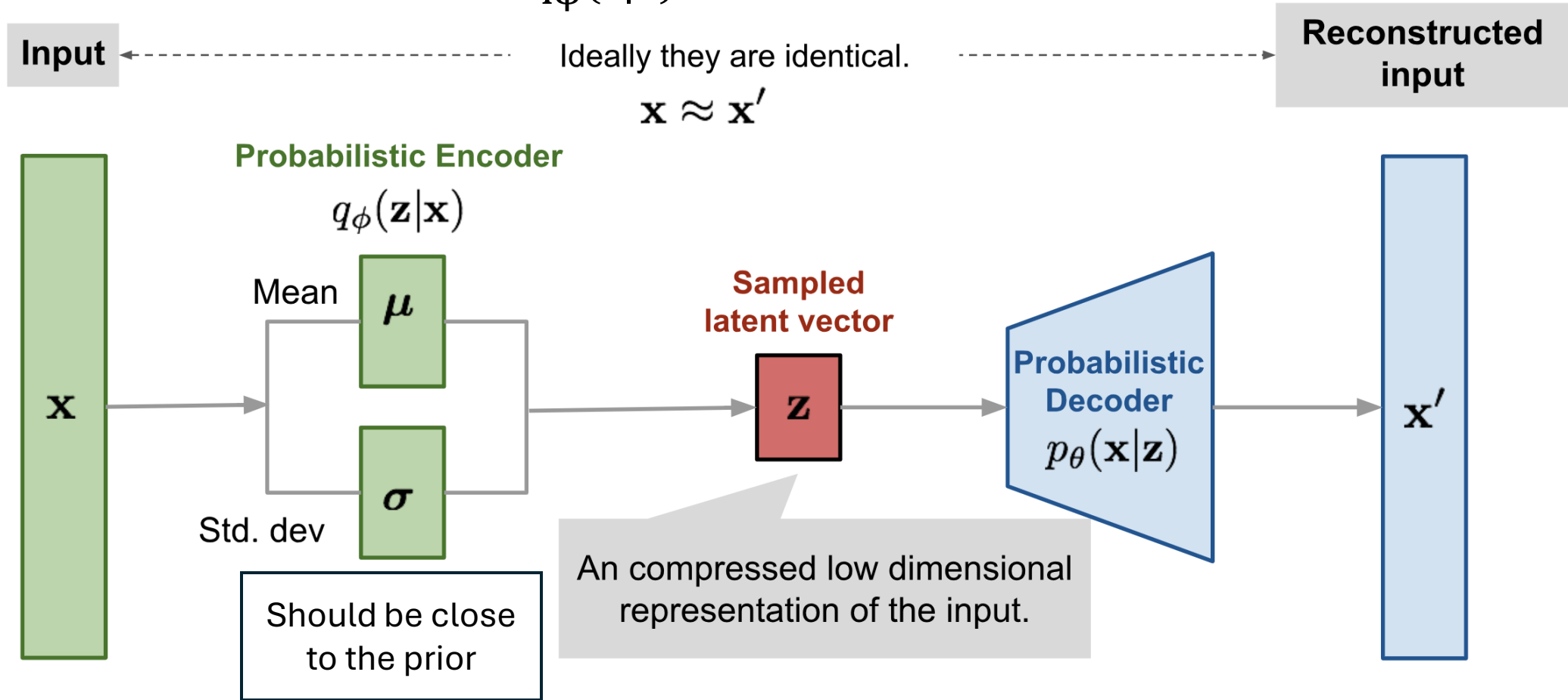
$$D_{\text{KL}} = -\frac{1}{2} \sum_{m=1}^M (1 + 2\log(\sigma_m) - \mu_m^2 - \sigma_m^2)$$

How to maximize this: reconstruction

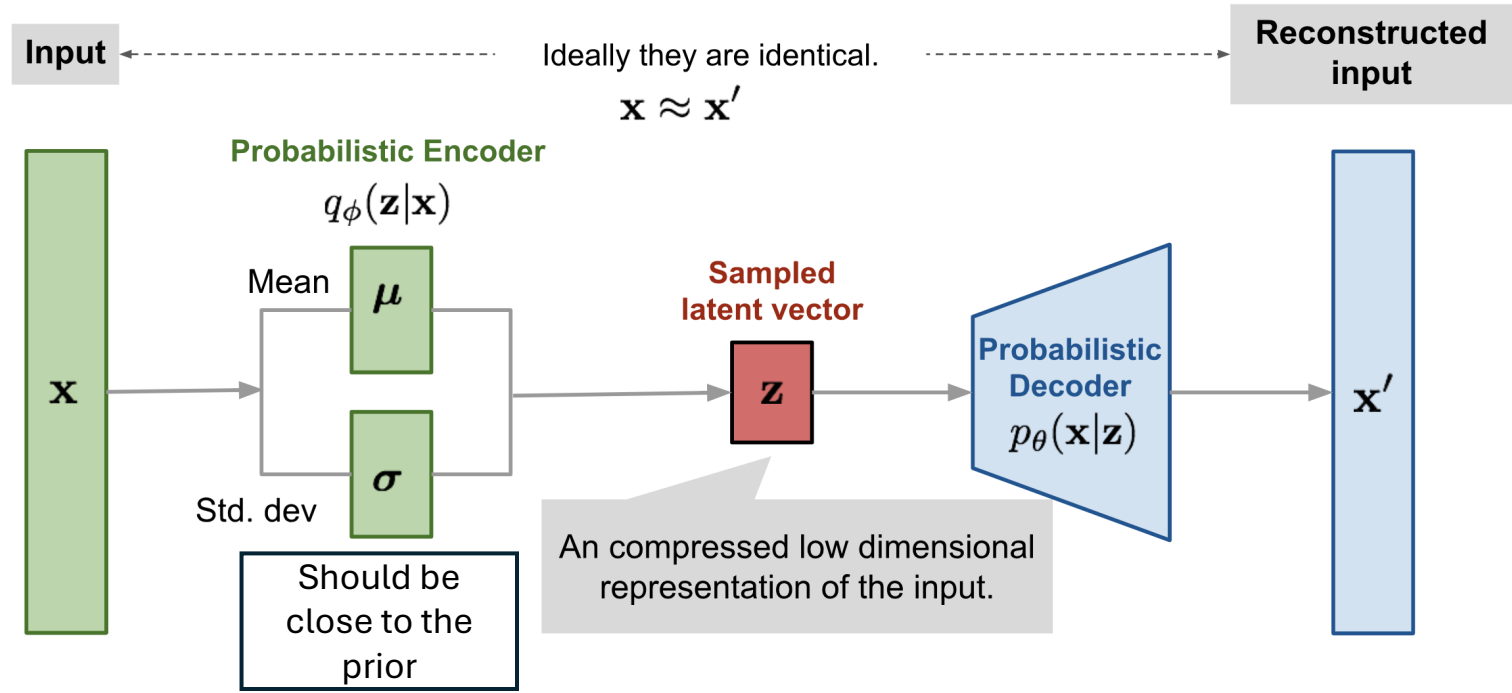
$$\text{ELBO} = \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})}[\ln p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_{\varphi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

- Starting from a sample $\mathbf{x}_i$
- Estimate the posterior $q_{\varphi}(\mathbf{z}|\mathbf{x}_i)$
 - (by the NN)
- The generative model θ should reconstruct $\mathbf{x}_i$
 - from samples of the posterior

$$\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\ln p_{\theta}(\mathbf{x}|\mathbf{z})]$$



$$-D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}))$$



- How many samples to draw for each $\mathbf{x}_i$?
 - More samples $\rightarrow$ better estimate of ELBO; also more compute
 - For large batches, 1 sample is enough!
- How to draw these samples?

Trick 2: Reparameterization

$$\boldsymbol{\varepsilon} \sim \mathcal{N}(0, \boldsymbol{I})$$

$$\boldsymbol{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$$

Trick 2: Reparameterization

$$\boldsymbol{\varepsilon} \sim \text{N}(\mathbf{0}, \mathbf{I})$$

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$$

$$\mathbf{z} \sim \text{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2 \mathbf{I})$$

Trick 2: Reparameterization

$$\boldsymbol{\varepsilon} \sim \mathcal{N}(0, \boldsymbol{I})$$

$$\boldsymbol{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$$

$$\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2 \boldsymbol{I})$$

- Sampling is independent of neural networks calculating the parameters

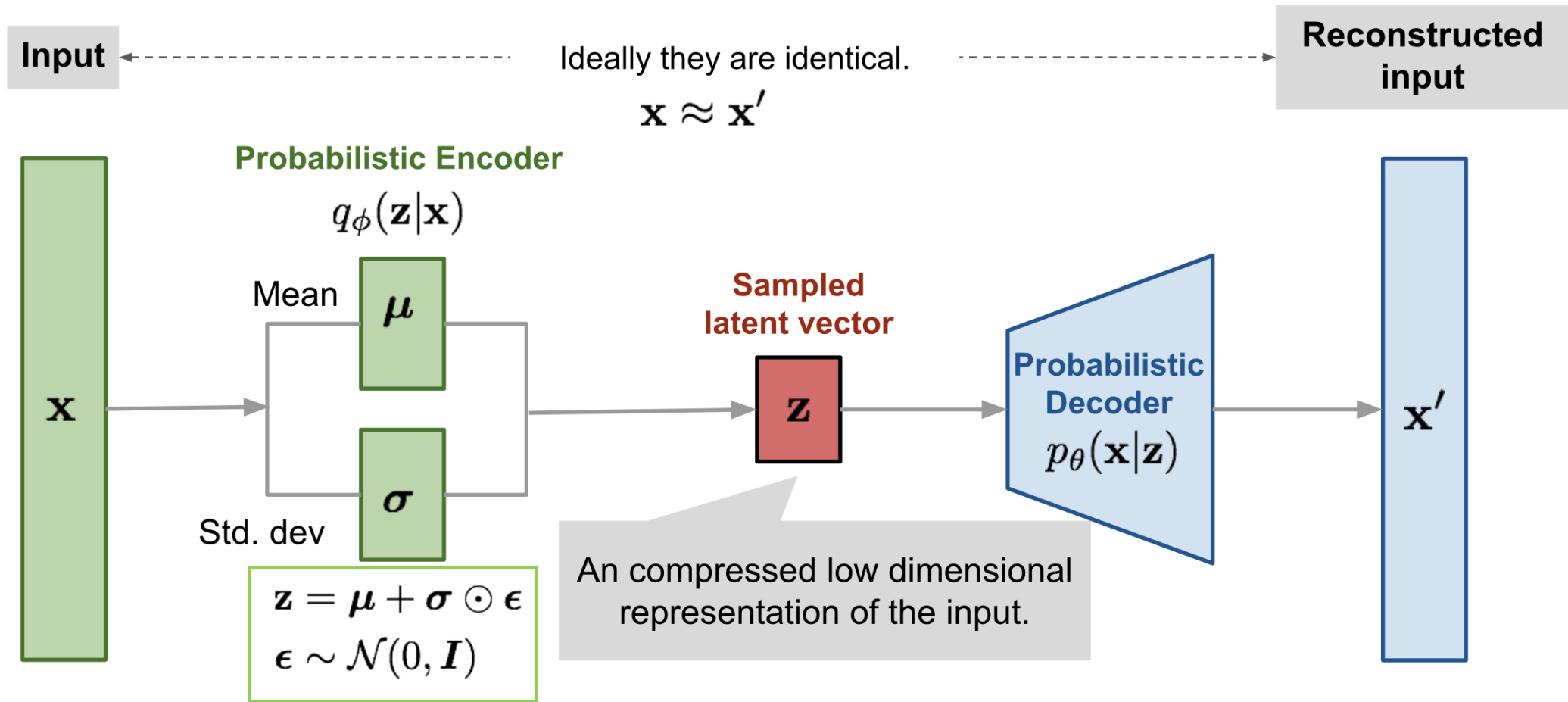
Trick 2: Reparameterization

$$\boldsymbol{\varepsilon} \sim \mathcal{N}(0, \boldsymbol{I})$$

$$\boldsymbol{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$$

$$\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2 \boldsymbol{I})$$

- Sampling is independent of neural networks calculating the parameters
- It's possible to back-propagate and train the encoder



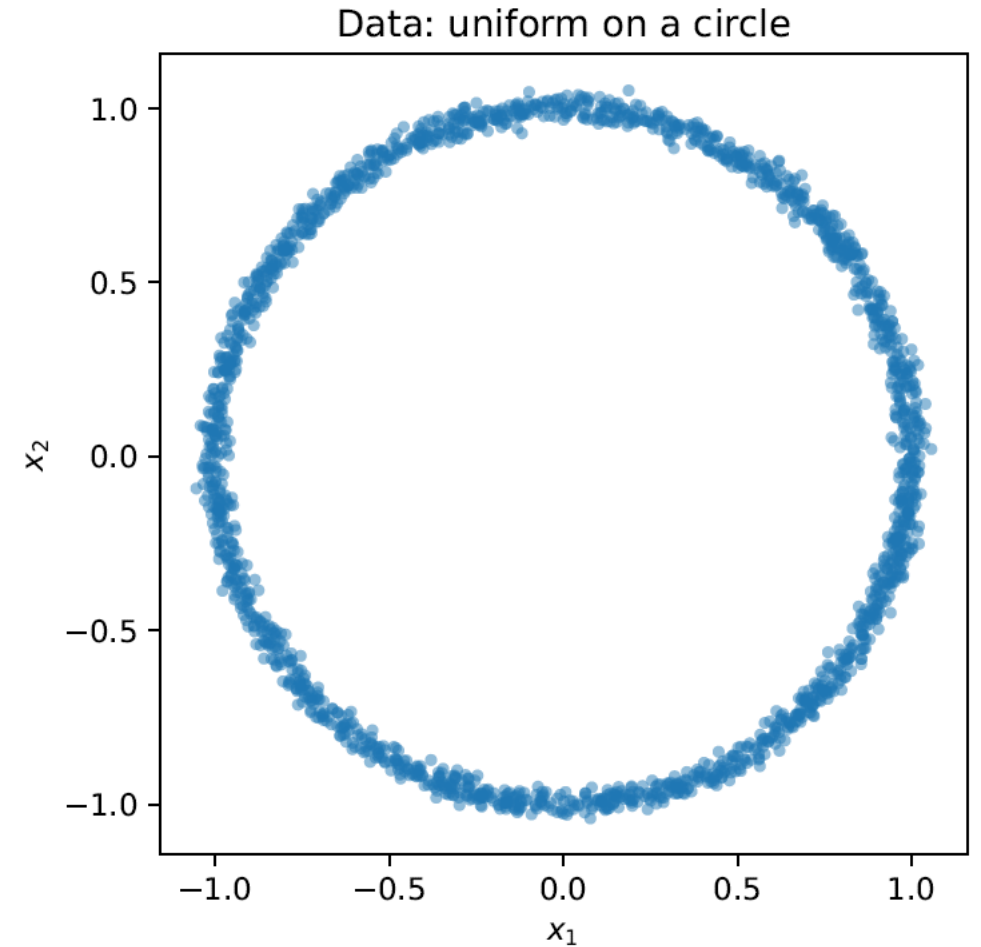
Comparison to regular auto-encoder

- VAE can generate data from latent space
- The KL term “turns off” “useless” latent dimensions
 - Posterior collapse

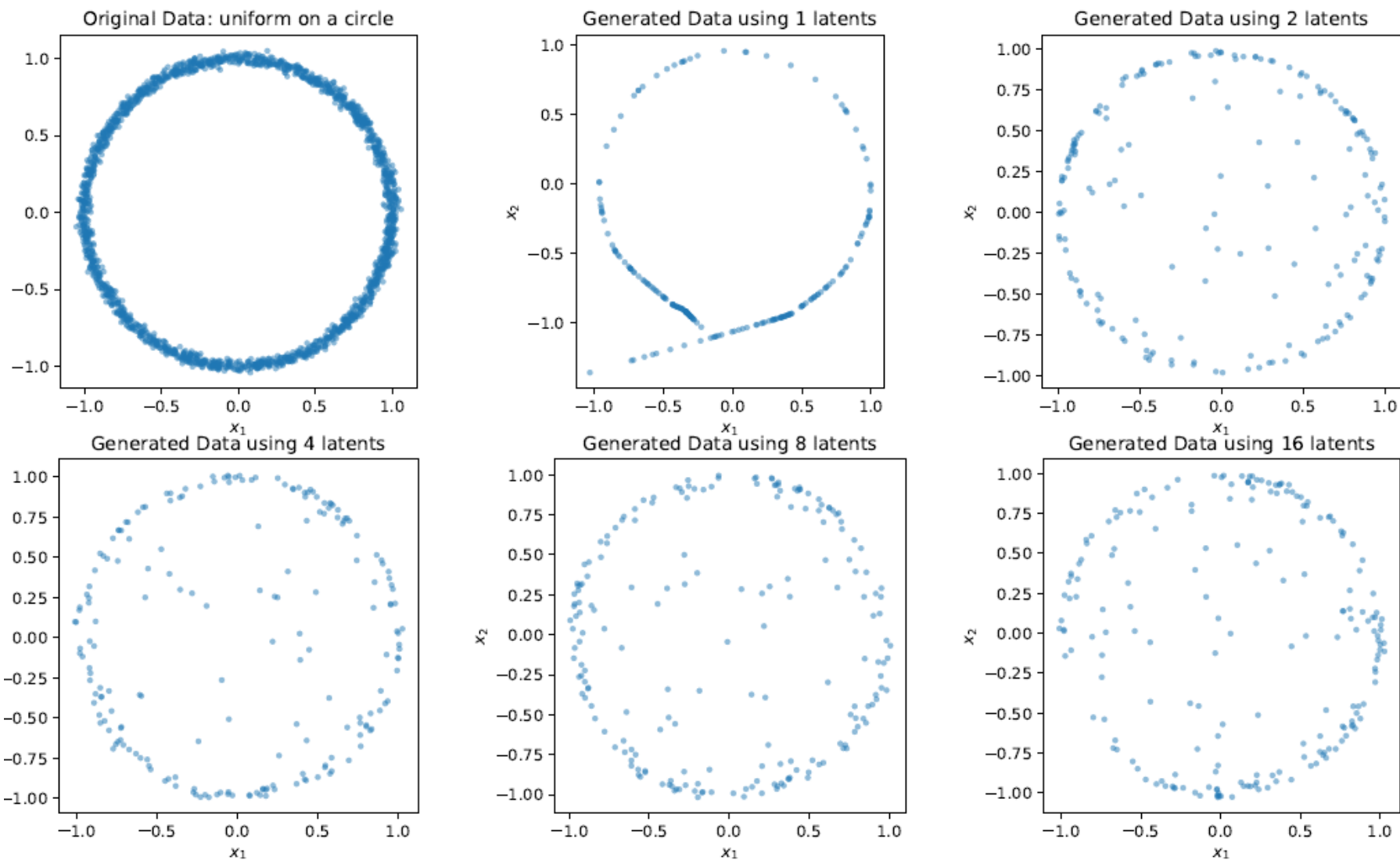
$$\mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})}[\ln p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_{\varphi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

Let's look into this

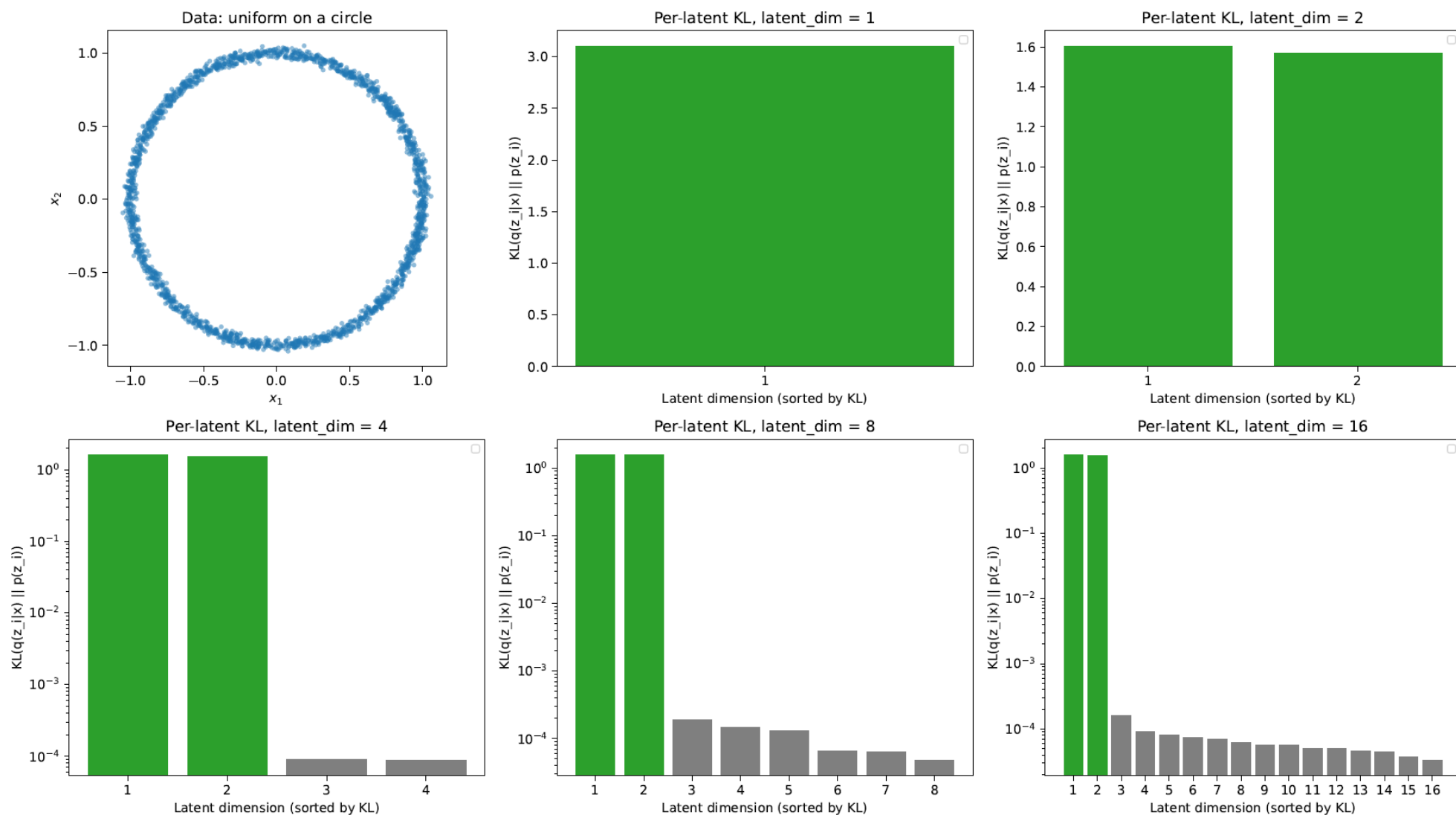
- 2D data on a circle
- Train VAE with
 - 1,2,4,8,16 latents
- Compare KL of each latent to prior



All models learn $p(\mathbf{x})$ well



Models essentially use 2 latents



A link with mutual information

- This KL is an approximation of the mutual information
 - between the data and each latent dimension
- Permutation-based statistical test to assess whether KL is “significantly” high

JOURNAL ARTICLE

An in-depth comparison of linear and non-linear joint embedding methods for bulk and single-cell multi-omics

[Stavros Makrodimitris](#) ✉, [Bram Pronk](#), [Tamim Abdelaal](#), [Marcel Reinders](#)

[Author Notes](#)

Briefings in Bioinformatics, Volume 25, Issue 1, January 2024, bbad416, <https://doi.org/10.1093/bib/bbad416>

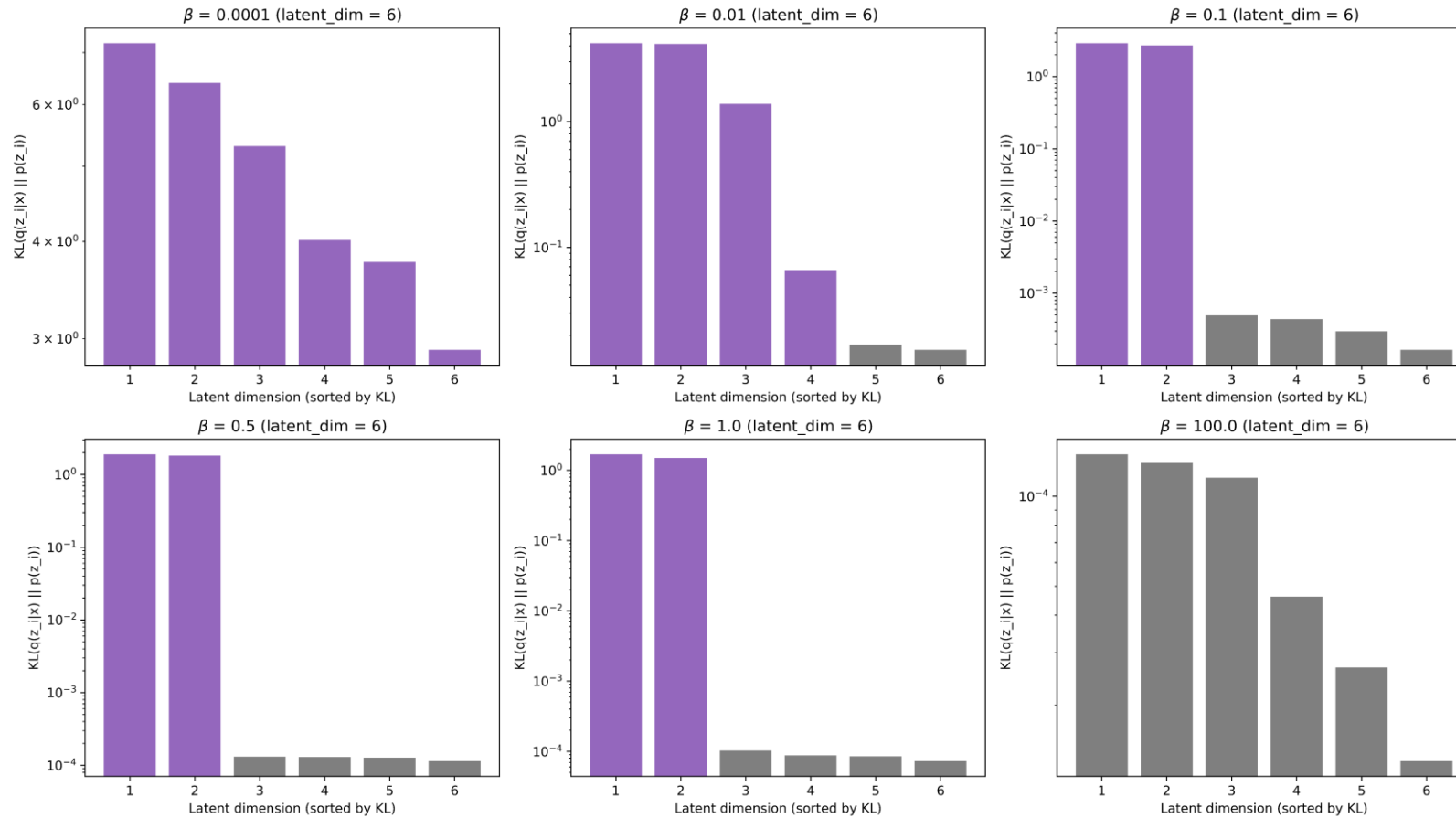
Published: 28 November 2023 **Article history** ▼

Modifying the KL regularization strength: β -VAE

$$\mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x})}[\ln p_{\theta}(\mathbf{x}|\mathbf{z})] - \beta D_{\text{KL}}(q_{\varphi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

- Larger $\beta \rightarrow$ more regularization $\rightarrow$ fewer latents active
- Smaller $\beta \rightarrow$ less regularization $\rightarrow$ more latents active

Let's test this on our circle dataset



Take-aways β -VAE

- Increasing β decreases capacity of each latent to encode the data
- Forces more latents to the prior
- Too much regularization leads to poor reconstruction (underfitting)
- Increasing β helps model find independent sources of variation



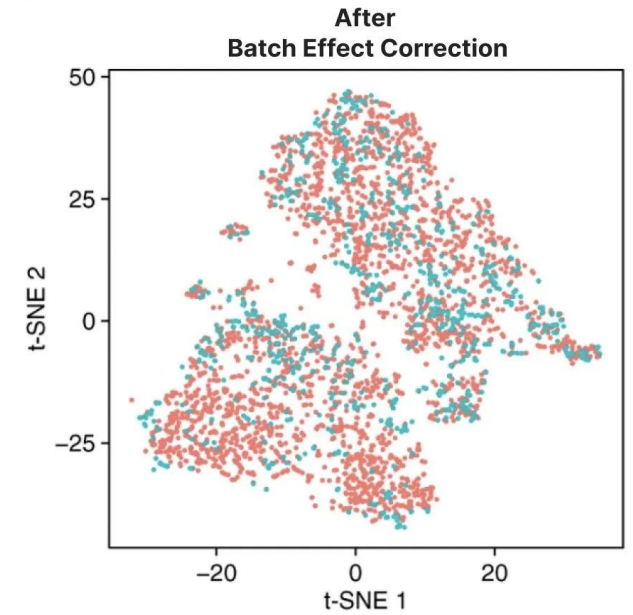
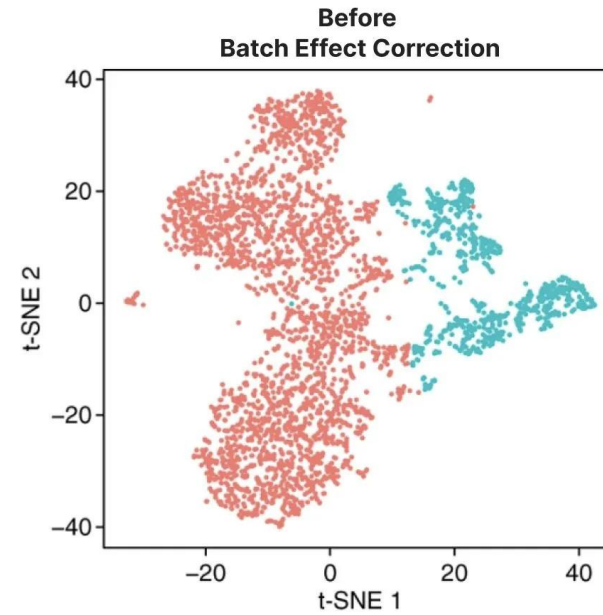
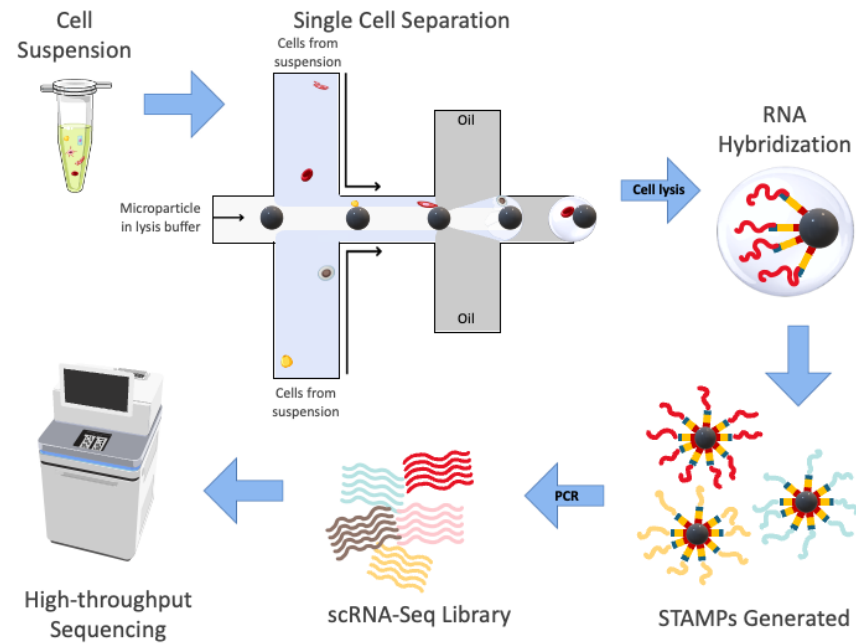
Statistics > Machine Learning

[Submitted on 10 Apr 2018]

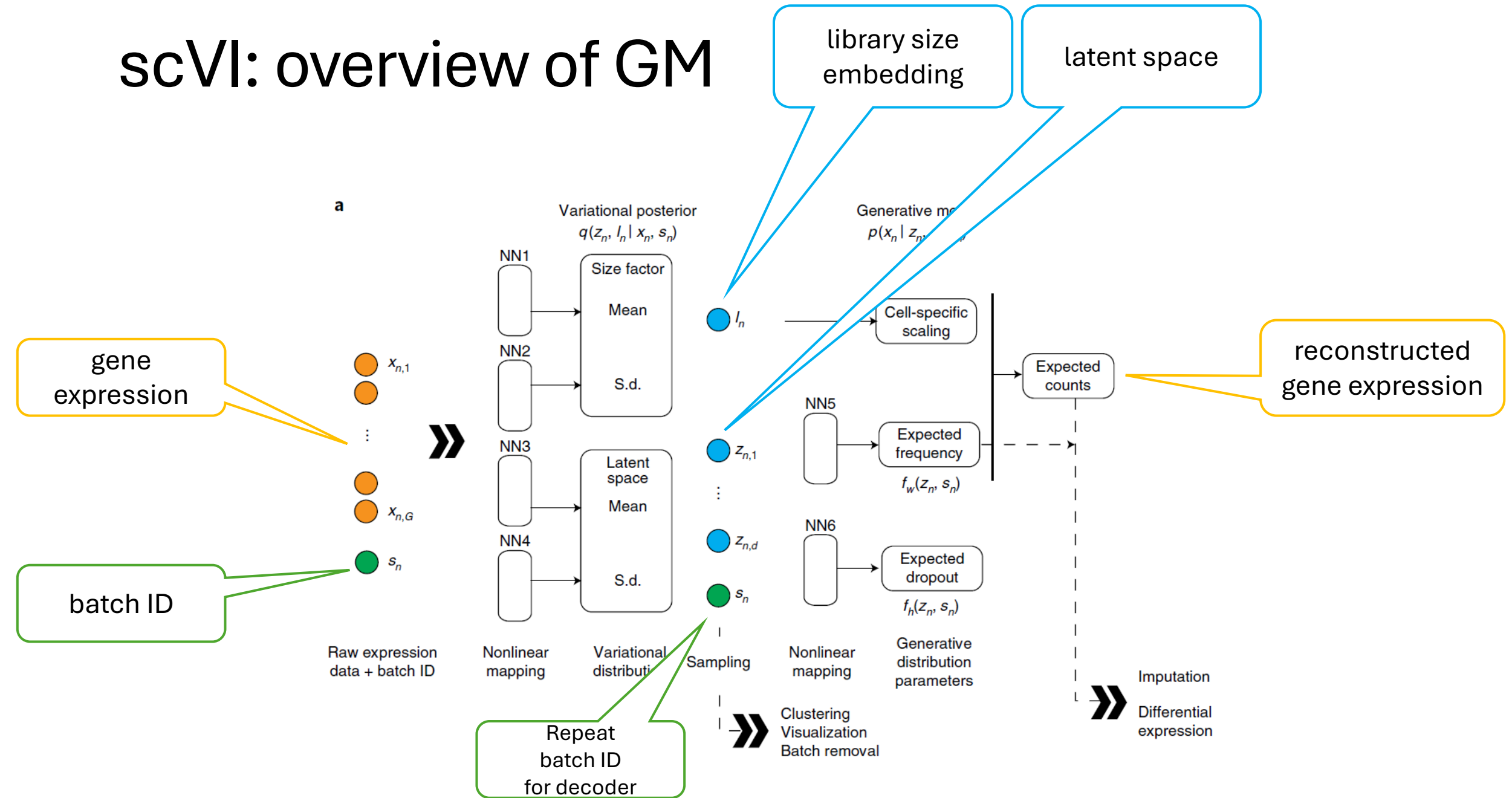
Understanding disentangling in β -VAE

Christopher P. Burgess, Irina Higgins, Arka Pal, Loic Matthey, Nick Watters, Guillaume Desjardins, Alexander Lerchner

Example: single-cell RNA

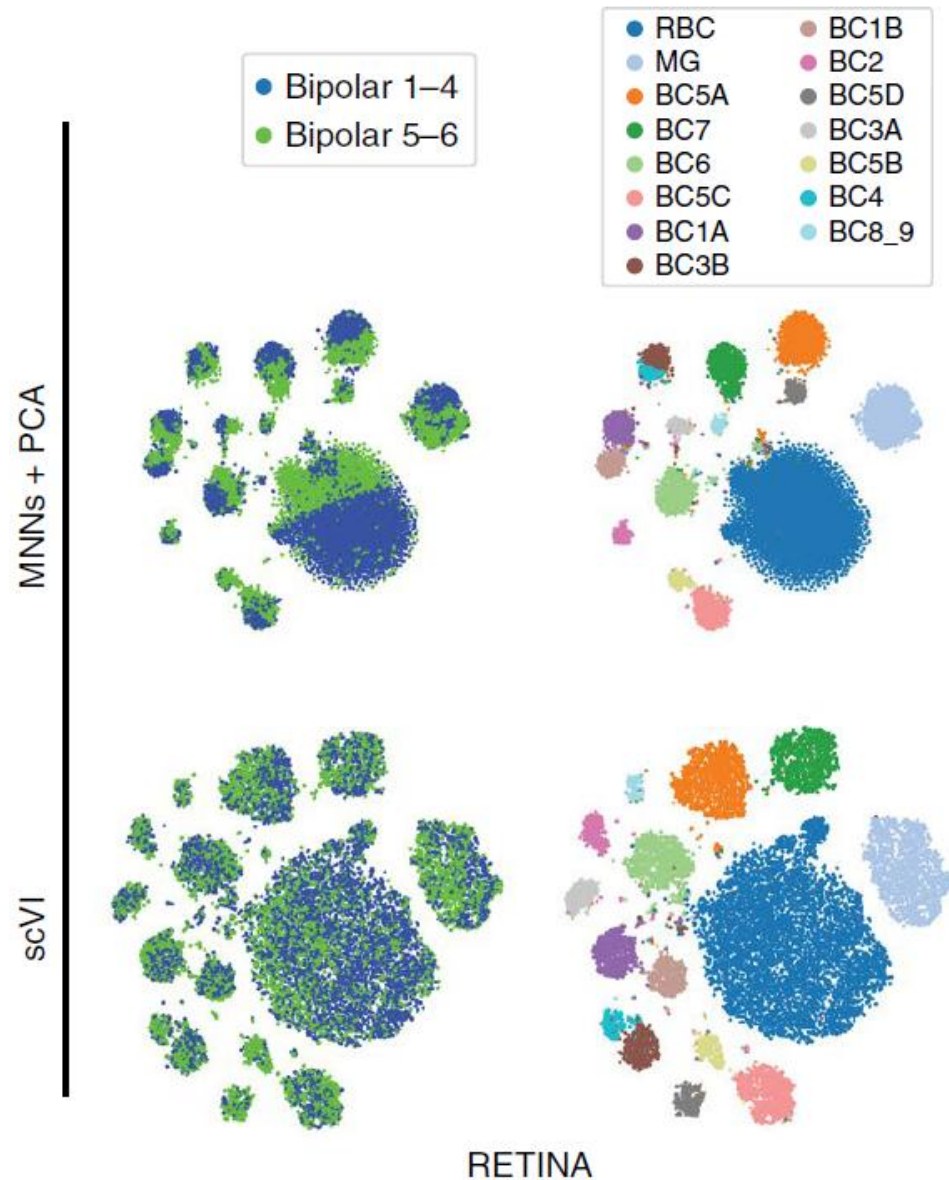


scVI: overview of GM



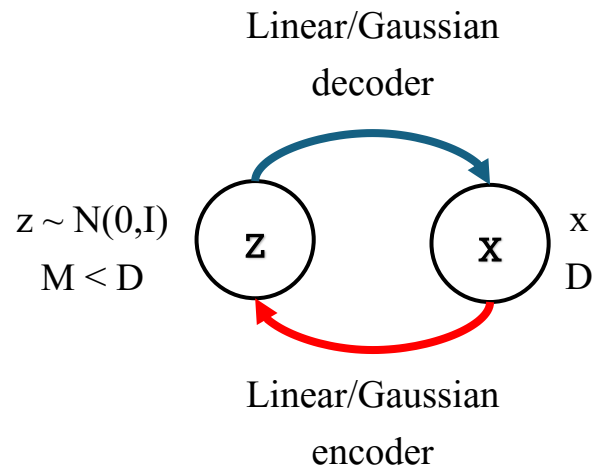
t-SNE of latent space

- Correct batch effects
- Maintain cell types

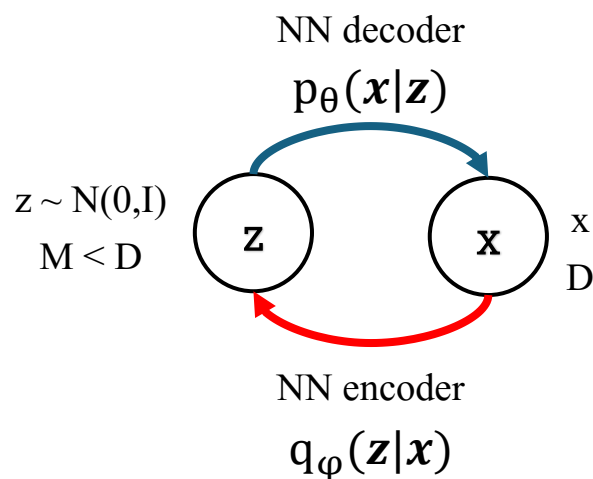


Why only one set of latent variables?

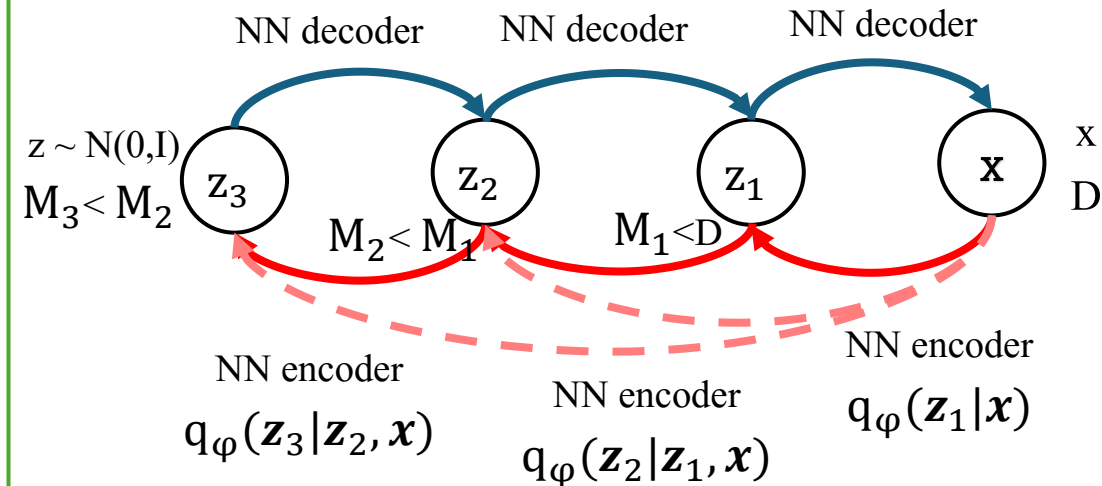
probabilistic PCA



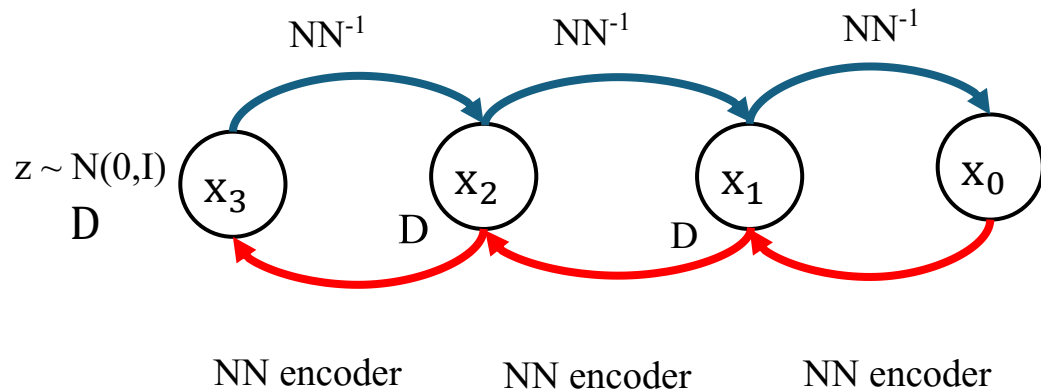
VAE



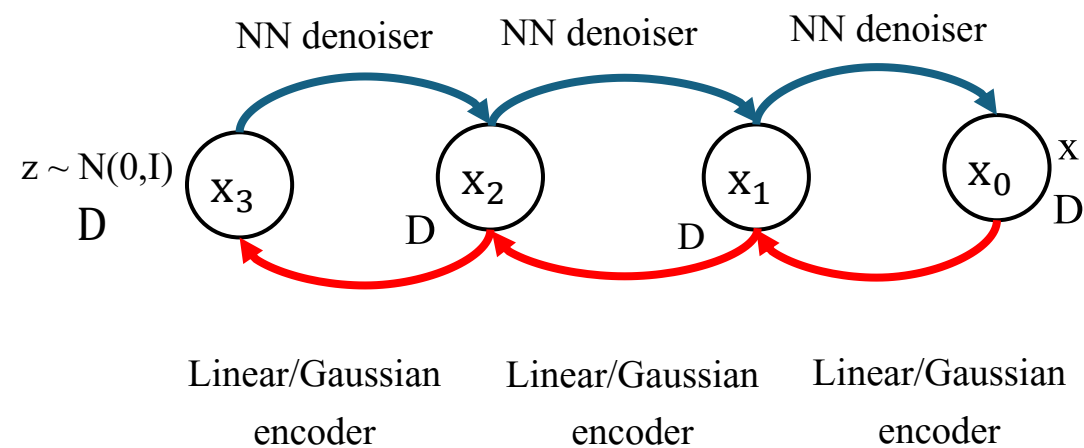
HVAE



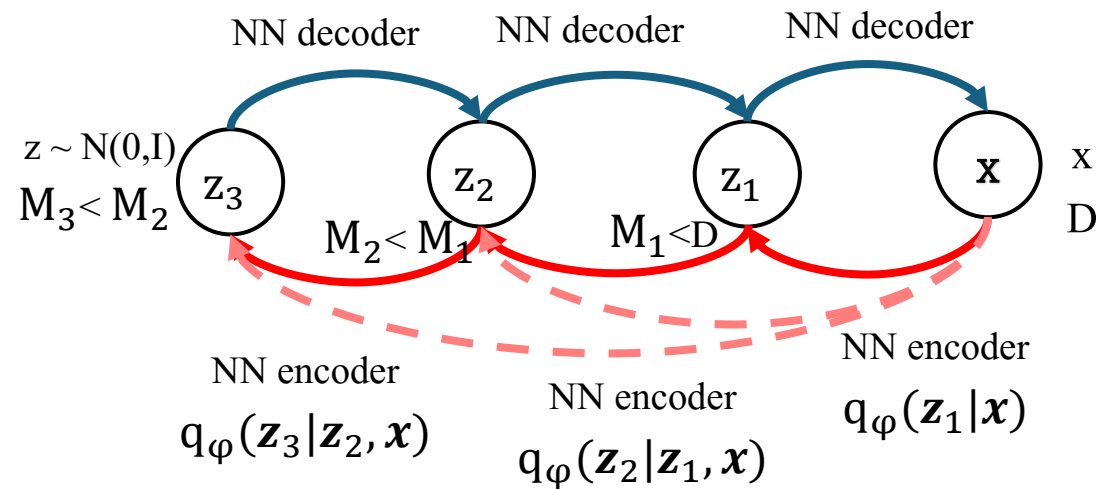
Normalizing Flows



Diffusion



Stack VAEs on top of each other: Hierarchical VAE



$$\text{ELBO: } \ln p(x) \geq \mathbb{E}_{q_\phi(z_{1:T}|x)} \left[\ln \frac{p(x, z_{1:T})}{q_\phi(z_{1:T}|x)} \right]$$

Hierarchical VAEs

- Learn a series of different latent variables
- More flexible models, capture more complex distributions
- Better generation quality
- Hierarchy might be meaningful

But:

- Deeper models, more parameters
 - More overfitting
- Optimization difficulties
 - e.g. posterior collapse
 - how to tune β in each layer?

Summary part 2

- VAE is a non-linear extension of PPCA
- Calculating $p(\mathbf{x})$ is impossible, so we maximize the ELBO
- ELBO = reconstruction error + KL regularization
- KL term forces latents to the prior if they don't explain the data
- We can stack VAEs to make a HVAE



Generative Models 3

Diffusion

Stavros Makrodimitris

s.makrodimitris@uva.nl



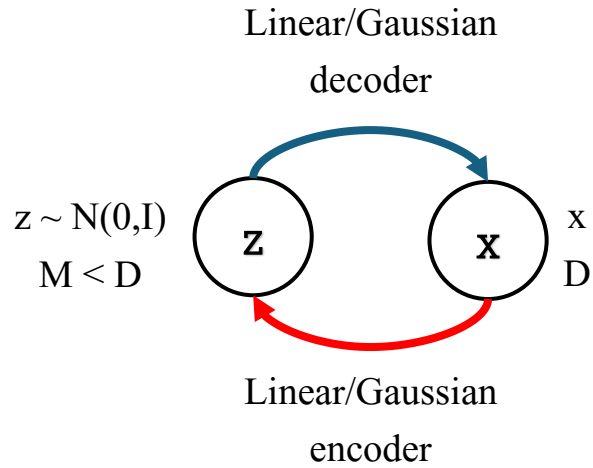
UNIVERSITY OF AMSTERDAM
Swammerdam Institute for Life Sciences

High-Quality generation

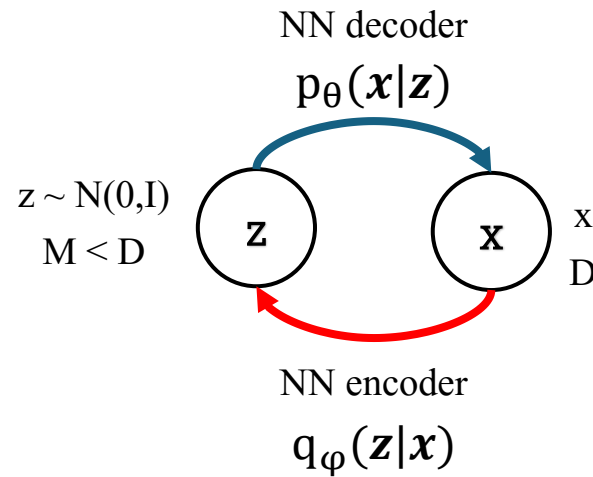
- VAEs learn useful latent representations,
 - but struggle at generating realistic data
- HVAEs give better generations,
 - but are harder to train
- Diffusion models are very good at generating realistic data,
 - but do not do dimensionality reduction
 - nor give meaningful latents

Types of generative models

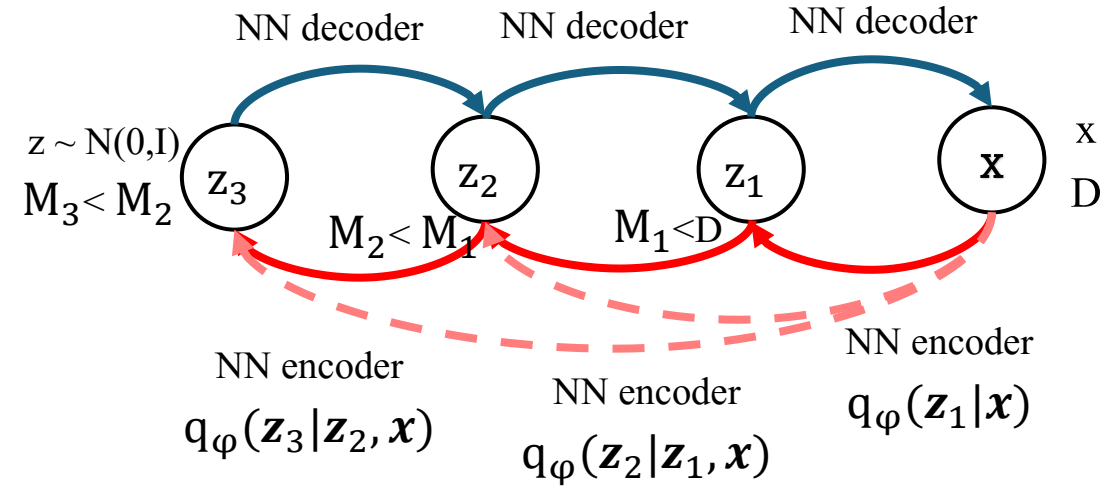
probabilistic PCA



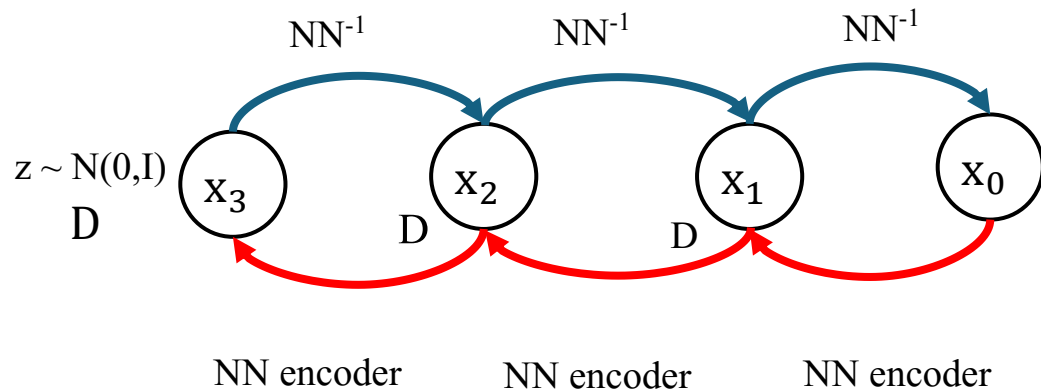
VAE



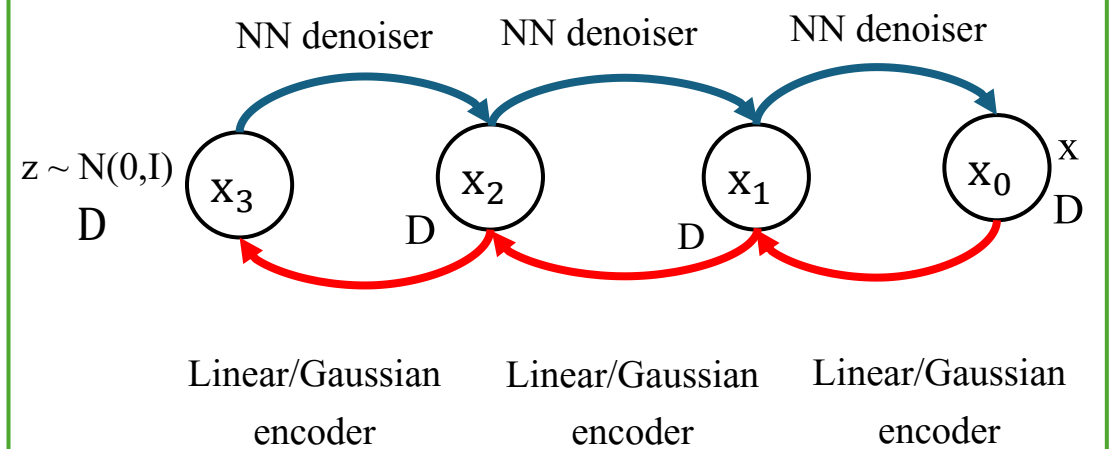
HVAE



Normalizing Flows



Diffusion

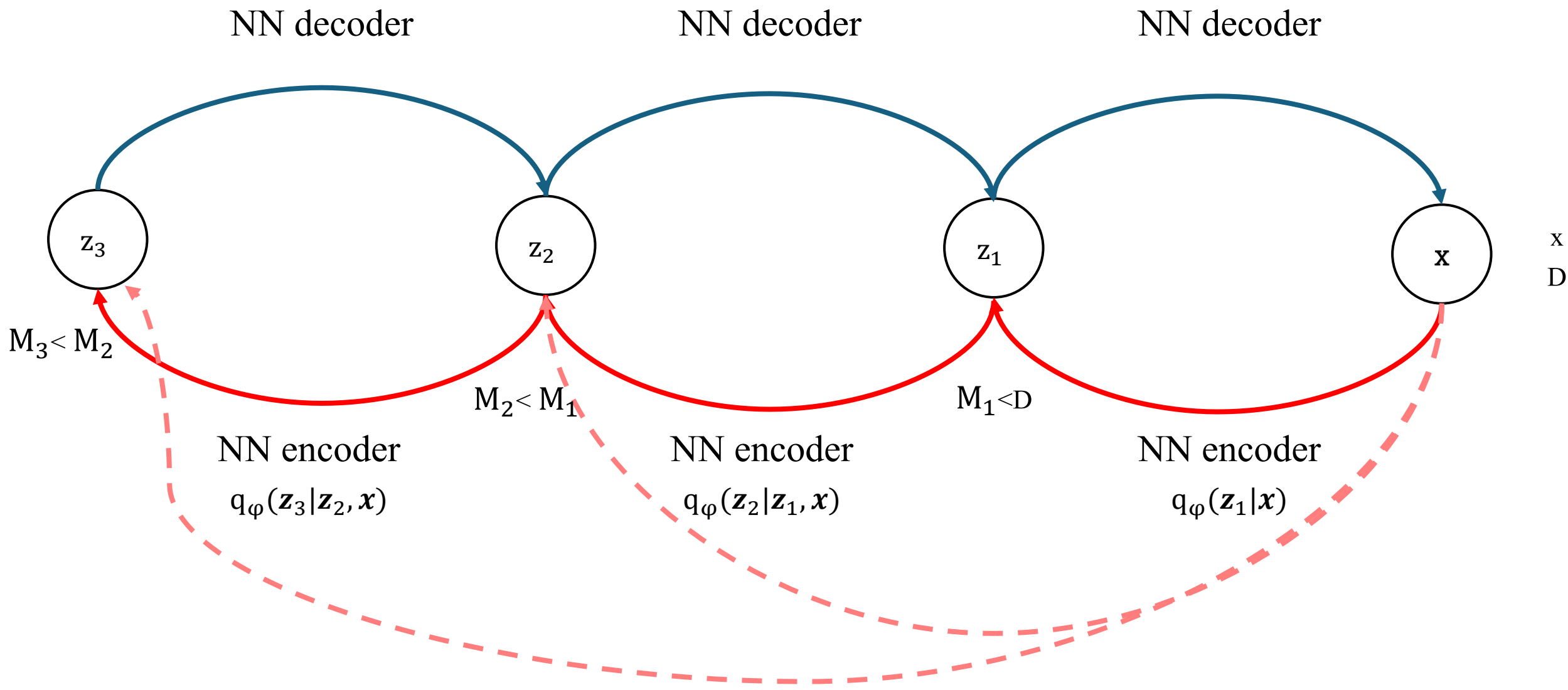


Learning goals – part 3

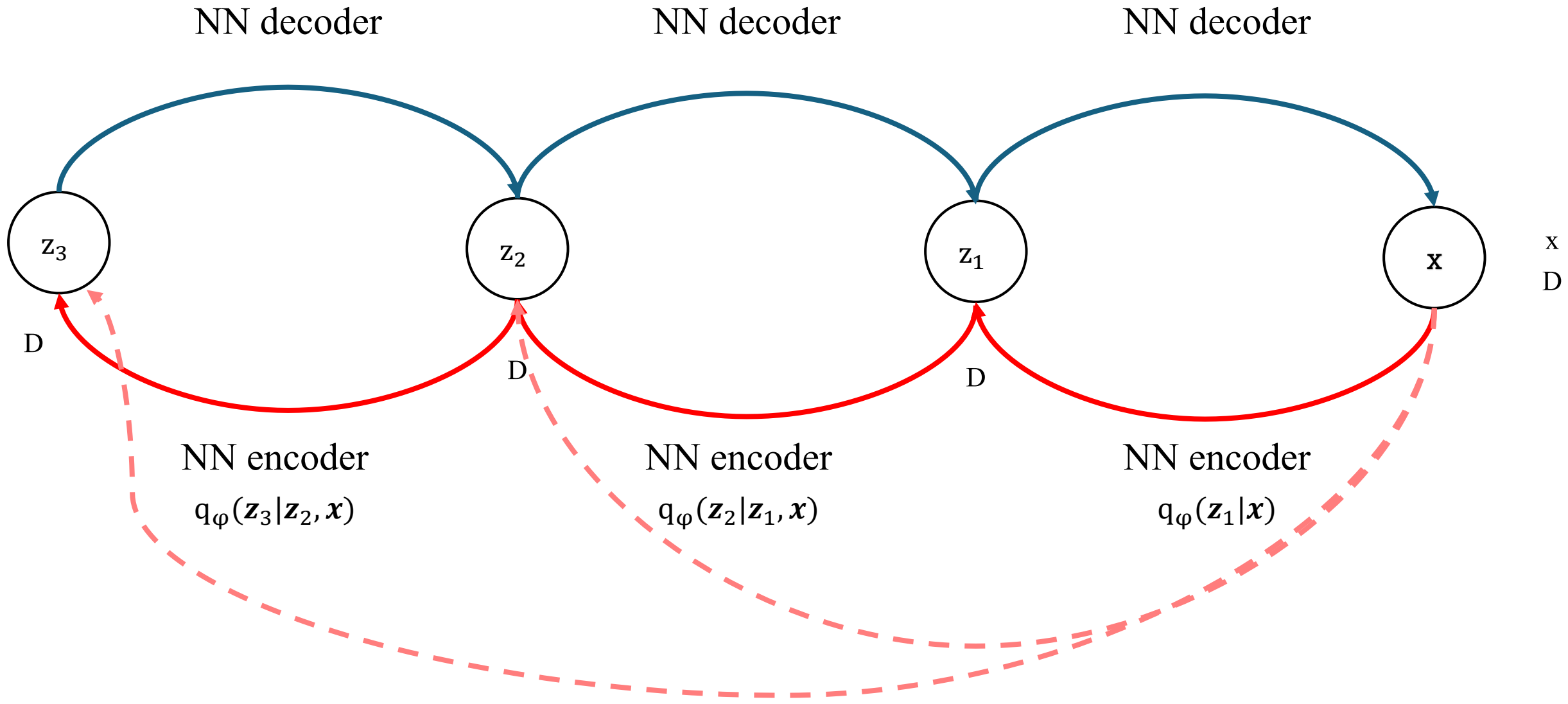
- Train a diffusion model
 - Diffusion
 - Reverse diffusion
 - ELBO calculation
- Understand basic conditional generation

Relation HVAEs and diffusion models

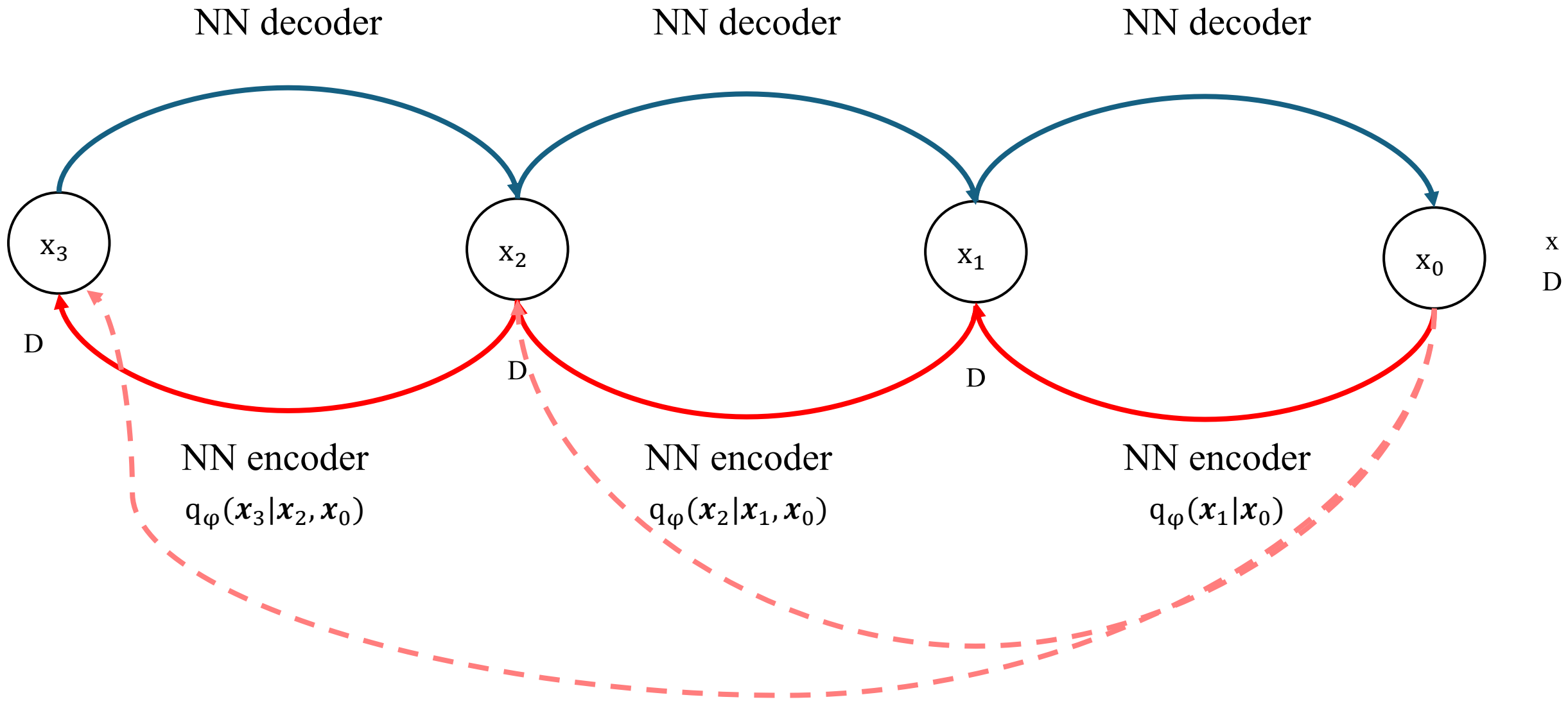
HVAE



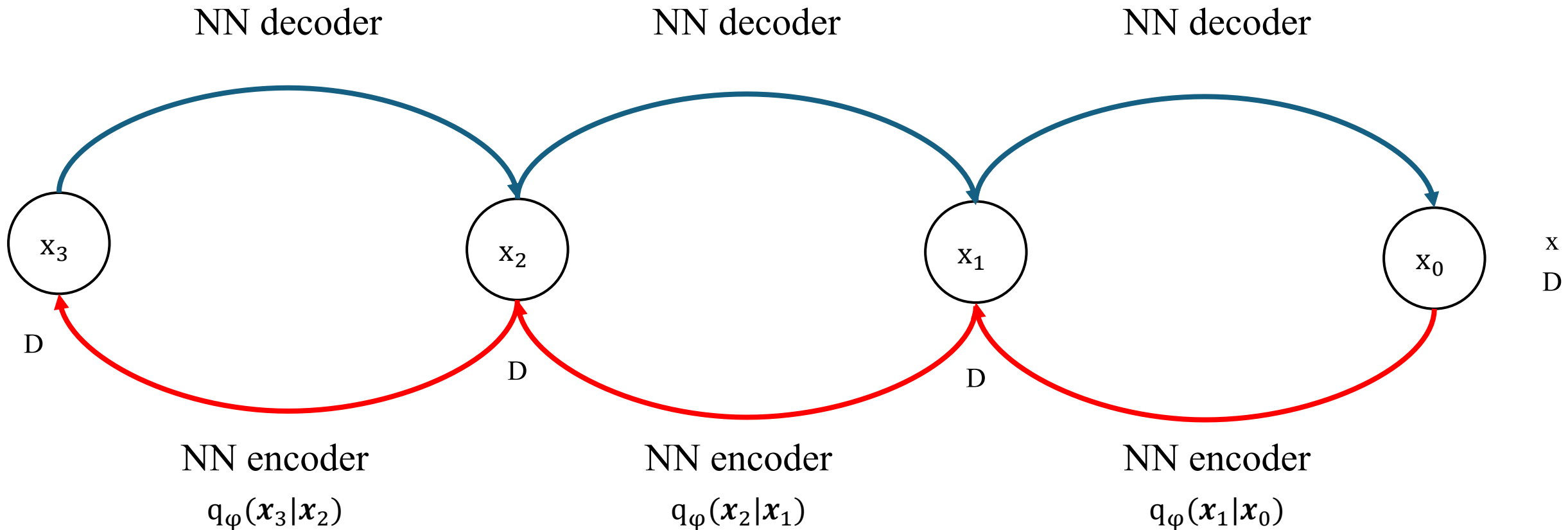
1) All latent variables have dimension D



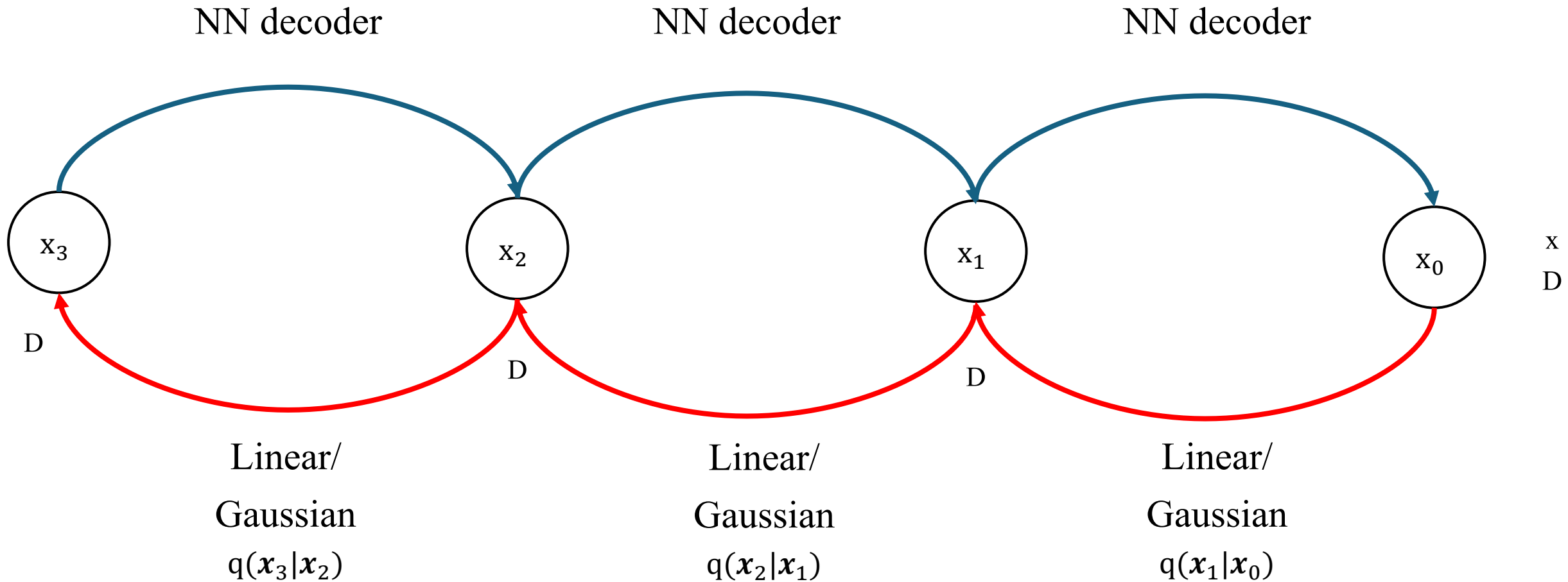
1.5) Let's call everything x instead of z



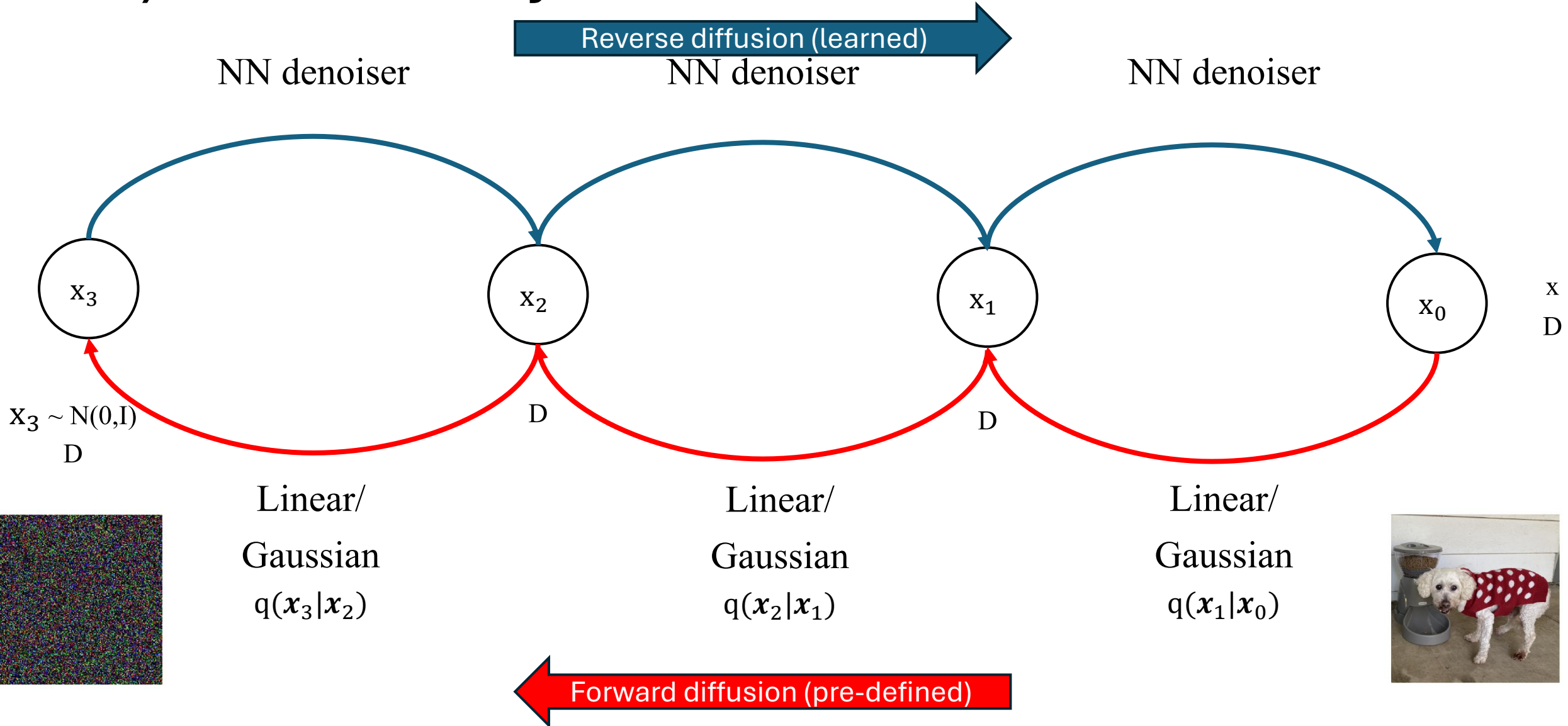
2) Every layer depends only on the previous one



3) Instead of encoder, add noise

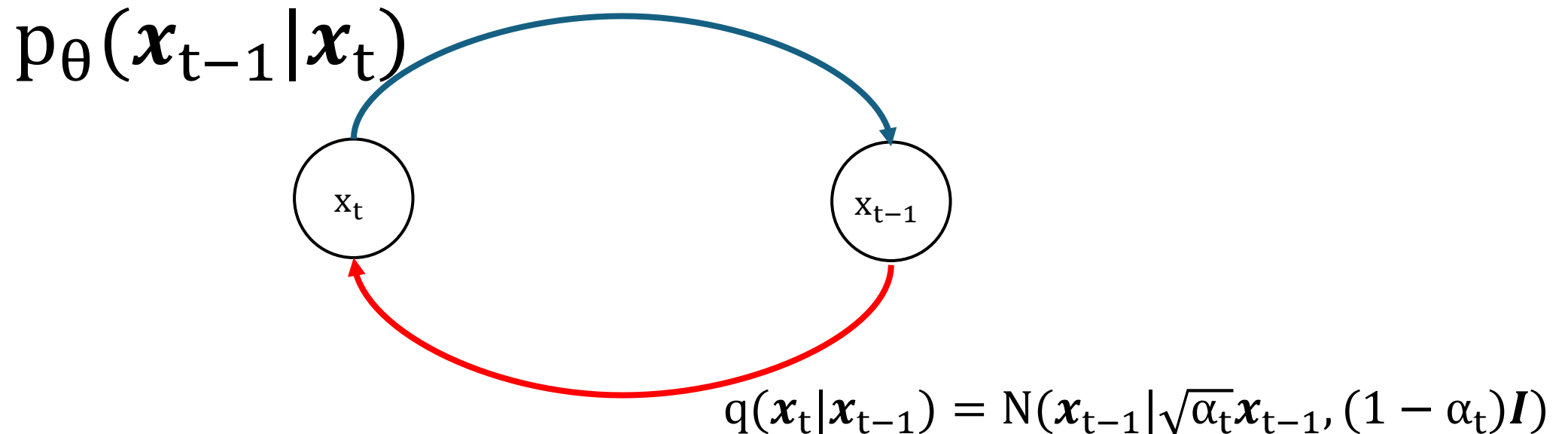


4) Decoders try to 'undo' the noise



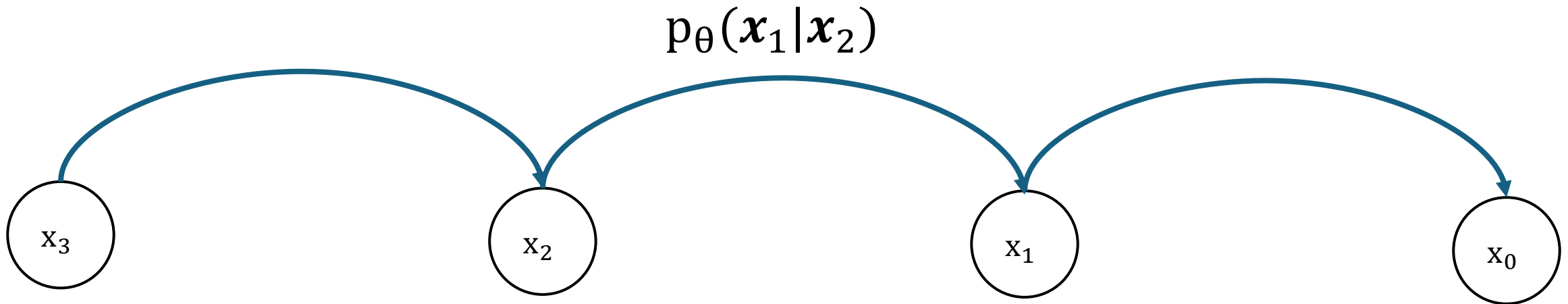
Diffusion model training

- Progressively add noise to input according to pre-defined $q(\mathbf{x}_t|\mathbf{x}_{t-1})$
 - Chose q 's so that $p(\mathbf{x}_T) = N(\mathbf{x}_T|0, \mathbf{I})$
- Train NNs to learn the inverse of each noising step



Using a trained diffusion model

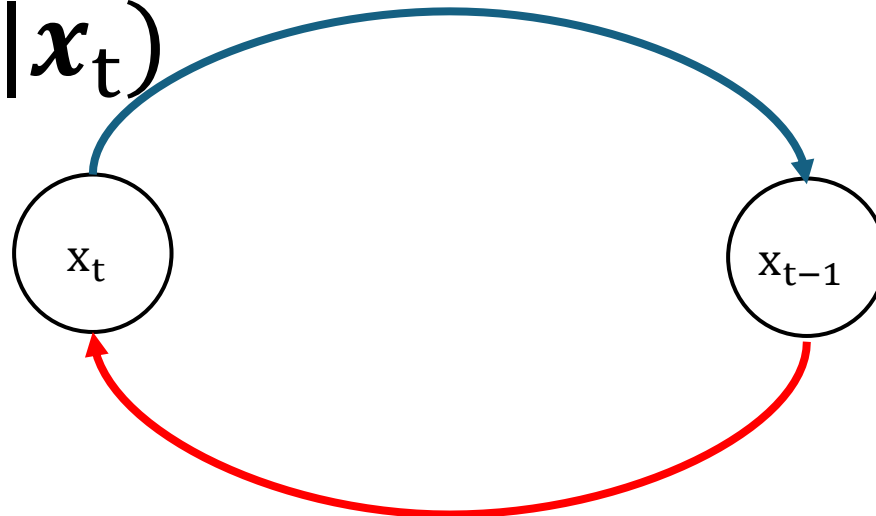
- Draw a random sample $\mathbf{x}_T$ from $N(0, I)$
- Apply reverse process:
 - Pass it iteratively from each network $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$
 - $\mathbf{x}_0$ is a sample from $p(\mathbf{x})$



Diffusion model training

- How to choose q
 - i.e. how to choose α_t
- How to train p

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)$$



$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = N(\mathbf{x}_{t-1} | \sqrt{\alpha_t} \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I})$$

How to choose q

- We need “enough” diffusion steps
 - so that x_T becomes a Gaussian
- Very big steps:
 - Diffusion converges quickly,
 - but each denoising step gets harder to learn
- Very small steps:
 - Denoising is easy,
 - but requires more steps, increasing computational cost

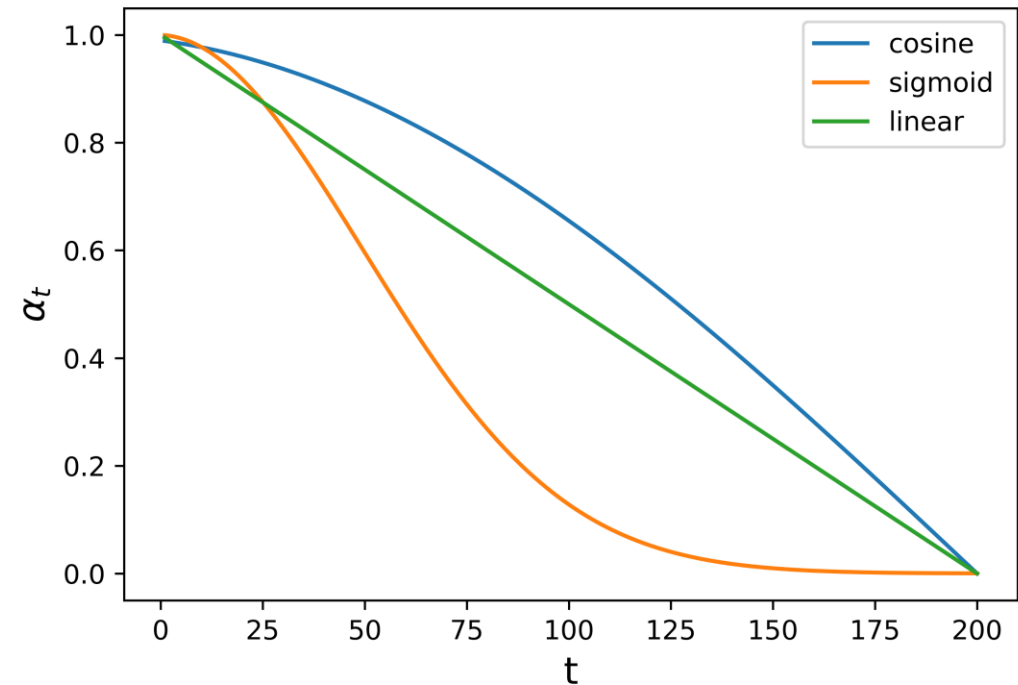
How to choose q

i.e. how to choose α_t

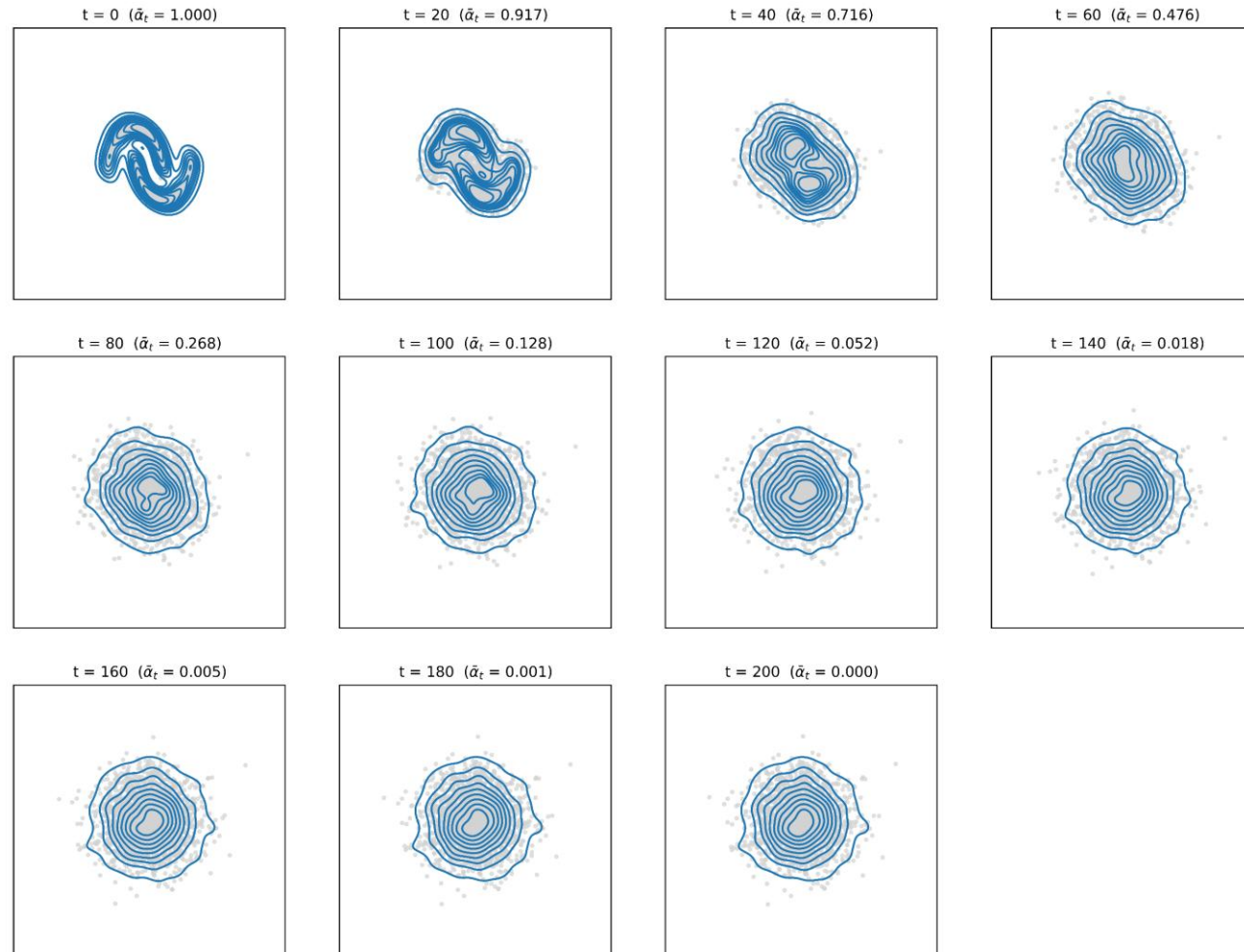
- Pre-define it
- Learn it during training

$$\alpha_t = \exp(-f(t))$$

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_{t-1} | \sqrt{\alpha_t} \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I})$$

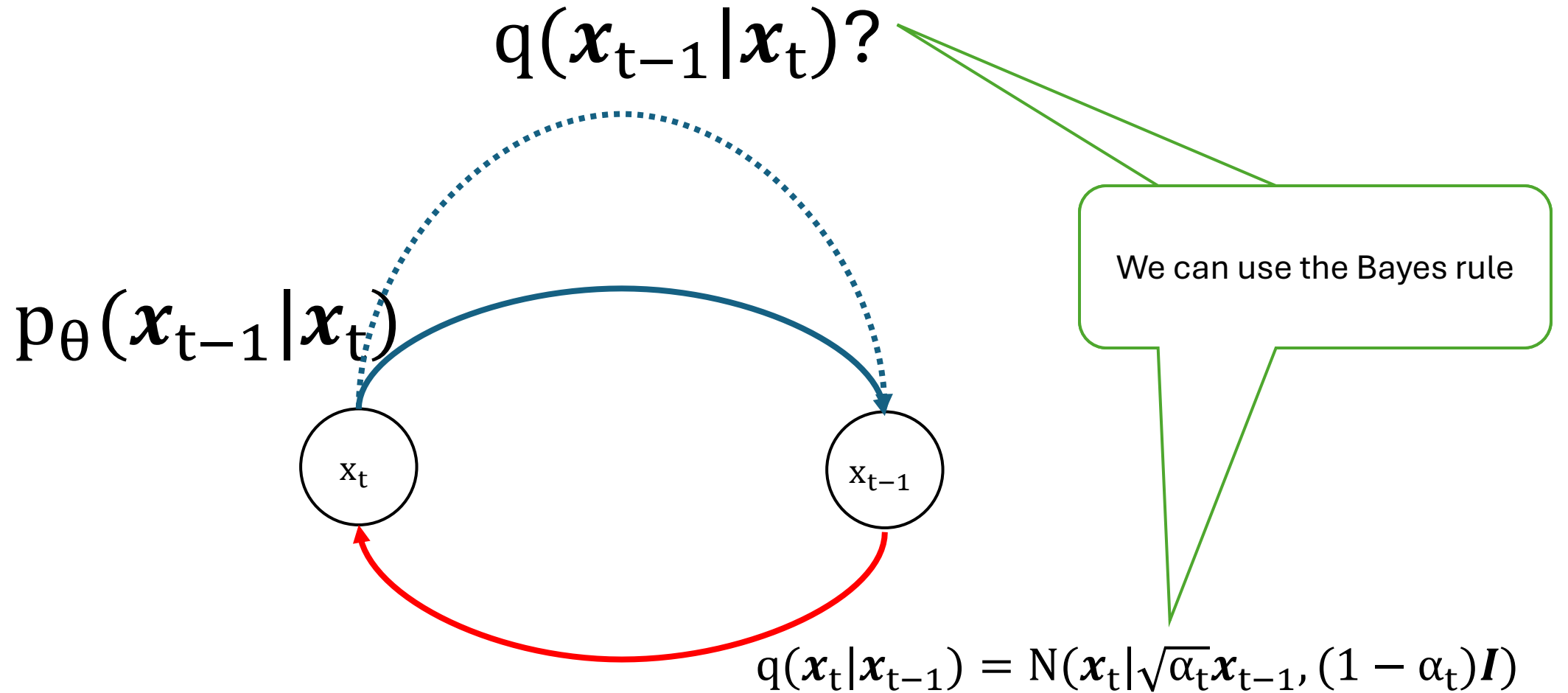


Example: linear schedule



How to choose p ?

What is the ground-truth denoising?



We can calculate this from original input $\mathbf{x}_0$

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}$$

Bayes' rule

$$= \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}$$

Markov property

forward diffusion

multiple forward
steps

What is $q(\mathbf{x}_t|\mathbf{x}_0)$?

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}$$

$$q(\mathbf{x}_t|\mathbf{x}_0) = q(\mathbf{x}_t|\mathbf{x}_{t-1}) \circ q(\mathbf{x}_{t-1}|\mathbf{x}_{t-2}) \circ \dots \circ q(\mathbf{x}_1|\mathbf{x}_0)$$

Applying t linear-Gaussian models:

$$q(\mathbf{x}_t|\mathbf{x}_0) = N(\mathbf{x}_t | \sqrt{\prod_{i=1}^t \alpha_i} \mathbf{x}_0, (1 - \prod_{i=1}^t \alpha_i) \mathbf{I})$$

Ugly, but easy to calculate

What is the ground-truth denoising?

Gaussian

Gaussian

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}$$

Gaussian

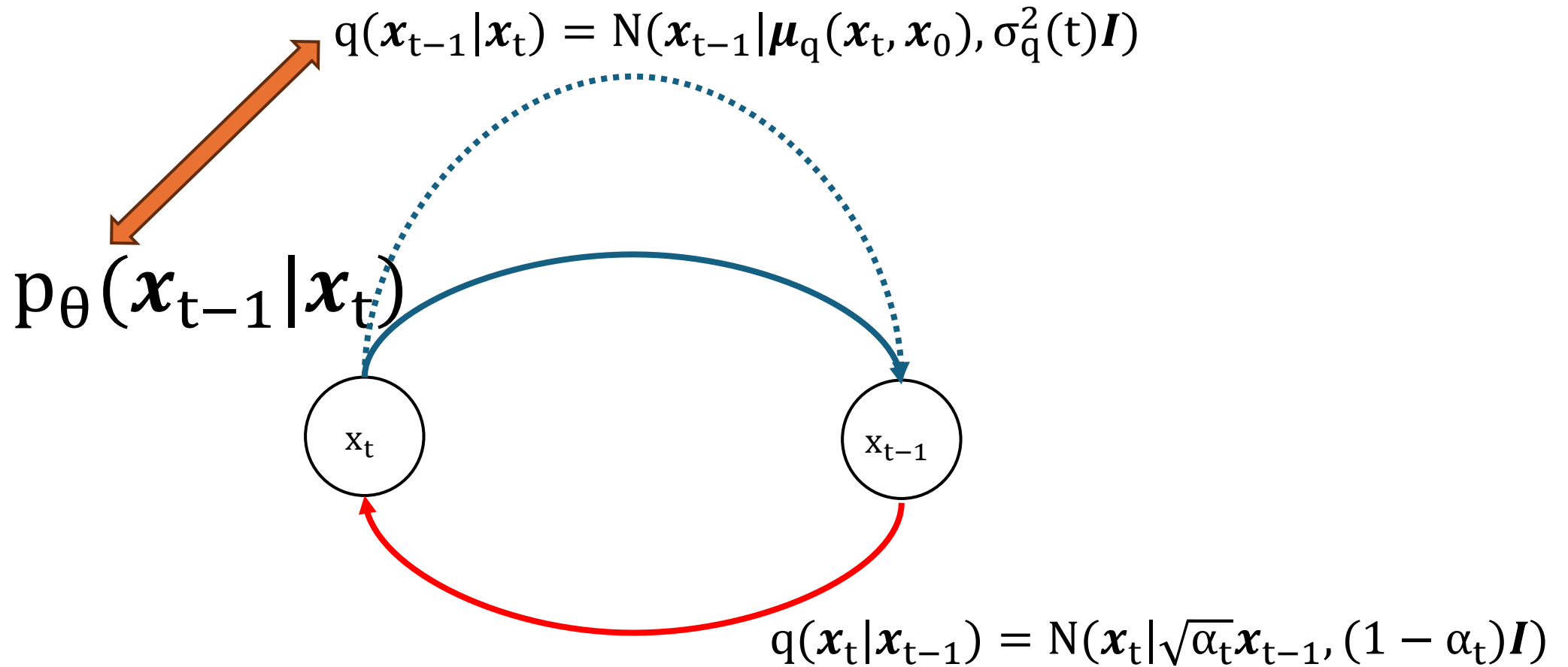
Also Gaussian

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = N(\mathbf{x}_{t-1}|\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0), \sigma_q^2(t)\mathbf{I})$$

Very ugly, but easy to
calculate
Depends on original
input $\mathbf{x}_0$ and t

Ugly, but easy to
calculate
Depends only on t !

Training intuition: make these two alike



How to train p

- With the ELBO!

HVAE notation: $\ln p(\mathbf{x}) \geq \mathbb{E}_{q_{\phi}(\mathbf{z}_{1:T}|\mathbf{x})} \left[\ln \frac{p(\mathbf{x}, \mathbf{z}_{1:T})}{q_{\phi}(\mathbf{z}_{1:T}|\mathbf{x})} \right]$

Diffusion notation: $\ln p(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\ln \frac{p(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right]$

Starting from the ELBO, after a lot of algebra

$$\ln p(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\ln \frac{p(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] =$$

Reconstruction
at first step

$$\mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} [\ln p_\theta(\mathbf{x}_0|\mathbf{x}_1)]$$

$$- D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) || p(\mathbf{x}_T))$$

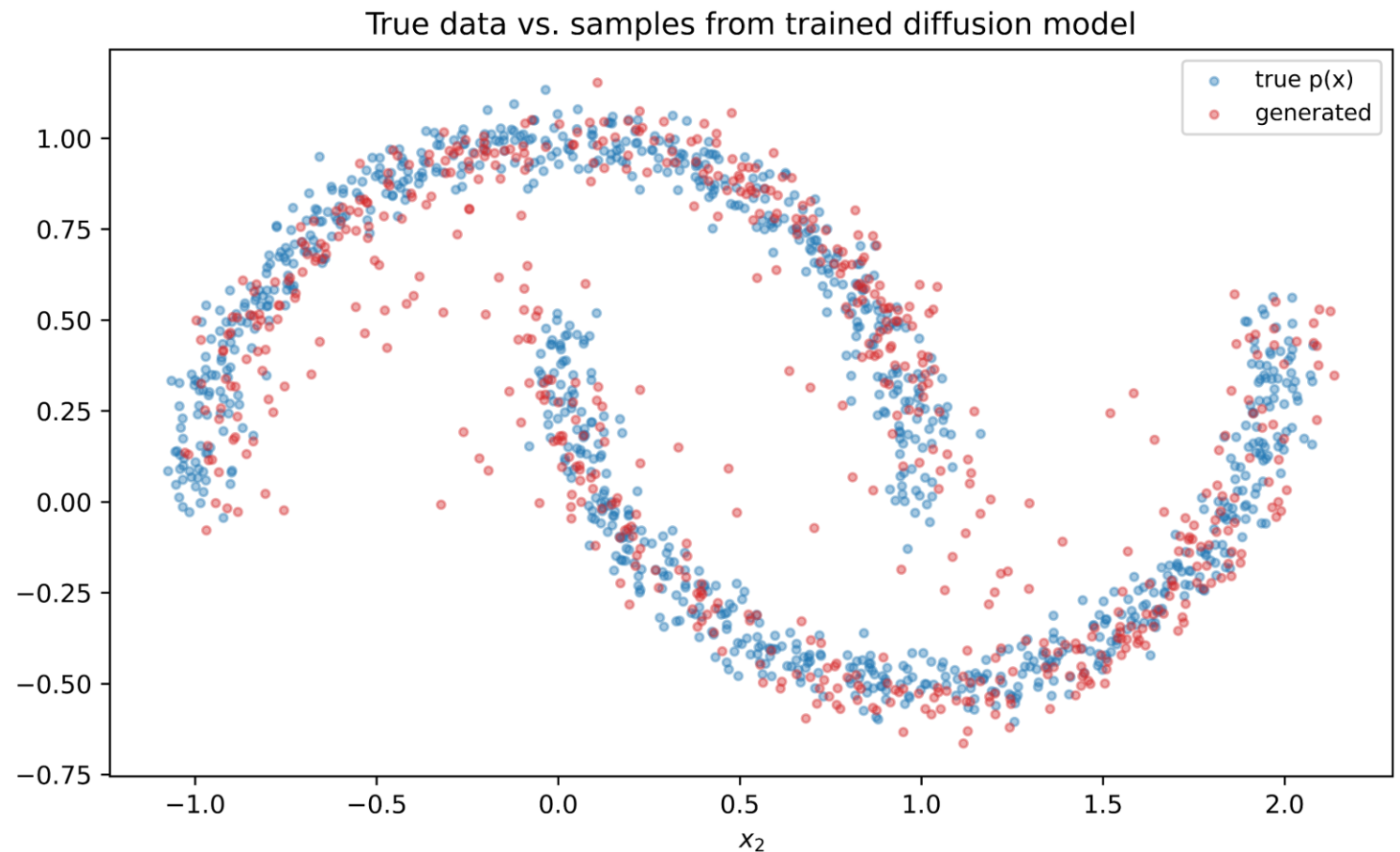
Forward process should
give Gaussian noise
Does not depend on θ

$$- \sum_{t=2}^T \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} [D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) || p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))]$$

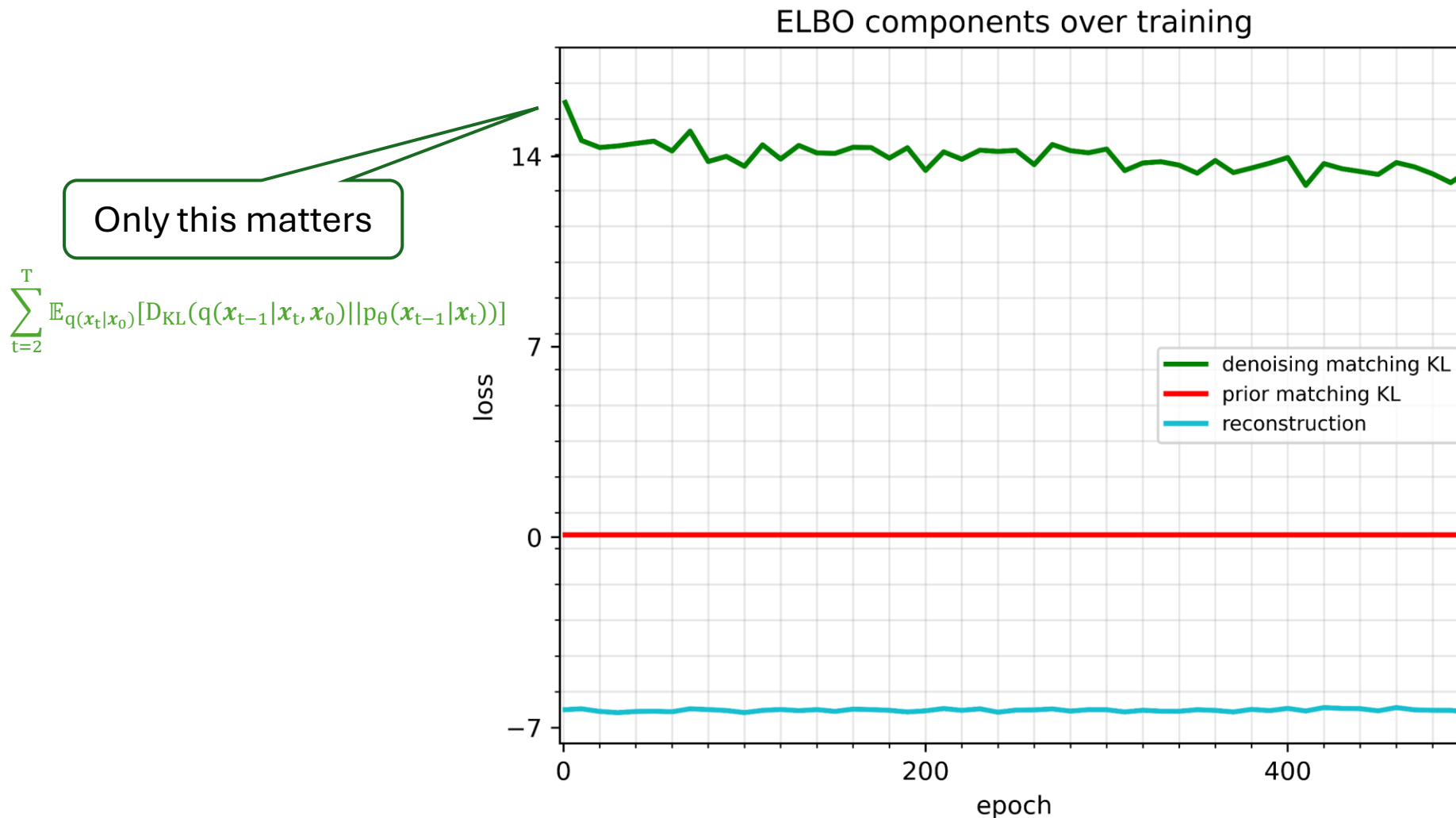
All denoising networks should learn the ground-truth denoising

Let's study these three terms during training

- 100 diffusion steps
- Denoise with
 - 2-layer MLP



Denoising KL term dominates the ELBO



Diffusion model training objective

So maximizing the ELBO boils down to minimizing

$$\sum_{t=2}^T \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} [D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) || p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t))]$$

Let's make this a Gaussian then!

$$= \sum_{t=2}^T \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} [D_{\text{KL}}(\mathcal{N}(\mathbf{x}_{t-1} | \boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0), \sigma_q^2(t)\mathbf{I}) || p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t))]$$

Diffusion model training objective

So maximizing the ELBO boils down to minimizing

$$\sum_{t=2}^T \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} [D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) || p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t))]$$

$$= \sum_{t=2}^T \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} [D_{\text{KL}}(N(\mathbf{x}_{t-1} | \boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0), \sigma_q^2(t)\mathbf{I}) || \textcolor{red}{N(\mathbf{x}_{t-1} | \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \sigma_q^2(t)\mathbf{I})})]$$

Use NN to learn this
NN should not see $\mathbf{x}_0$

Does not depend on $\mathbf{x}$
Set it to correct value

Diffusion model training objective

$$D_{\text{KL}} = \frac{1}{2\sigma_q^2(t)} \|\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t)\|^2$$

- $\boldsymbol{\mu}_q$ is a linear combination of $\mathbf{x}_t, \mathbf{x}_0$

- KL can be re-written as $\sim \|\mathbf{x}_0 - \hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)\|^2$

Reconstruct original image
from noisy version

OR

- $\sim \|\boldsymbol{\varepsilon}_0 - \hat{\boldsymbol{\varepsilon}}_\theta(\mathbf{x}_t, t)\|^2$

Predict the noise that
transformed $\mathbf{x}_0$ to $\mathbf{x}_t$

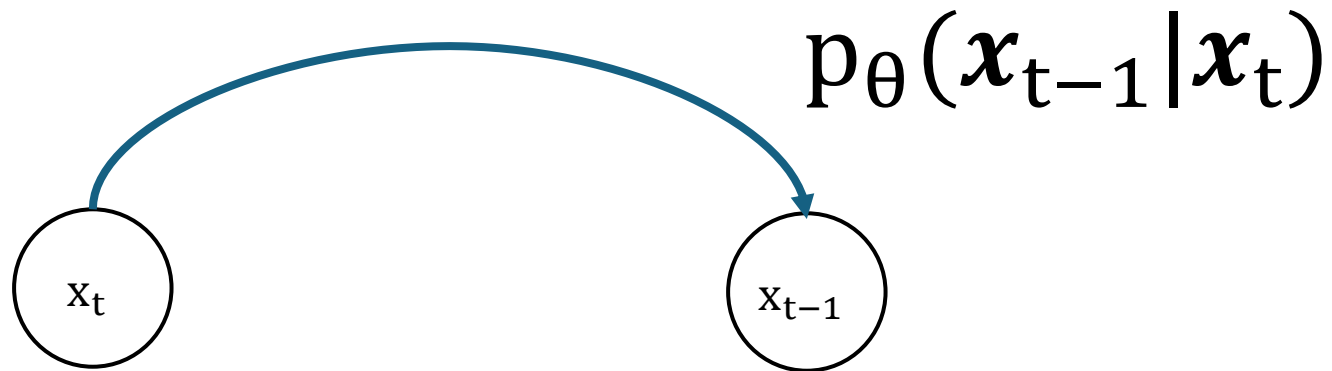
Diffusion model training objective

$$\sum_{t=2}^T \frac{1}{2\sigma_q^2(t)} \|\mu_q(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)\|^2$$

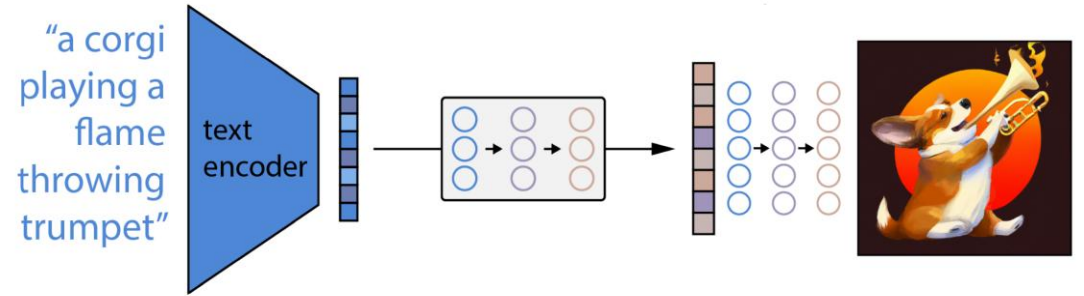
- In practice, during training you only do this for 1 random t
 - Different t for each sample in each batch

Using a trained diffusion model revisited

- Draw a random sample $\mathbf{x}_T$ from $N(0, \mathbf{I})$
- Apply reverse process:
 - Pass it iteratively from each denoising network $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$
 - But this predicts the mean of the denoising distribution
 - Add random noise $\sim N(\mathbf{0}, \sigma_q^2(t)\mathbf{I})$
 - $\mathbf{x}_0$ is a sample from $p(\mathbf{x})$



Conditional generation



- “antibody that binds to SARS-COV2 protein”
- Condition to a discrete quantity
 - e.g. class label (cat)
- Or continuous
 - e.g. text embedding vector (from prompt)

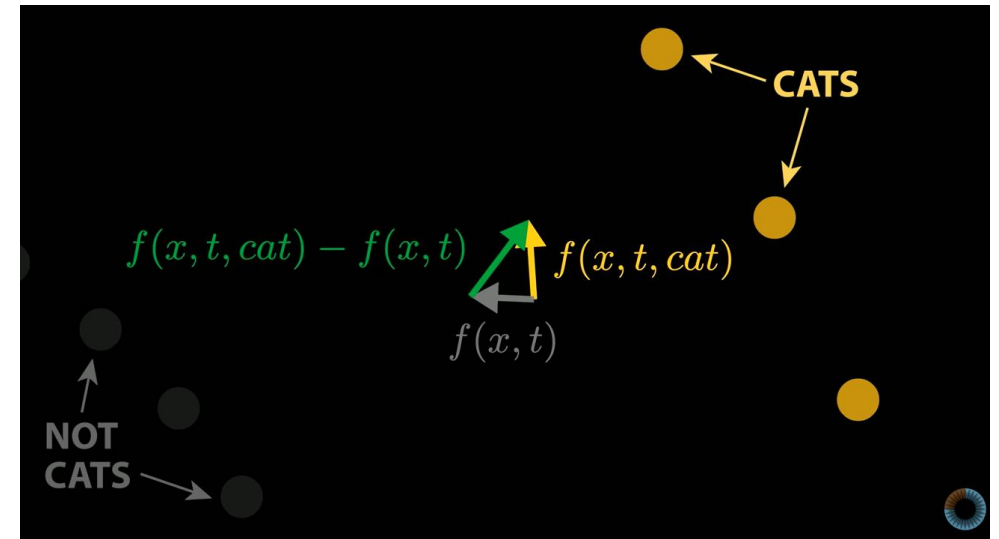
Classifier-free guidance

- At each step, learn two types of denoising:
 - Standard (unconditional): $\|\epsilon_0 - \hat{\epsilon}_\theta(\mathbf{x}_t, t)\|^2$
 - Conditioned on variable y : $\|\epsilon_0 - \hat{\epsilon}_\theta(\mathbf{x}_t, t, y)\|^2$
- Final predicted noise is $(1 + w)\hat{\epsilon}_\theta(\mathbf{x}_t, t, y) - w \cdot \hat{\epsilon}_\theta(\mathbf{x}_t, t)$

Classifier-free guidance

$$(1 + w)\hat{\epsilon}_{\theta}(\mathbf{x}_t, t, \mathbf{y}) - w \cdot \hat{\epsilon}_{\theta}(\mathbf{x}_t, t)$$

- When t is large (noise is high)
 - (away from true data manifold)
 - the two terms are similar
 - Denoising points towards $p(\mathbf{x}_0)$ as before
- When t is small (noise is low)
 - (close to true data manifold)
 - Denoising points towards the difference

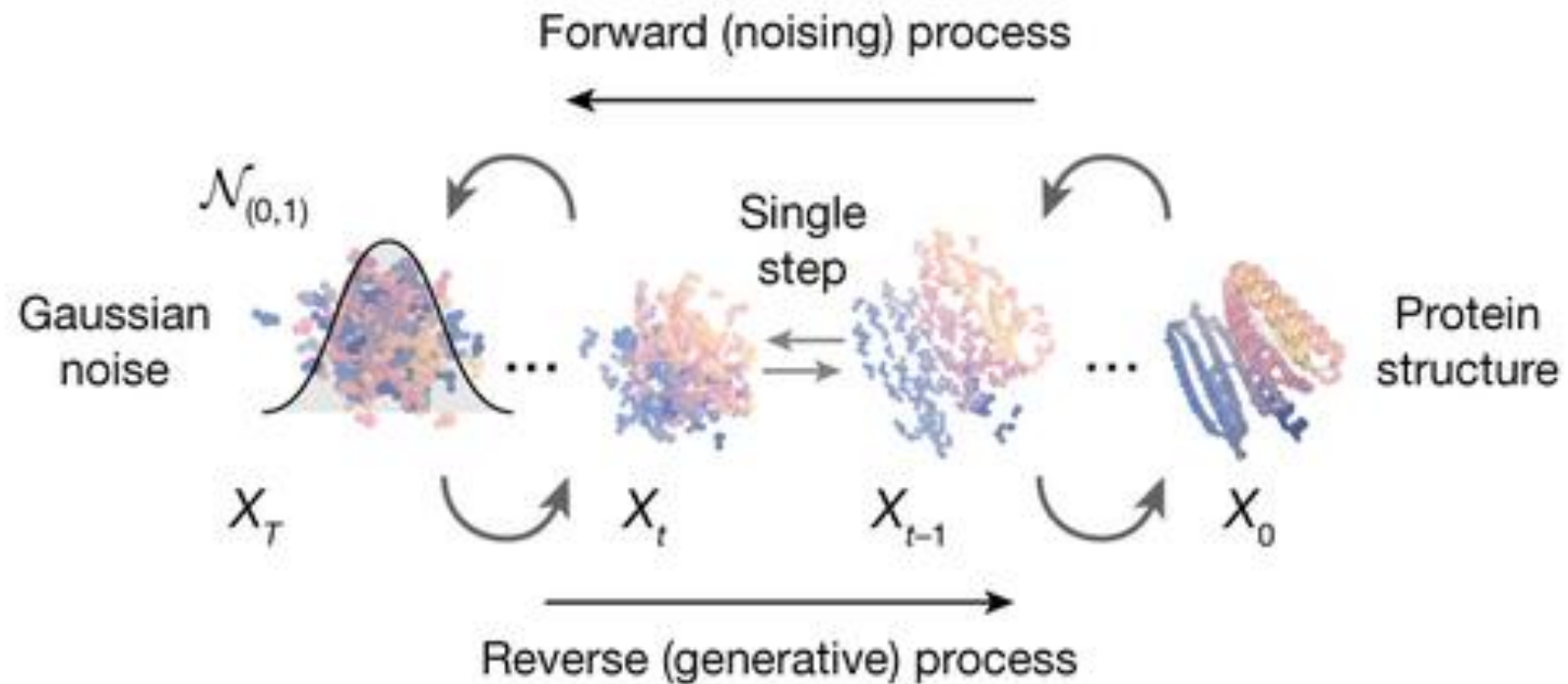


We can do this with one denoising network

- Represent unconditional with special token in conditional network
 - $\hat{\epsilon}_{\theta}(\mathbf{x}_t, t) = \hat{\epsilon}_{\theta}(\mathbf{x}_t, t, \emptyset)$
- For 1-hot encoded classes:
 - A vector of 0's for unconditional

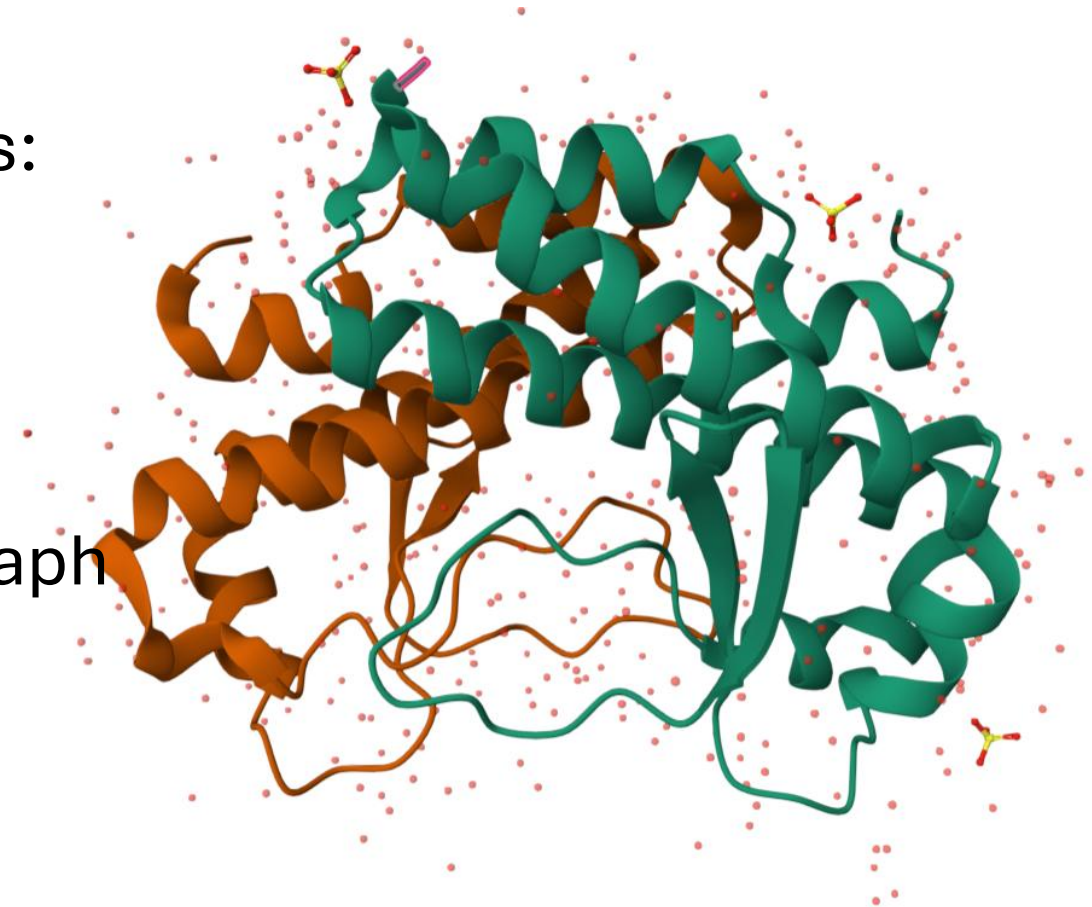
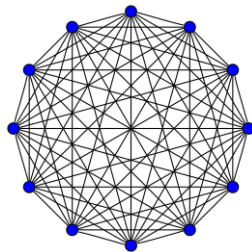
Example: Rfdiffusion for protein structures

Diffusion model



RFdiffusion: input data

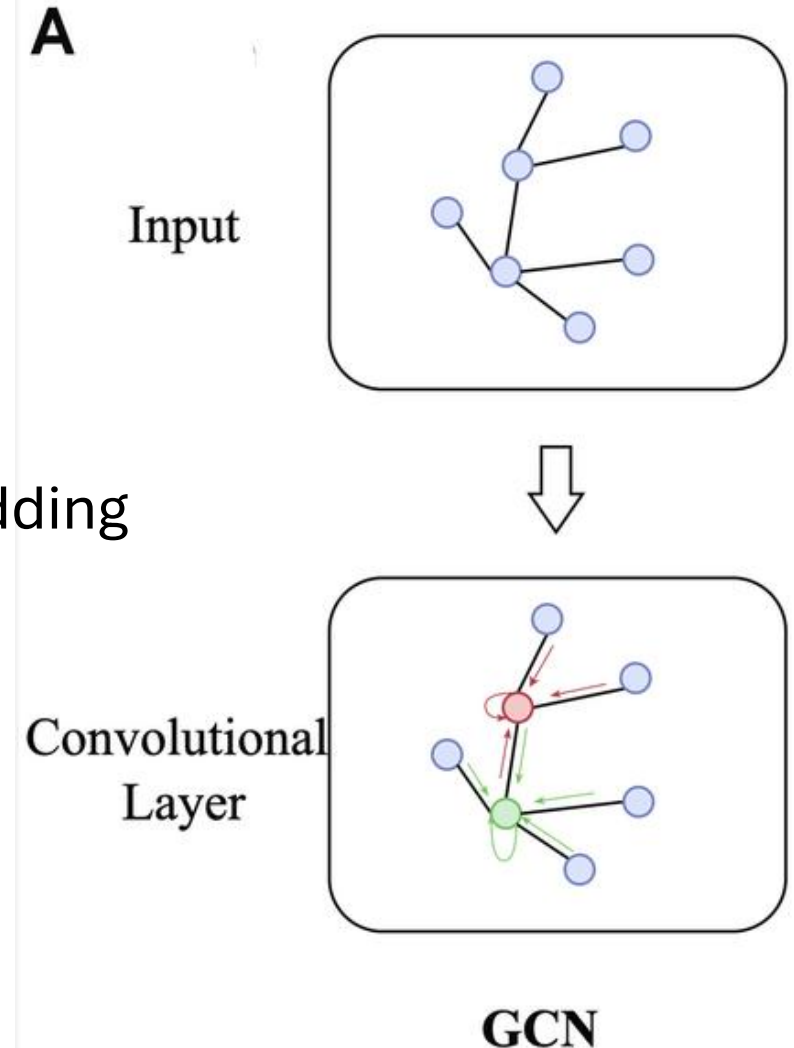
- Sequence of residues (amino acids)
- For each residues, two sets of features:
 - Translation: x,y,z co-ordinates
 - Where is this residue in space?
 - Rotation: a 3x3 rotation matrix
 - How is the residue oriented in space?
- Residues: edges of fully-connected graph
 - With edge features



Denoising layers: Graph neural networks

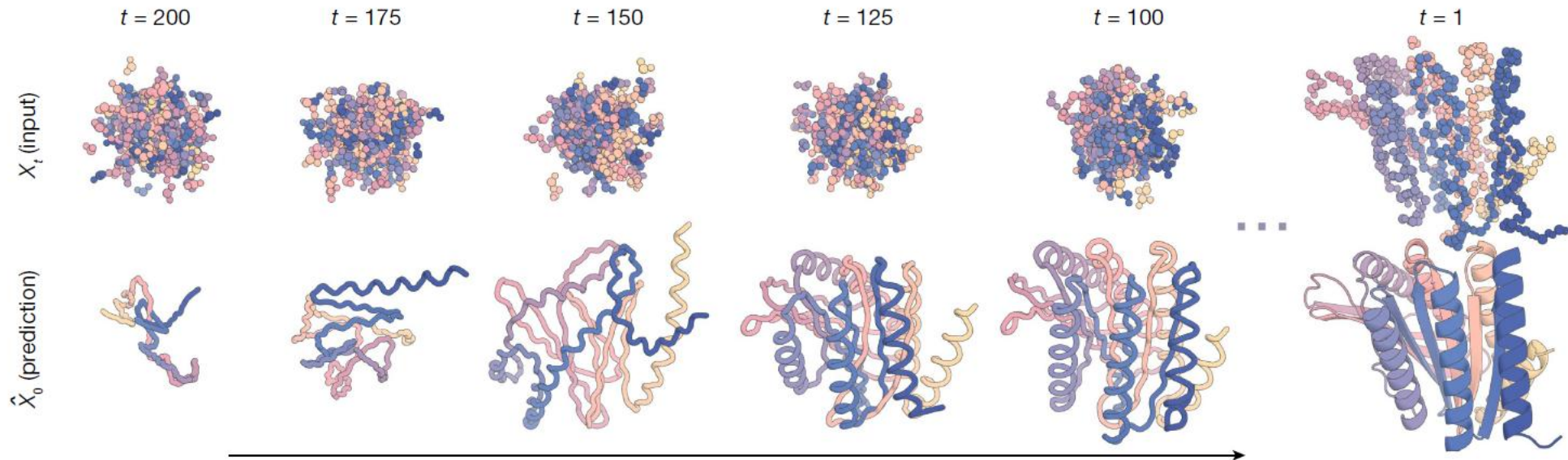
- For each node:
 - Calculate a 'message' vector from each neighbor
 - based on node and edge features
 - Weigh importance of each message
 - using an attention mechanism
 - Add all weighted messages to current node embedding

A lot more constraints and domain-specific knowledge



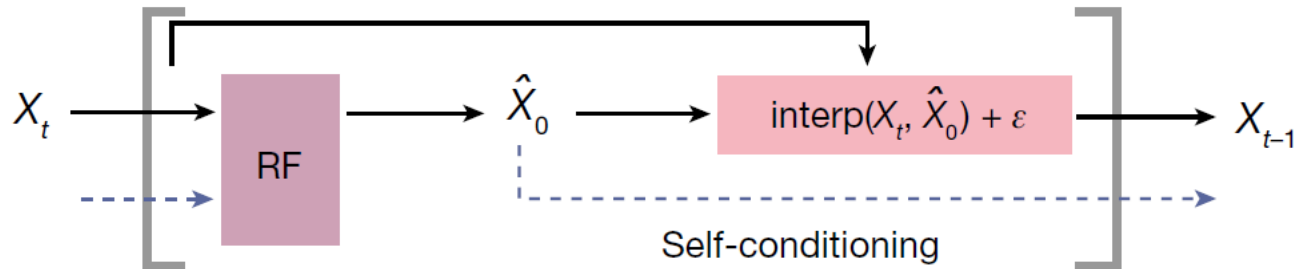
Training by predicting $\mathbf{x}_0$

- Remember $KL \sim \|\mathbf{x}_0 - \hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)\|^2$
- We can get $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ by passing $\hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)$ through t forward steps
 - Because everything is linear/Gaussian

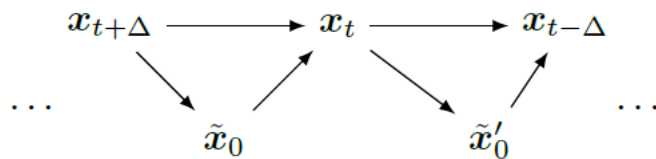


Self-conditioning

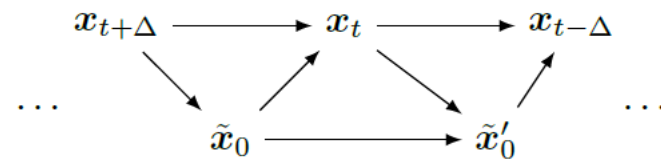
Single RFdiffusion step



- Use predicted at $\mathbf{x}_0$ at next step
- Transfers information between layers more efficiently
 - Better generation



(a) Standard reverse diffusion steps.



(b) Self-Conditioning on the previous x_0 estimate.

Summary part 3

- Diffusion is a special HVAE
 - with no dimensionality reduction
 - and pre-defined, linear encoders
- Training a diffusion model means learning to reconstruct a sample from a noisy version of it
- Training with and without label at the same time allows for conditional generation



Generative Models 4

Normalizing Flows and other generative models

Stavros Makrodimitris

s.makrodimitris@uva.nl



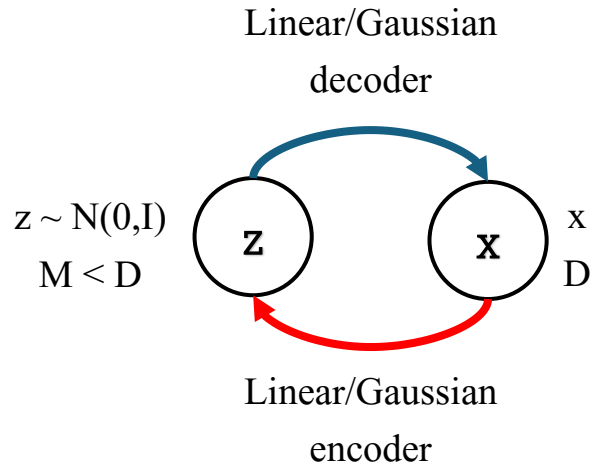
UNIVERSITY OF AMSTERDAM
Swammerdam Institute for Life Sciences

Learning goals

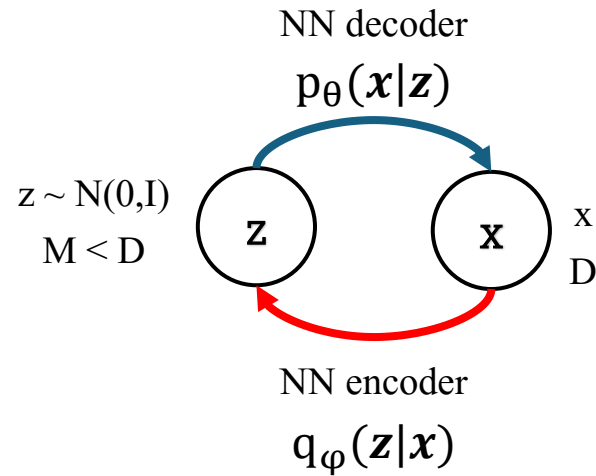
- Understand how a normalizing flow is trained
- Appreciate the connection between diffusion and flow-based models
- Recognize other important generative models

Types of generative models

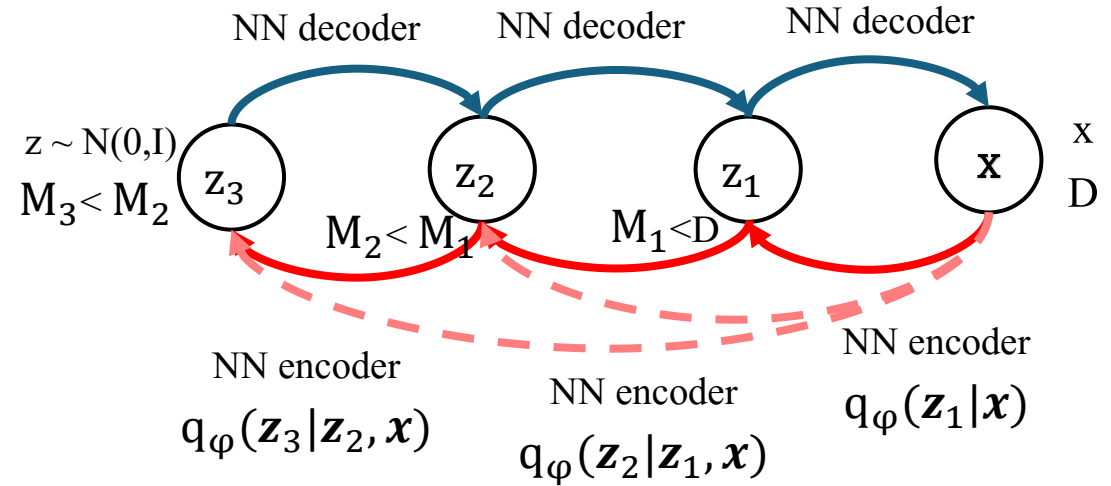
probabilistic PCA



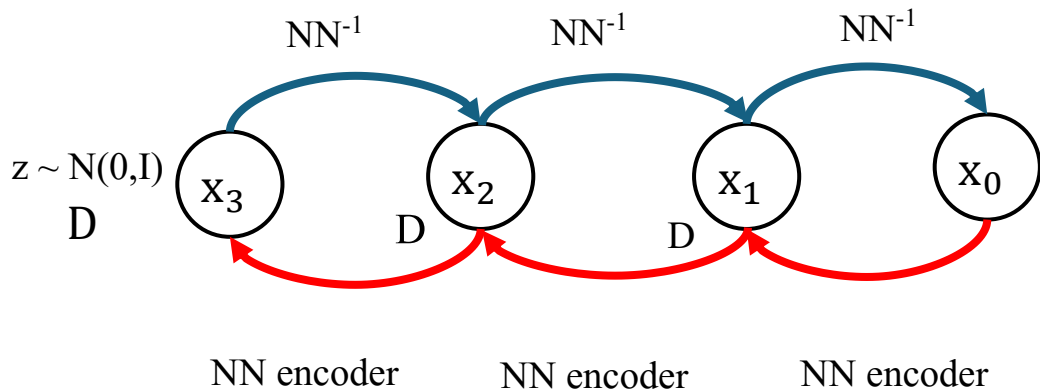
VAE



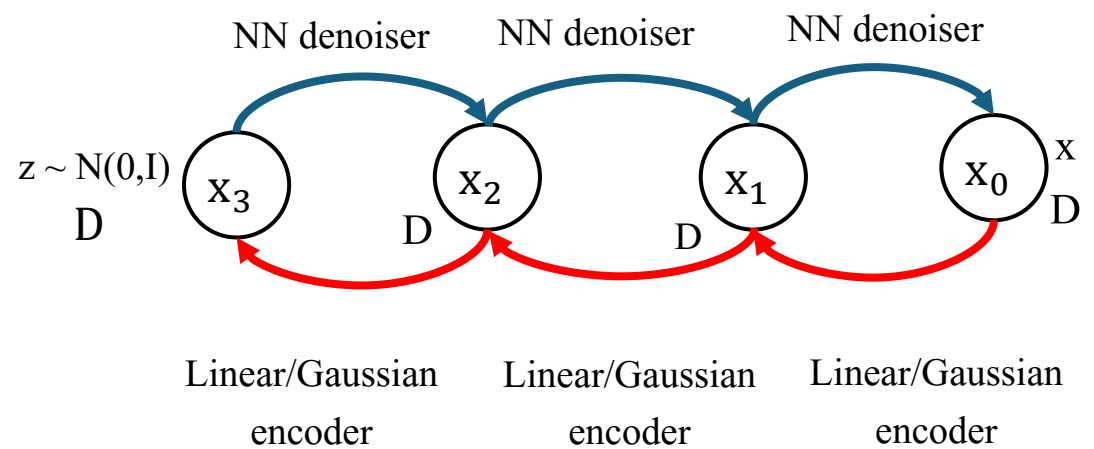
HVAE



Normalizing Flows



Diffusion



Normalizing flow

- Convert target distribution to Gaussian
- Using non-linear, invertible transformations
- Inverse transformation can be used to generate new data

Normalizing flow

- Latent variables $\mathbf{z}$

$$p(\mathbf{z}) = N(\mathbf{z}|\mathbf{0}, \mathbf{I})$$

- Target complex distribution $p(\mathbf{x})$
- Invertible function $\mathbf{z} = f(\mathbf{x}), \mathbf{x} = f^{-1}(\mathbf{z})$

$$p_X(\mathbf{x}) = p_Z(f(\mathbf{x})) \left| \det\left(\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}\right) \right|$$

Let's convince ourselves this makes sense

If X in 1-D and f increasing:

$$F_X(x) = P(X \leq x)$$

CDF definition

$$= P(f^{-1}(Z) \leq x)$$

$Z=f(X)$

$$= P(Z \leq f(x))$$

Apply f on both sides

$$= F_Z(f(x))$$

CDF definition

$$p_X(x) = p_Z(f(x)) \frac{df(x)}{dx}$$

Derivative wrt x

Back to normalizing flows

- Latent variables $\mathbf{z}$

$$p(\mathbf{z}) = N(\mathbf{z}|\mathbf{0}, I)$$

- Target complex distribution $p(\mathbf{x})$
- Invertible function $\mathbf{z} = f(\mathbf{x}), \mathbf{x} = f^{-1}(\mathbf{z})$

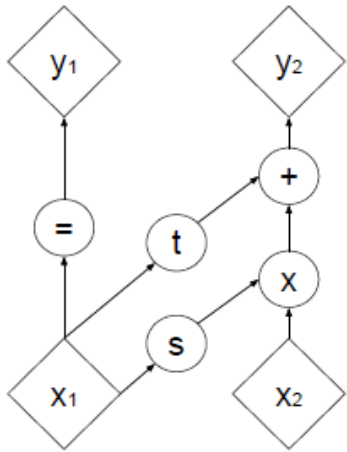
$$p_X(\mathbf{x}) = p_Z(f(\mathbf{x})) \left| \det\left(\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}\right) \right|$$

What should function f be?

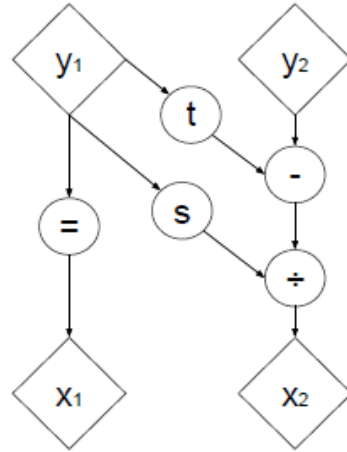
$$p_X(\mathbf{x}) = p_Z(f(\mathbf{x})) \left| \det\left(\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}\right) \right|$$

- Non-linear
- Invertible
- Easy to calculate derivatives

RealNVP



(a) Forward propagation



(b) Inverse propagation

- In each layer, split $\mathbf{x}$ into two parts
 - One is changed
 - Other is constant

1	2	5	6
3	4	7	8

4	8		
	3	7	
		2	6
		1	5

RealNVP

$$y_1 = x_1$$

$$y_2 = x_2 \odot \exp(s(x_1)) + t(x_1)$$

s, t : *neural networks*

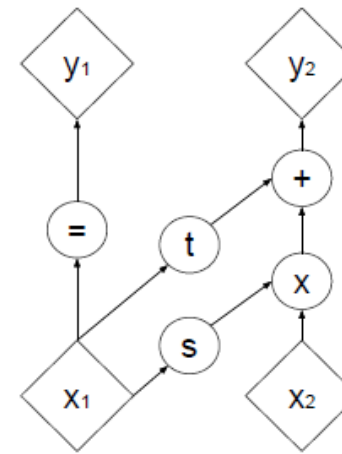
Inverse:

$$x_1 = y_1$$

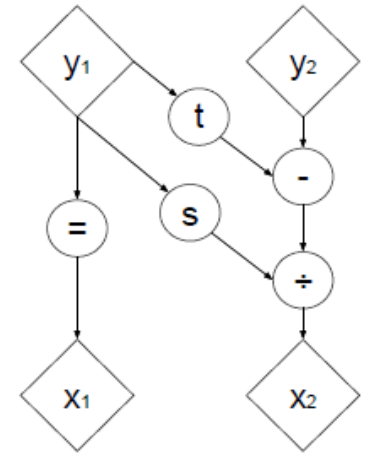
$$x_2 = (y_2 - t(y_1)) \odot \exp(-s(y_1))$$

Gradient determinant:

$$\exp(\sum s(x_1))$$



(a) Forward propagation

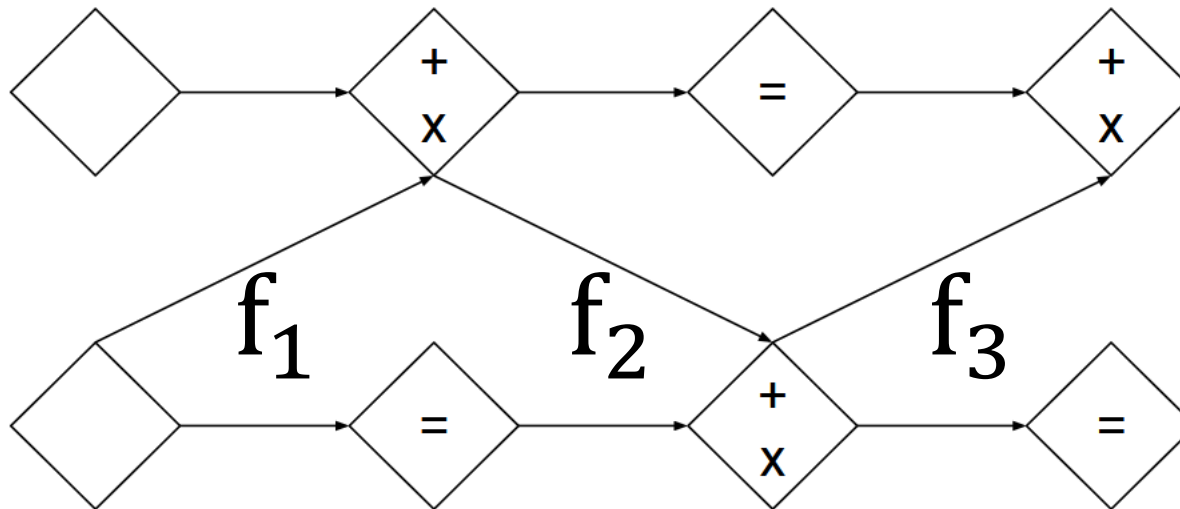


(b) Inverse propagation

Stacking multiple layers is also easy

$$(f_1 \circ f_2)^{-1} = f_2^{-1} \circ f_1^{-1}$$

$$\det\left(\frac{\partial(f_1 \circ f_2)(x)}{\partial x}\right) = \det\left(\frac{\partial f_1(x)}{\partial x}\right) \det\left(\frac{\partial f_2(f_1(x))}{\partial x}\right)$$

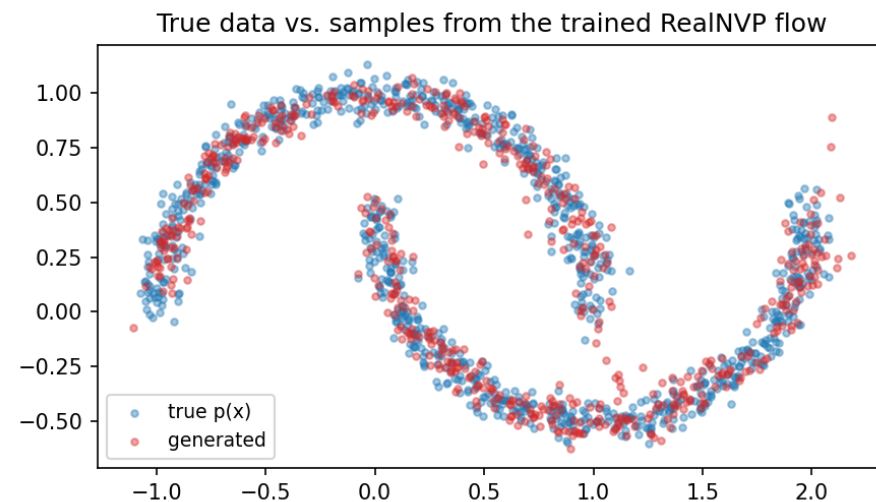
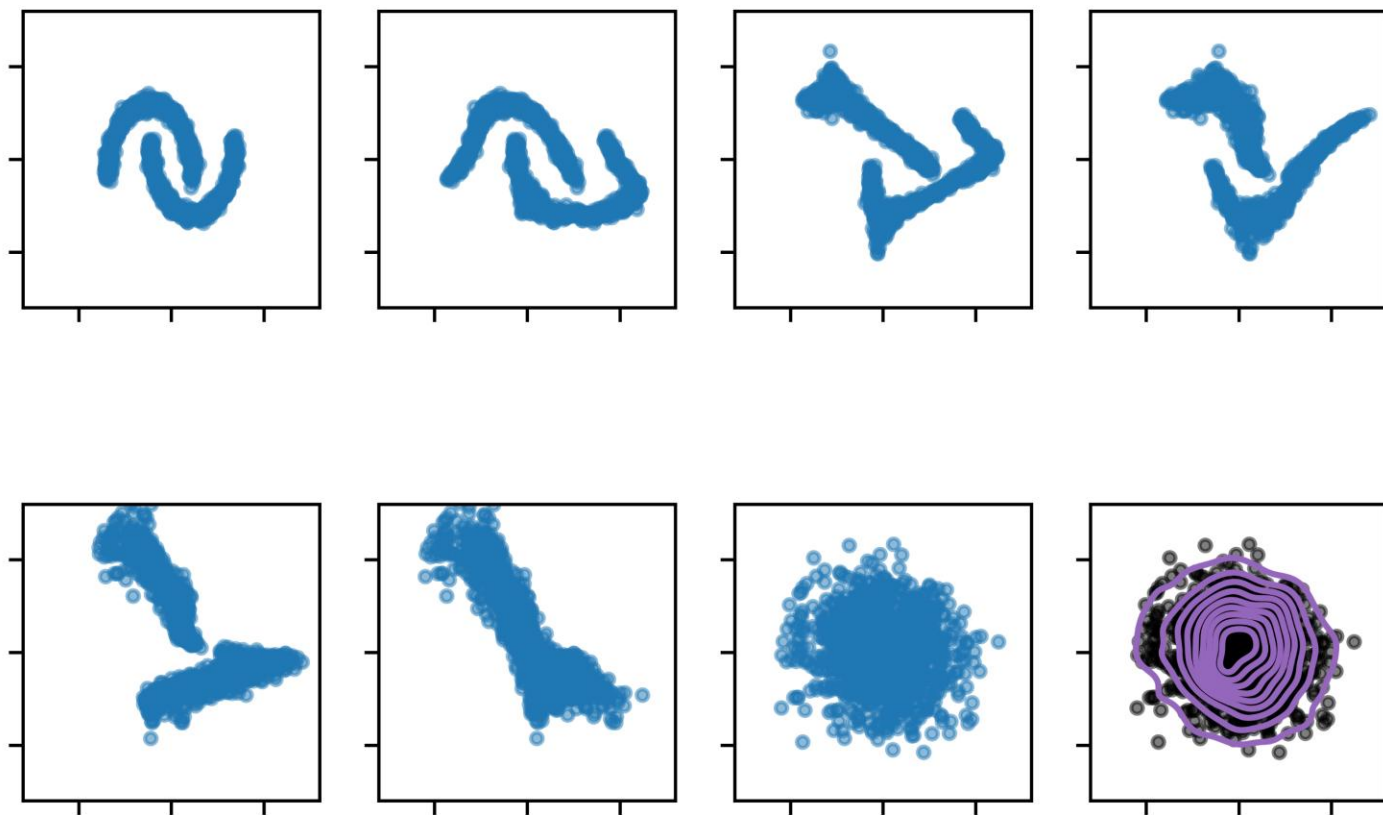


Train by maximum likelihood

$$p_X(\mathbf{x}) = p_Z(f(\mathbf{x})) \left| \det\left(\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}\right) \right|$$

$$\ln p(\mathbf{x}) = \sum_i \ln(p_Z(f(\mathbf{x}_i))) + \sum_i \ln\left(\left| \det\left(\frac{\partial f(\mathbf{x}_i)}{\partial \mathbf{x}}\right) \right|\right)$$

RealNVP transformations



Other types of GMs

Recall: diffusion models

- Maximizing the ELBO means minimizing

- $\|\mathbf{x}_0 - \hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)\|^2$

Reconstruct original image
from noisy version

- $\|\boldsymbol{\varepsilon}_0 - \hat{\boldsymbol{\varepsilon}}_\theta(\mathbf{x}_t, t)\|^2$

Predict the noise that
transformed $\mathbf{x}_0$ to $\mathbf{x}_t$

- Turns out, this noise is related to the gradient of $p(\mathbf{x})$

$$\boldsymbol{\varepsilon}_0 = - \sqrt{1 - \prod_{m=1}^t \alpha_m} \nabla \ln p(\mathbf{x}_t)$$

Yet another formulation of diffusion

- Maximizing the ELBO means minimizing

- $\|\mathbf{x}_0 - \hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)\|^2$

- $\|\boldsymbol{\varepsilon}_0 - \hat{\boldsymbol{\varepsilon}}_\theta(\mathbf{x}_t, t)\|^2$

- $\|\nabla \ln p(\mathbf{x}_t) - \hat{\mathbf{s}}_\theta(\mathbf{x}_t, t)\|^2$

Predict the gradient of the
likelihood
(also called the **score**)

- Denoising = “moving” towards region with highest likelihood

Denoising as a differential equation

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \beta_t \left[\frac{1}{2} \mathbf{x}_t + \hat{\mathbf{s}}_\theta(\mathbf{x}_t, t) \right] \Delta t + \sqrt{\beta_t \Delta t} \boldsymbol{\varepsilon}$$

Subtract mean of
added noise from
 $\mathbf{x}_t$

Add random
noise

$$\beta_t = 1 - \alpha_t$$

What did we gain from this?

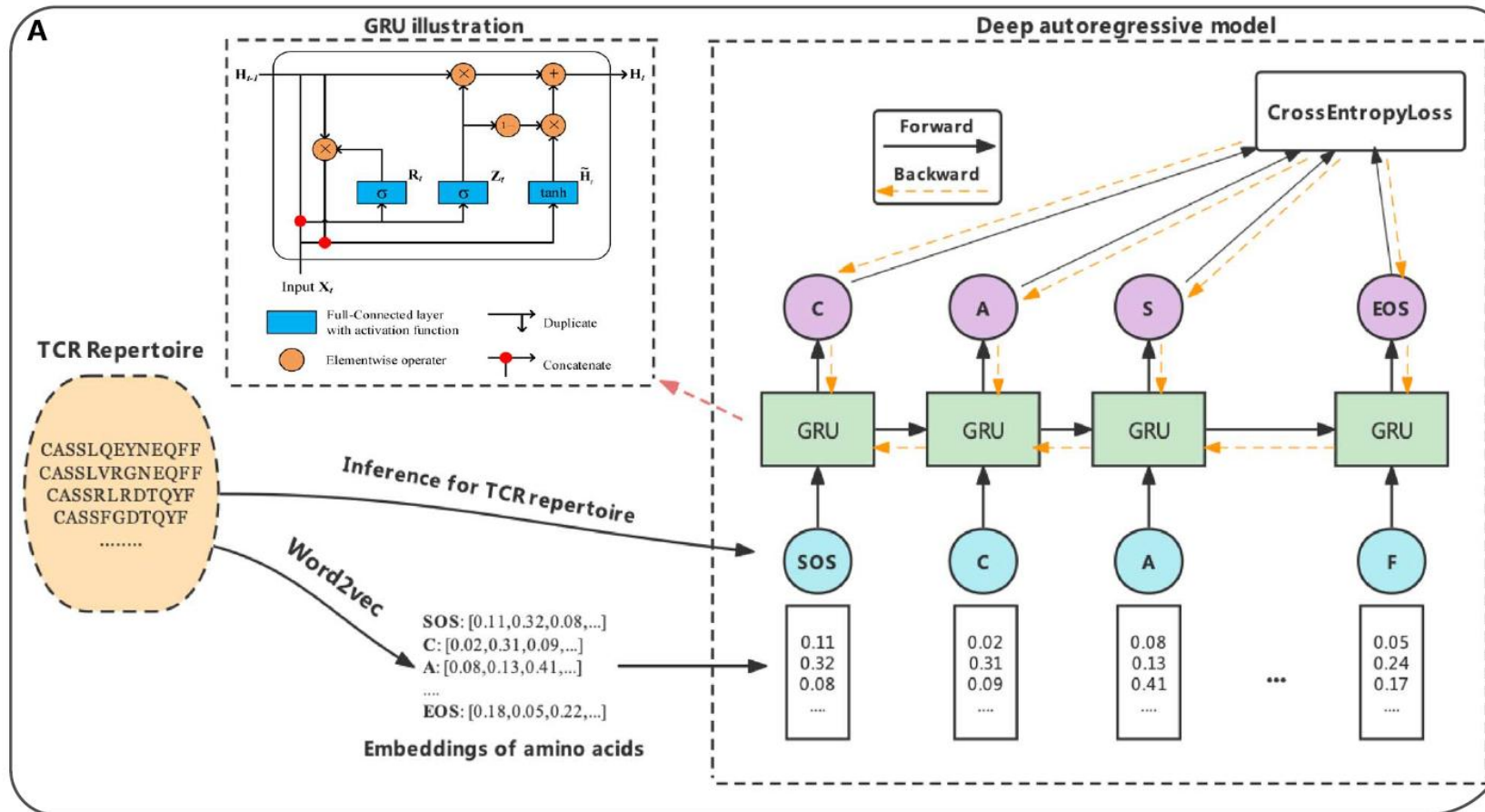
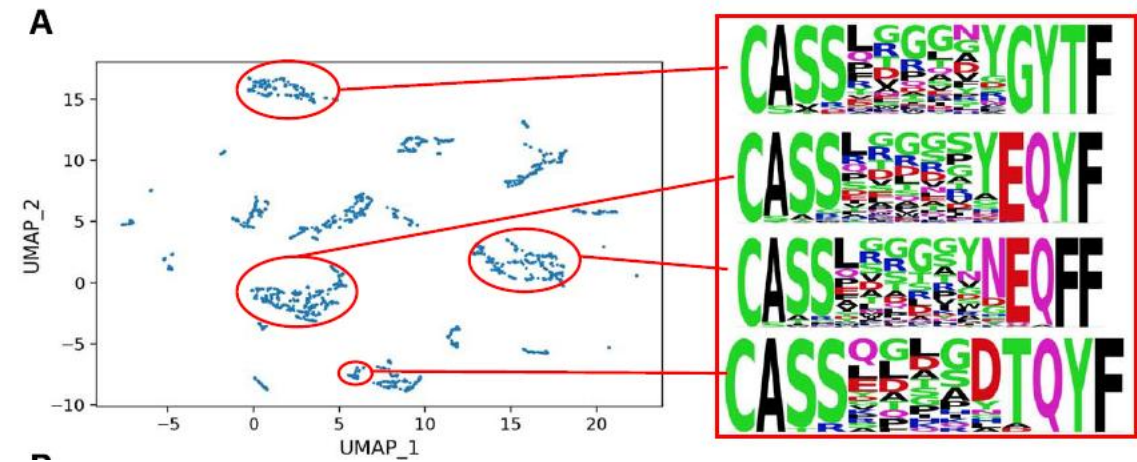
- We can use specialized solvers for more efficient generation
- We can think in continuous time
 - And sample however we want and at $t=1,2,3\dots$
 - More flexibility during generation
- This is a form of continuous normalizing flow
 - provides a link between diffusion and score-based generative models
 - allows calculation of likelihood (instead of ELBO) from change of variables
- We can extend this to other differential equations rather than diffusion
 - e.g. optimal transport $\rightarrow$ Flow Matching

Auto-regressive models

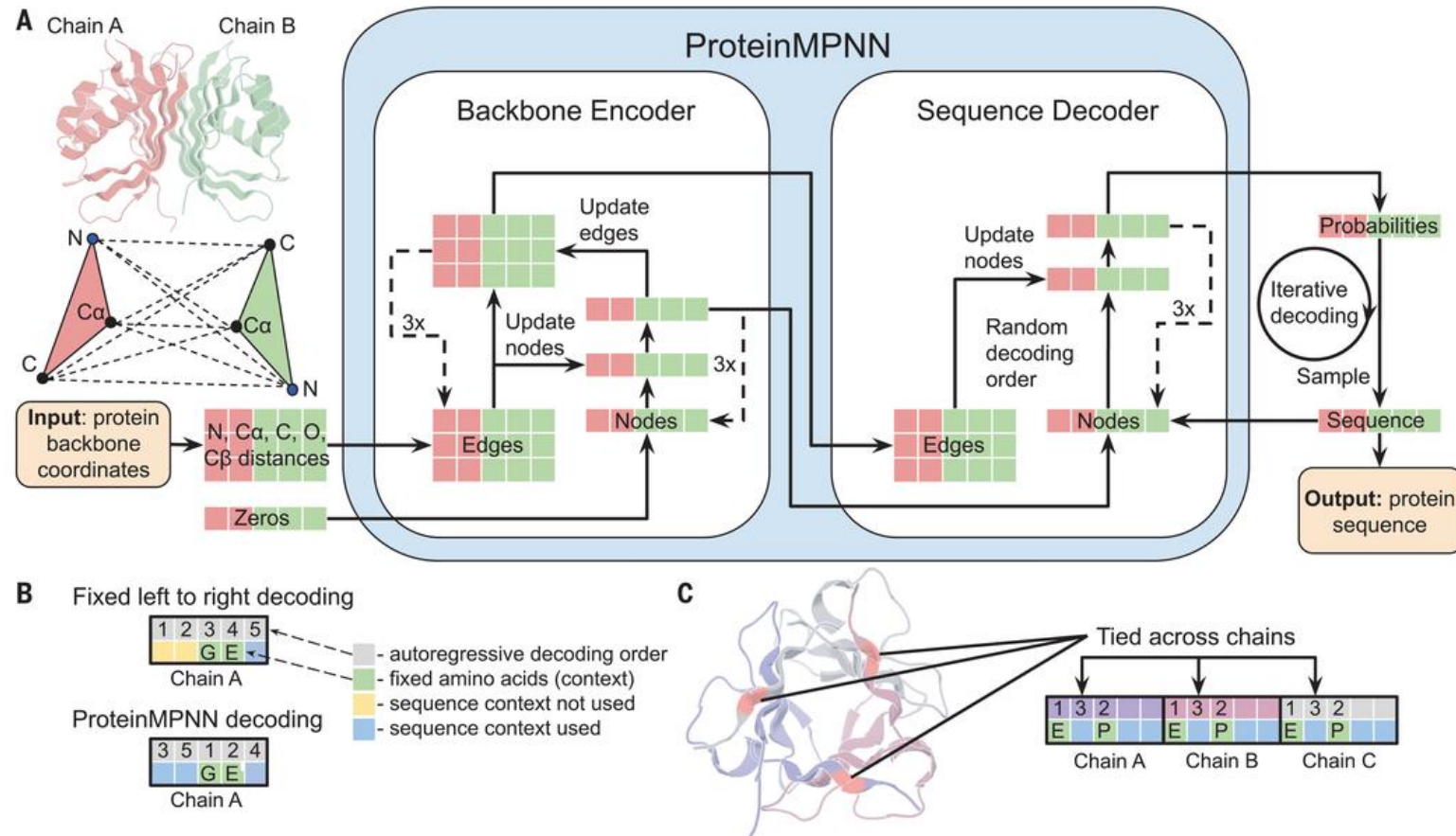
- Useful for modeling sequential data
 - e.g. amino acids in a protein sequence

$$p(\mathbf{x}) = p(x_1) \prod_{i=2}^L p(x_i | x_1, \dots, x_{i-1})$$

Model T-cell receptors

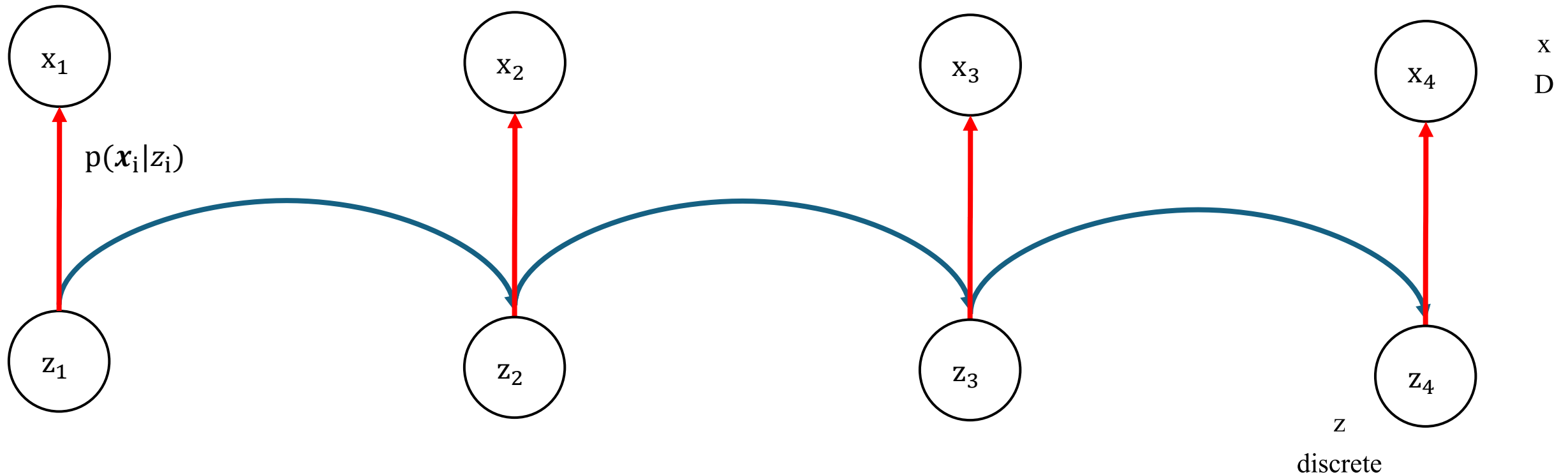


Generate sequence based on input fold



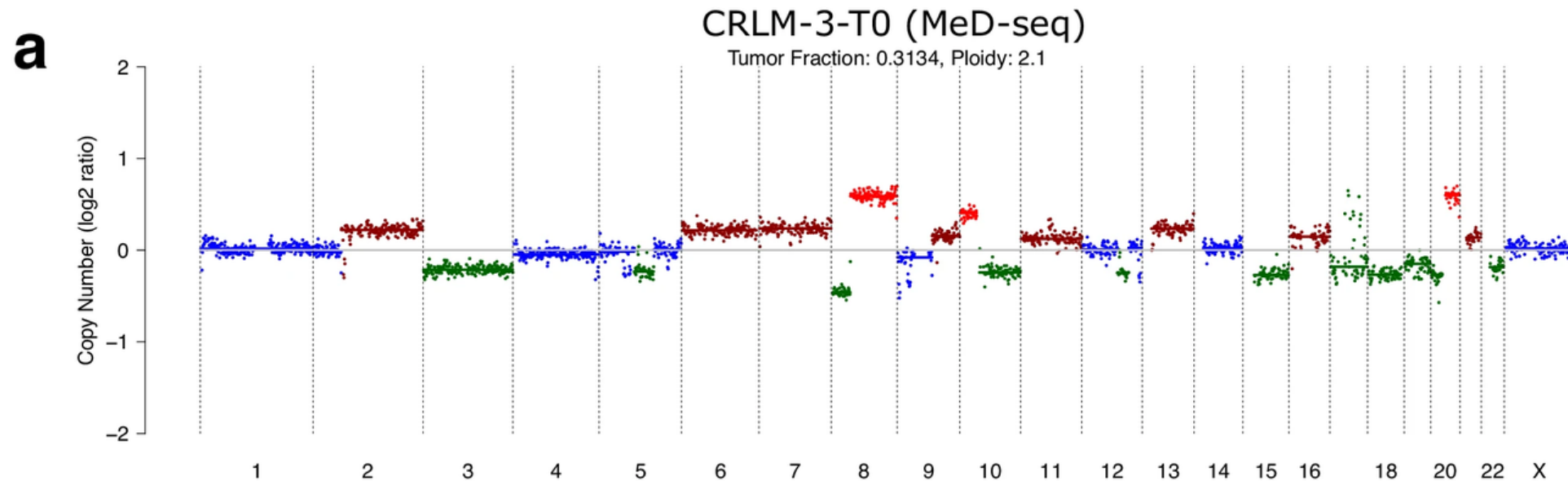
Hidden Markov Models

- $x \rightarrow$ observed sequence
- $z \rightarrow$ latent variables



Find tumor-derived aneuploidy in blood

- $x \rightarrow$ read count in a bin
- $z \rightarrow$ copy number of DNA in a bin



Overall summary

- Generative models model the entire data distribution $p(\mathbf{x})$
- Unsupervised models learn by maximum likelihood:
 - Closed form
 - E-M
 - ELBO
 - Change of variables
- After lunch:
 - Hands-on session, playing with RFdiffusion and proteinMPNN
 - Approximate ELBO for diffusion model

REFERENCES

Part 1

- Watson et al., Nature, 2023
- Lopez et al., Nat Methods, 2018
- Ferreira DaSilva et la., Nat Gen, 2025
- King et al., Science, 2026
- Theodoridis, Koutroumbas, “Pattern Recognition”, chapter 3
- Bishop, “Pattern recognition and machine learning”, chapters 9,12
- Argelaguet et al., Mol Sys Biol 2018

Part 2

- Kingma and Welling, [arXiv:1312.6114](#)
- Makrodimitris et al., Briefings in Bioinformatics, 2023
- Higgins et al., ICLR proceedings, 2017
- Burgess et al., arXiv:1804.03599
- Lopez et al., Nat Methods, 2018
- Luo, arXiv:2208.11970

Part 3

- Ho et al., NeurIPS, 2020
- Luo, arXiv:2208.11970
- Song et al., ICLR, 2021
- Ho et al., arXiv:2207.12598
- https://www.youtube.com/watch?v=iv-5mZ_9CPY&
- Watson et al., Nature, 2023
- Chen et al., arXiv:2208.04202

Part 4

- Dinh et al., arXiv:1605.08803
- Ho et al., NeurIPS, 2020
- Hyvarinen, JMLR, 2005
- Lipman et al., arXiv:2210.02747
- Jiang et al., Briefings In Bioinformatics, 2023
- Dauparas et al., Science, 2022
- Hazelaar et al., NPJ Prec Onco, 2025